

Alexandria University

Faculty of Engineering

Communications and Electronics Engineering Department

EdgeCore-V: Performance Evaluation of a RISC-V Processor with Different Memory Architectures on FPGA for Edge-Like Workloads

By:

Mennatullah Waleed Mohamed El-Sayed Ali

Ahmed Saeed Mohamed Abdel-Aal Mohamed

Mostafa Mohei El-Sayed Ibrahim Mostafa

Muhammad Abdelmonem Makram Bahgat Ismail

Ali Eslam Fathy Badr

Supervised By:

Prof. Mohamed Rizk

Alexandria University, July, 2026

Table of Contents

Table of Contents i

List of Figures vii

List of Tables x

1 Introduction 1
- 1.1 Motivation and Research Context . . . 1
- 1.2 Research Problem . . . 2
- 1.3 Research Gap . . . 3
- 1.4 Proposed Solution . . . 4
- 1.5 Project Contributions . . . 5
- 1.6 Thesis Organization . . . 5

2 Instruction Set Architecture 7
- 2.1 Overview of RISC-V . . . 7
  - 2.1.1 History and Motivation . . . 7
  - 2.1.2 Design Philosphy . . . 7
  - 2.1.3 Base ISA Variants and the Implemented ISA Prole . . . 8
- 2.2 Instruction Formats . . . 9
  - 2.2.1 Instruction Field Definitions . . . 9
  - 2.2.2 R-Type (Register-Register Operations) . . . 11
  - 2.2.3 I-Type (Immediate Operations and Loads) . . . 11
  - 2.2.4 S-Type (Store Operations) . . . 12
  - 2.2.5 B-Type (Branch Operations) . . . 12
  - 2.2.6 U-Type (Upper Immediate Operations) . . . 13
  - 2.2.7 J-Type (Jump Operations) . . . 13
- 2.3 RV64I Base Integer Instruction Set . . . 14
  - 2.3.1 Arithmetic and Logical Instructions . . . 14
  - 2.3.2 Load and Store Instructions . . . 15
  - 2.3.3 Branch and Jump Instructions . . . 17
  - 2.3.4 Upper Immediate Instructions . . . 18
- 2.4 Privilege Model, CSRs, and Exception-Handling Instructions . . . 20
  - 2.4.1 CSR Instruction Format . . . 20
  - 2.4.2 SYSTEM Instructions: ECALL, EBREAK, and MRET . . . 21
  - 2.4.3 Implemented CSR Registers . . . 21
- 2.5 M Extension: Integer Multiplication and Division . . . 22
- 2.6 MAC Extension (Custom) . . . 24

2.7 Summary . . . 25

3 Processor Architecture **26**
3.1 Design Overview . . . 26
3.2 Single-Cycle Implementation . . . 27
3.2.1 Instruction Fetch Unit . . . 28
3.2.2 Instruction Decode Unit . . . 28
3.2.3 Execution Unit . . . 29
3.2.4 Memory Access Unit . . . 29
3.2.5 Write-Back Unit . . . 30
3.2.6 Control Unit . . . 30
3.2.6.1 Main Decoder . . . 31
3.2.6.2 ALU Decoder . . . 32
3.3 From Single-Cycle to Pipelined: The Motivation . . . 33
3.4 5-Stage Pipeline Architecture . . . 36
3.4.1 Fetch Stage (IF) . . . 37
3.4.2 Decode Stage (ID) . . . 37
3.4.3 Execute Stage (EX) . . . 38
3.4.4 Memory Stage (MEM) . . . 38
3.4.5 Write-Back Stage (WB) . . . 39
3.5 Hazard Handling . . . 39
3.5.1 Data Hazards and Forwarding . . . 39
3.5.2 Control Hazards and Branch Handling . . . 40
3.5.3 Structural Hazards . . . 40
3.6 Branch Prediction . . . 40
3.6.1 Branch Prediction Techniques . . . 41
3.6.2 One-Bit Dynamic Predictor . . . 42
3.6.3 Two-Bit Saturating Predictor . . . 43
3.6.4 Selection of the Prediction Algorithm . . . 43
3.6.5 Branch Prediction Unit Architecture . . . 44
3.6.6 Branch Target Buffer . . . 45
3.6.7 Branch State Machine . . . 46
3.6.8 Prediction and Update Sequence . . . 47
3.6.9 Handling Unconditional Jumps . . . 48
3.7 Exception and CSR Subsystem . . . 49
3.7.1 Exception Sources and Precise Trap Priority . . . 49
3.7.2 Trap Entry, CSR Update, and Pipeline Flush . . . 50
3.7.3 Branch Misprediction and Illegal-Instruction Interaction . . . 50
3.7.4 Software Trap Handling . . . 51
3.8 M-Extension and MAC Instructions . . . 52
3.8.1 Multiplier Architecture . . . 52
3.8.1.1 Sign Handling . . . 52
3.8.1.2 Wallace Tree Structure . . . 53
3.8.1.3 Result Selection . . . 53
3.8.1.4 Why a Wallace Tree? . . . 54
Array Multiplier: . . . 54
Wallace Tree Multiplier: . . . 54

3.8.1.5 How the Wallace Tree Reduction Works . . . 55
Step 1 — Partial Product Generation. . . . 55
Step 2 — Carry-Save Reduction. . . . 56
Step 3 — Final Addition. . . . 56
Hierarchical Decomposition in this Design . . . 56
3.8.1.6 Division Unit . . . 56
Algorithm. . . . 56
Sign Handling. . . . 57
Word Operations. . . . 57
Corner Cases. . . . 57
FSM. . . . 57
3.8.1.7 Multi-Cycle Hazard and Stall Resolution . . . 58
Why Division Creates a Hazard. . . . 58
Detection. . . . 58
Stall Response. . . . 59
Interaction with Other Stall Sources. . . . 59
3.8.1.8 MAC Unit . . . 59
What is a MAC Operation? . . . 59
Why a Dedicated MAC Instruction? . . . 59
Instruction Encoding. . . . 60
Control Signals from the Main Decoder. . . . 60
Datapath Implementation inside the ICU. . . . 60
The Third-Register Hazard. . . . 61
Resolution via ForwardCE. . . . 61
3.9 Summary . . . 62

4 Memory Subsystem 64
4.1 Overview . . . 64
4.2 Memory Hierarchy Fundamentals . . . 65
4.2.1 The Locality Principle and the Memory Hierarchy . . . 65
4.2.2 Placement Policy: How a Block Maps to the Cache . . . 66
4.2.3 Replacement Policy: What to Evict on a Miss . . . 66
4.2.4 Write Policy: Handling Stores . . . 67
On a write hit: write-through vs. write-back. . . . 67
On a write miss: write-allocate vs. no-write-allocate. . . . 68
4.2.5 Hardware-Managed vs. Software-Managed Memory . . . 68
4.3 AXI Bus Interface . . . 68
4.3.1 AXI Channels and Signals . . . 69
Write Address Channel (AW) – master to slave. . . . 69
Write Data Channel (W) – master to slave. . . . 69
Write Response Channel (B) – slave to master. . . . 70
Read Address Channel (AR) – master to slave. . . . 70
Read Data Channel (R) – slave to master. . . . 70
The VALID/READY handshake rule. . . . 70
4.3.2 Burst Sizing in This Design . . . 70
4.3.3 Two-Level Arbitration . . . 71
4.4 Version 1: I-Cache with Direct AXI Memory Access . . . 71

4.4.1 I-Cache Architecture . . . 71
Tag array, data array, and valid bits. . . . 72
Replacement policy chosen: counter-based LRU. . . . 72
4.4.2 I-Cache Controller FSM . . . 73
4.4.3 Limitations of Version 1 . . . 73
4.5 Version 2: I-Cache and D-Cache with AXI . . . 74
4.5.1 D-Cache Architecture . . . 74
Write policy chosen: . . . 74
4.5.2 D-Cache Controller FSM . . . 74
4.5.3 Cache Arbitration . . . 75
Timeout-based bus fault detection. . . . 76
4.5.4 Limitations of Version 2 . . . 76
4.6 Version 3: Hybrid System — I-Cache, D-Cache, SPM, and DMA . . . 77
4.6.1 Scratchpad Memory (SPM) . . . 78
Why 8KB. . . . 78
Dual-port structure. . . . 78
4.6.2 CPU Memory-Request Handshake . . . 78
States. . . . 79
Why this belongs to the CPU side, not the target. . . . 79
4.6.3 DMA Engine . . . 79
Descriptor fields. . . . 80
Transfer modes and direction encoding. . . . 80
Configuration-error checking. . . . 80
Burst splitting for arbitrary transfer sizes. . . . 80
CPU/DMA concurrency. . . . 81
4.6.4 CPU/DMA Arbitration at the AXI Level . . . 81
4.6.5 SPM Port Arbitration: DMA Priority Over the CPU . . . 81
4.6.6 Memory Map . . . 81
4.6.7 Why the Hybrid Approach? . . . 82
4.7 Summary and Version Comparison . . . 82

5 Verification and Testing **84**
5.1 Verification Strategy . . . 84
5.2 RISC-V Assembly Test Programs . . . 85
5.2.1 Arithmetic, Logic, and Custom Instruction Tests . . . 85
Base ISA arithmetic/logic/shift (`RV64I.asm`). . . . 85
Custom MAC instruction (`mac.s`). . . . 87
5.2.2 Memory Access Tests . . . 87
D-cache line packing and hazard stress (`cache.s`). . . . 87
5.2.3 Hybrid Test . . . 88
Hybrid Caches+SPM round trip (`hybrid_test.asm`). . . . 88
5.2.4 Exception Handling Tests . . . 90
5.3 Summary . . . 92

6 Implementation Results **93**
6.0.1 Static Design Linting with SpyGlass . . . 93
6.1 Implementation Environment . . . 95

6.1.1 Target FPGA Platform . . . 95
6.1.2 Board I/O Interface . . . 96
6.1.2.1 Register File Observation via Switches and Seven-Segment Display . . . 97
6.1.3 Custom Synthesis and Implementation Flow . . . 97
Stage 0 — Environment setup. . . . 98
Stage 1 — Leaf modules. . . . 98
Stage 2 — Integer compute unit. . . . 98
Stage 3 — Exception handler. . . . 99
Stage 4 — Scratchpad memory, DMA, and arbitration. . . . 99
Stage 5 — Branch prediction. . . . 99
Stage 6 — Hazard unit. . . . 99
Stage 7 — Control path. . . . 99
Stage 8 — Datapath. . . . 99
Stage 9 — CPU top levels. . . . 99
Stage 10 — Full-hierarchy top synthesis. . . . 99
Stage 11 — Implementation. . . . 99
6.2 Static Timing Analysis . . . 100
6.2.1 Clock Generation Strategy . . . 100
6.2.1.1 Clocking Wizard (MMCM) versus a Hand-Written RTL Clock Divider . . . 100
6.2.1.2 MMCM versus PLL . . . 101
6.2.2 Final Timing Results . . . 102
6.3 Area Utilization . . . 102
6.3.1 Device-Level Summary . . . 103
6.3.2 Hierarchical Breakdown . . . 104
6.3.3 Discussion . . . 105
6.4 Power Analysis . . . 105
6.4.1 Dynamic Power . . . 106
6.4.2 Breakdown by Contributor . . . 107
6.4.3 Discussion . . . 107

7 Benchmarking Using MNIST Inference 109
7.1 Benchmarking Methodology . . . 109
7.2 MNIST Neural Network . . . 110
7.2.1 MNIST Dataset . . . 111
7.2.2 Neural Network Architecture . . . 112
7.2.3 Python Model Training . . . 113
7.2.4 INT8 Quantization . . . 114
7.3 Software Deployment . . . 115
7.3.1 C Implementation . . . 115
7.3.2 RISC-V Assembly Implementation . . . 116
7.3.3 Deployment on the Three Processor Configurations . . . 118
7.4 Performance Evaluation . . . 118
7.4.1 CPU Performance Metrics . . . 119
7.4.2 Instruction Cache Metrics . . . 119
7.4.3 Data Cache Metrics . . . 120

7.4.4 Branch Predictor Metrics . . . 122
7.4.5 System Bus Metrics . . . 123
7.4.6 Benchmarking Results . . . 124
7.4.6.1 Version 1: Baseline Processor . . . 124
7.4.6.2 Version 2: Processor with Data Cache . . . 125
7.4.6.3 Version 3: Processor with Data cache and SPM . . . 126
7.4.7 Instruction Count Analysis . . . 126
7.4.8 Cycle Count Analysis . . . 127
7.4.9 IPC and CPI Analysis . . . 128
7.4.10 Speedup Analysis . . . 129
7.4.11 Cache Performance Analysis . . . 130
7.4.12 Pipeline Stall Analysis . . . 131
7.4.13 Bus Utilization Analysis . . . 132
7.5 Summary . . . 134

8 Conclusion and Future Work 135
8.1 Summary of Work . . . 135
8.2 Conclusions . . . 136
8.3 Limitations . . . 137
8.4 Future Work . . . 138

Appendix 140

A Verification and Benchmarking Source Code 140
A.1 Verification Codes . . . 140
A.1.1 Instruction and Branch Prediction Test . . . 140
A.1.2 MAC Test . . . 143
A.1.3 Exception Handling Test . . . 143
A.1.4 Hybrid Memory Test . . . 145
A.1.5 Hello World Test . . . 147
A.2 Benchmarking Codes . . . 148
A.2.1 MNIST Training in Python . . . 148
A.2.2 MNIST Weight Quantization . . . 153
A.2.3 MNIST Inference in C . . . 158
A.2.4 MNIST Cache Behaviour and Direct Assembly Implementation . . . 162
A.2.5 MNIST Hybrid System Assembly Implementation . . . 168
A.2.6 Benchmark Analysis and Plotting Code . . . 176

List of Figures

3.1 Complete single-cycle RV64I datapath. 27

3.2 Complete Single-Cycle Control Unit 30

3.3 Five-stage pipelined RV64I. 36

3.4 One Bit Branch Predictors State Machine 42

3.5 Two Bit Branch Predictors State Machine 43

3.6 Pipeline with Branch Prediction Unit. 44

3.7 Branch Prediction Unit. 45

3.8 BTB Entries 46

3.9 Exception and CSR unit: detection sources from each pipeline stage feed the priority and CSR-update logic. 49

3.10 Simulation waveform before the fix: a wrong-path instruction raises `illegal_instr_flag` during a branch misprediction and incorrectly enables exception entry. . . 51

3.11 Simulation waveform after the fix during the misprediction window. 51

3.12 wallace_dot_reduction_8bit $P_{15} \dots P_0$. 55

4.1 I-Cache controller FSM 73

4.2 D-ache controller FSM 75

4.3 Caches AXI Bridge. 76

4.4 Full System Diagram 77

4.5 Finite state machine of the CPU memory-request handshake. 79

5.1 Setup Base Values, Section Of `RV64I.asm`. 85

5.2 Arithmetic Operations, Section Of `RV64I.asm`. 86

5.3 Logical operations, section of `RV64I.asm`. 86

5.4 Shift operations (32-bit and 64-bit), section of `RV64I.asm`. 86

5.5 Memory Test (Base address = 0x800), Section Of `RV64I.asm`. 86

5.6 Branch Tests Section Of `RV64I.asm`. 86

5.7 Jump Tests, Section Of `RV64I.asm`. 87

5.8 MAC Unsigned Tests, Section Of `MAC.asm`. 87

5.9 MAC Signed Tests, Section Of `MAC.asm`. 87

5.10 D-Cache Test Output, Section Of `caches.s`. 88

5.11 DMA init to load SPM, Section Of `hybrid_test.asm`. 89

5.12 SPM Load, Section Of `hybrid_test.asm`. 89

5.13 CPU reads From SPM, Section Of `hybrid_test.asm`. 90

5.14 CPU Writes to SPM, Section Of `hybrid_test.asm`. 90

5.15 DMA Init for SPM Writeback, Section Of `hybrid_test.asm`. 90

5.16 Data Transfer From SPM to Main Memory, Section Of `hybrid_test.asm`.. 90

5.17 `ecall` exception path, showing trap entry and CSR updates for `mcause`, `mepc`, and `mtval`. 91

5.18 Misaligned-load exception path, showing trap redirection and CSR updates for the faulting load address. 91

5.19 Illegal-instruction exception path, showing trap-handler entry and CSR updates for the invalid instruction. 92

5.20 Exception return after `mret`, showing control leaving the trap handler and resuming at the next PC. 92

6.1 Hazard Unit Before Linting. 94

6.2 Hazard Unit Before Linting. 95

6.3 Digilent Nexys 4 DDR development board (Xilinx Artix-7 XC7A100T), the target platform for EdgeCore-V. 95

6.4 General Purpose I/O devices on the Nexys4 DDR. 96

6.5 Timing Summary. 102

6.6 Utilization of RiscV. 103

6.7 Utilization of RiscV. 103

6.8 Power Summary. 105

7.1 Sample handwritten digits from the MNIST dataset. 111

7.2 Architecture of the fully connected neural network used for MNIST digit classification. 113

7.3 MNIST Python Model Training 114

7.4 MNIST Training Accuracy Graph 114

7.5 MNIST Weights Quantization 116

7.6 MNIST Quantized Accuracy 116

7.7 Diect Memory Result. 124

7.8 D-Cache Result. 125

7.9 Hybrid Result. 126

7.10 CPI and IPC comparison across the three EdgeCore-V configurations. . . 128

7.11 Speedup of each configuration relative to the Direct (V1) baseline. 129

7.12 Radar chart summarizing the relative performance of the three configurations across all evaluated metrics. 129

7.13 Stacked breakdown of pipeline stall cycles by cause, per configuration. . . 131

7.14 AXI bus utilization and throughput behaviour across the Direct, Cache-based, and Hybrid memory architectures. 133

List of Tables

2.1 RV64I Arithmetic and Logical Instructions 16

2.2 RV64I Load and Store Instructions 17

2.3 RV64I Branch and Jump Instructions 19

2.4 RV64I Upper Immediate Instructions 19

2.5 CSR instruction `funct3` encoding. 21

2.6 SYSTEM instruction encodings and behaviour. 21

2.7 Implemented Machine-mode CSR registers. 22

2.8 M-Extension R-Type Encoding 22

2.9 M-Extension Multiplication Instructions 23

2.10 M-Extension Division Instructions 23

2.11 Semantics for Division by Zero and Division Overflow 23

2.12 MAC Instruction Encoding 24

3.1 Main decoder control signal truth table. 31

3.2 ALU decoder truth table. 32

3.3 RAW data hazard without forwarding. 34

3.4 Load-use hazard requiring one pipeline stall. 35

3.5 Supported exception types and reported trap information. 50

3.6 Multiplier output selection by instruction. 53

3.7 MAC instruction encoding. 60

4.1 Memory map of the Hybrid (Version 3) configuration. 81

4.2 DMA control register offsets, relative to `0xFFFF_0000`. 82

4.3 Feature comparison across the three memory subsystem versions. 82

6.1 Design Timing Summary at 40 MHz 102

6.2 Post-route resource utilization summary on the XC7A100T-1CSG324C. . 104

6.3 Post-implementation on-chip power summary. 106

6.4 Dynamic power breakdown by contributor. 107

7.1 Dynamic instruction count per configuration. 126

7.2 Total execution cycles per configuration. 127

7.3 CPI and IPC per configuration. 128

7.4 Speedup of each configuration relative to the others. 129

7.5 Instruction- and data-cache hit rates per configuration. 130

7.6 Breakdown of pipeline stall cycles by cause and configuration. 131

7.7 DMA transfer activity for the Hybrid (V3) configuration. 133

Chapter 1

Introduction

1.1 Motivation and Research Context

Artificial Intelligence (AI) has become one of the defining technologies of modern computing, enabling intelligent decision-making across a wide range of application domains including healthcare, industrial automation, robotics, autonomous vehicles, smart surveillance, consumer electronics, and the Internet of Things (IoT). Rapid advances in machine learning, particularly deep neural networks, have significantly improved the capability of computers to perform complex tasks such as image classification, speech recognition, object detection, and natural language processing. As these techniques continue to mature, AI is increasingly transitioning from cloud-based infrastructures to embedded computing platforms capable of performing inference directly on the device.

Traditionally, AI inference has been executed on cloud servers equipped with powerful processors, large memory capacities, and virtually unlimited computational resources. In this computing paradigm, data generated by embedded devices is transmitted to remote servers where inference is performed before the results are returned to the originating device. Although cloud computing offers substantial computational capability, it also introduces several practical limitations. Communication with remote servers increases response latency, consumes network bandwidth, raises concerns regarding data privacy and security, and requires reliable network connectivity. These limitations become particularly significant in real-time applications such as autonomous robots, industrial control systems, drones, intelligent surveillance, and medical monitoring devices, where timely decision-making is essential and network availability cannot always be guaranteed.

These challenges have accelerated the adoption of *Edge AI*, a computing paradigm in which inference is performed locally on embedded devices rather than in centralized cloud infrastructure. By processing data closer to its source, Edge AI reduces communication

latency, minimizes bandwidth requirements, improves privacy by keeping sensitive information on-device, and enables autonomous operation even in disconnected environments. Consequently, Edge AI has emerged as one of the primary drivers of modern embedded processor development, motivating both academia and industry to design increasingly efficient computing platforms capable of executing machine learning workloads under strict resource constraints.

Unlike cloud servers, however, embedded systems operate under severe limitations in computational resources, power consumption, silicon area, memory capacity, and manufacturing cost. Many edge devices are battery-powered or deployed in thermally constrained environments where energy efficiency is equally as important as computational performance. These constraints fundamentally change the processor design process, requiring architects to carefully balance execution speed, hardware complexity, power consumption, and memory efficiency. Achieving high performance within these limitations has therefore become one of the central challenges in modern computer architecture.

1.2 Research Problem

As processor technology has advanced over the past several decades, improvements in computational performance have consistently outpaced improvements in memory access speed. While modern processors are capable of executing billions of instructions per second, retrieving data from external memory remains comparatively slow. This growing disparity, commonly referred to as the *memory wall*, has become one of the dominant factors limiting overall system performance. Rather than spending the majority of execution time performing useful computation, processors frequently stall while waiting for data to arrive from memory.

The impact of the memory wall is particularly pronounced in AI inference workloads. Neural networks repeatedly access large collections of model parameters, intermediate activations, and input data throughout the execution process. Although inference consists primarily of arithmetic operations such as multiplication and accumulation, the continuous movement of data between the processor and external memory often consumes more execution time and energy than the computation itself. Consequently, improving computational capability alone is insufficient to achieve high overall performance if the underlying memory subsystem cannot efficiently supply data to the processor.

To alleviate this bottleneck, modern processors employ hierarchical memory organizations that combine small, high-speed on-chip memories with larger off-chip memory systems. Different memory organizations attempt to reduce average memory access latency through different mechanisms, each offering distinct advantages and disadvantages with

respect to hardware complexity, predictability, silicon utilization, software overhead, and overall execution performance. Selecting an appropriate memory architecture therefore represents a fundamental design decision that directly influences the efficiency of embedded computing systems, particularly those intended for data-intensive workloads such as machine learning inference.

The importance of memory hierarchy design continues to increase as AI models become larger and embedded applications demand higher levels of performance under increasingly restrictive hardware budgets. Consequently, understanding the architectural trade-offs associated with different memory subsystem organizations has become essential for the design of modern embedded processors.

1.3 Research Gap

The emergence of the RISC-V instruction set architecture has significantly expanded opportunities for processor research, education, and open-source hardware development. Its open and modular nature allows researchers to develop custom processors without the licensing restrictions associated with proprietary architectures, making RISC-V an attractive platform for architectural exploration. As a result, numerous RISC-V processor implementations have been proposed, ranging from lightweight microcontrollers to high-performance application processors targeting embedded and general-purpose computing.

Despite the rapid growth of the RISC-V ecosystem, comparatively little work has focused on systematically evaluating alternative memory subsystem organizations while maintaining an identical processor architecture. Existing studies typically emphasize pipeline design, instruction-set implementation, branch prediction, or peripheral integration, whereas memory hierarchy evaluations are often performed across different processors, implementation technologies, benchmark suites, or experimental methodologies. Such variations make it difficult to isolate the true impact of the memory subsystem itself, since observed performance differences may result from unrelated architectural changes rather than from the memory organization being investigated.

Furthermore, many published processor implementations evaluate performance primarily through simulation or synthetic benchmarks, providing limited insight into the practical implications of different memory hierarchies when implemented on real hardware. For embedded AI applications, where memory behavior frequently dominates execution performance, controlled experimental evaluation of alternative memory organizations is particularly valuable.

This thesis addresses this gap by developing a single RISC-V processor platform capable of supporting multiple memory subsystem architectures while maintaining an identical processor pipeline, instruction set, implementation platform, benchmark applications, and evaluation methodology. By isolating the memory subsystem as the primary experimental variable, the proposed platform enables a fair and systematic comparison of different architectural approaches, providing quantitative insight into their impact on processor performance, hardware utilization, and overall system efficiency.

1.4 Proposed Solution

To address the challenges identified in the previous sections, this thesis presents **EdgeCore-V**, a synthesizable 64-bit RISC-V processor developed as an experimental platform for investigating the impact of memory hierarchy design on embedded AI workloads. Rather than proposing a new instruction set architecture or an application-specific accelerator, the primary objective of this work is to provide a controlled framework for evaluating how different memory subsystem organizations influence processor performance, hardware utilization, and overall system efficiency.

The proposed processor implements the RV64IM instruction set and incorporates a custom Multiply-Accumulate (MAC) instruction to accelerate arithmetic operations commonly encountered in machine learning and digital signal processing applications. The processor architecture remains unchanged throughout this work, allowing the memory subsystem to be evaluated independently from the execution pipeline.

Three progressively more sophisticated memory subsystem architectures are developed and implemented on the same processor core. The first configuration establishes a baseline system by employing an instruction cache while servicing all data accesses directly from external memory. The second configuration extends the baseline through the addition of a hardware-managed data cache, reducing memory access latency by exploiting data locality. Finally, the third configuration introduces a hybrid memory architecture that combines conventional caching with a software-managed Scratchpad Memory (SPM) and a Direct Memory Access (DMA) engine, enabling efficient and predictable movement of data between external and on-chip memory.

By maintaining an identical processor architecture, implementation platform, software stack, and benchmark suite across all three configurations, this work enables a fair comparison in which performance differences can be attributed solely to the organization of the memory subsystem. The evaluation is conducted using both hardware implementation results and application-level benchmarking, providing quantitative insight into the trade-offs between different memory hierarchy techniques for embedded AI applications.

1.5 Project Contributions

The principal contributions of this thesis are summarized as follows:

- Design and implementation of **EdgeCore-V**, a synthesizable 64-bit five-stage pipelined RISC-V processor supporting the RV64IM instruction set together with a custom Multiply-Accumulate (MAC) instruction for accelerating arithmetic-intensive workloads.
- Development of three interchangeable memory subsystem architectures built upon a common processor core, enabling controlled evaluation of different memory hierarchy organizations without modifying the processor pipeline.
- Design and implementation of a configurable memory subsystem incorporating instruction caching, data caching, software-managed Scratchpad Memory (SPM), and a Direct Memory Access (DMA) engine for efficient data movement.
- Development of a complete hardware verification framework, including unit-level RTL verification, processor integration testing, and comprehensive RISC-V assembly test programs to validate functional correctness.
- FPGA implementation and validation of the complete processor on the Xilinx development platform, demonstrating the practical feasibility of the proposed architecture.
- Development of an end-to-end benchmarking framework based on a handwritten RISC-V assembly implementation of an INT8-quantized MNIST neural network inference application.
- Integration of a hardware Performance Monitoring Unit (PMU) capable of collecting detailed execution statistics, including cycle counts, cache behavior, instruction throughput, and pipeline stall information, enabling quantitative architectural analysis.
- Comprehensive experimental evaluation comparing the three memory subsystem architectures in terms of execution performance, memory behavior, FPGA resource utilization, timing performance, and overall architectural trade-offs.

1.6 Thesis Organization

The remainder of this thesis is organized as follows.

Chapter 2 presents the background required for this work, introducing the RISC-V instruction set architecture together with the RV64I base instruction set, the standard

M extension, and the custom Multiply-Accumulate (MAC) instruction implemented in the proposed processor.

Chapter 3 describes the overall architecture of EdgeCore-V, including the processor datapath, pipeline organization, control logic, hazard handling mechanisms, and execution units.

Chapter 4 presents the design of the memory subsystem and describes the three memory hierarchy configurations investigated in this thesis. The chapter discusses the architectural design and implementation of the cache subsystem, Scratchpad Memory (SPM), DMA controller, and memory interface.

Chapter 5 details the verification methodology adopted throughout the development process, including unit-level verification, processor validation, and assembly-based functional testing.

Chapter 6 presents the FPGA implementation results, including synthesis reports, timing analysis, hardware resource utilization, and power estimation for the proposed processor.

Chapter 7 evaluates the performance of the three memory subsystem architectures using application-level benchmarks and hardware Performance Monitoring Unit (PMU) statistics, providing a quantitative comparison of their performance and architectural trade-offs.

Chapter 8 concludes the thesis by summarizing the main achievements of this work, discussing its limitations, and outlining directions for future research.

Chapter 2

Instruction Set Architecture

2.1 Overview of RISC-V

2.1.1 History and Motivation

RISC-V (pronounced *"risk five"*) is an open, royalty-free Instruction Set Architecture (ISA) developed at UC Berkeley in 2010 by Andrew Waterman, Yunsup Lee, David Patterson, and Krste Asanović. The project arose because existing ISAs were either proprietary and licensed (like ARM and x86) or academic designs ill-suited for real hardware implementation. The Berkeley team required a single ISA that could offer academic flexibility, industrial viability, and long-term architectural stability.

While its name follows earlier Berkeley RISC research projects, RISC-V was uniquely designed from the outset to be a practical, complete ISA capable of scaling from embedded microcontrollers to large-scale server processors. Today, the ISA is governed by the non-profit organization RISC-V International, ensuring it remains open and standardized. Its ratified base and extension specifications are now widely adopted across global academia, industry, and national research programs.

2.1.2 Design Philosphy

RISC-V is built on the Reduced Instruction Set Computing (RISC) philosophy, which emphasizes simple, regular, and fixed-length instructions. Unlike the Complex Instruction Set Computing (CISC) approach typified by x86, which encodes complex operations into single instructions, RISC-V's regularity simplifies instruction decoding and reduces control logic complexity, enabling efficient pipelining and predictable latency.

A distinguishing feature of RISC-V is its modularity. The ISA consists of a mandatory base integer instruction set and optional standard extensions, preventing the "ISA bloat"

common in mature commercial architectures. This allows designers to tailor implementations precisely to specific workloads. Standard extensions are denoted by letter suffixes: M for integer multiplication and division, F and D for single- and double-precision floating-point, and C for 16-bit compressed instructions to improve code density.

Furthermore, RISC-V deliberately avoids mandating microarchitectural choices such as pipeline depth, cache organization, or branch predictors. This freedom allows implementers to explore a diverse design space—ranging from ultra-low-power, in-order single-issue pipelines to high-performance, out-of-order superscalar cores. This flexibility is highly advantageous for specialized applications, such as the edge AI inference workloads targeted in this work, which demand strict optimizations for area, power, and computational throughput.

2.1.3 Base ISA Variants and the Implemented ISA Prole

This thesis implements the following ISA profile:

`RV64I + M + MAC (custom)`

This profile was selected through deliberate co-design of the ISA and the target workload characteristics. The three components serve distinct roles:

- **RV64I** forms the mandatory base. The 64-bit register width and address space are essential for edge AI inference: large model parameter sets, activation buffers, and intermediate feature maps may collectively exceed the 4 GiB addressable range of a 32-bit architecture. The 64-bit width also simplifies pointer arithmetic in software runtimes and ensures compatibility with 64-bit Linux-based inference frameworks such as TensorFlow Lite and ONNX Runtime.

- **M extension** supplies hardware integer multiplication and division. Neural network inference workloads, particularly quantised 8-bit integer (INT8) models, rely on frequent multiply operations for weight-activation products. Without hardware multiply support, these reduce to costly software routines that dominate execution time. The `M` extension eliminates this bottleneck and provides the integer multiply infrastructure upon which the MAC extension builds.

- **MAC extension** is a custom non-standard extension developed in this work, encoded in the `custom-0` opcode space. It introduces a fused Multiply-Accumulate instruction that performs $acc \leftarrow acc + (rs1 \times rs2)$ in a single instruction cycle. This operation is the fundamental compute primitive in convolutional and fully-connected neural network layers. By reducing a three-operation sequence (LOAD, MUL, ADD) to a single instruction, the MAC extension reduces instruction fetch pressure, shortens inner loop iteration counts, and enables dedicated hardware

accumulation paths. The design and encoding of this extension are detailed in Section 2.5.

Together, this profile defines an ISA that is fully binary-compatible with the standard RV64IM software ecosystem,any program compiled for `RV64IM` runs unmodified, while providing an additional hardware acceleration layer that can be exploited by a compiler or hand-written assembly kernel targeting edge AI inference. The remainder of this chapter describes each component of this profile in detail.

2.2 Instruction Formats

All RISC-V instructions in the base integer ISA are encoded in a fixed 32-bit word. Fixed-length encoding is a deliberate design choice: it simplifies instruction fetch (every aligned 32-bit word is a complete instruction), eliminates the variable-length decode logic required by architectures such as x86, and enables straightforward pipelining with predictable fetch-decode timing. The RISC-V specification defines six canonical 32-bit instruction formats, each of which partitions the 32 available bits into fields according to the type of operation being encoded[1].

A key design principle shared across all six formats is that certain fields are *always* placed at the same bit positions regardless of format type. Specifically, the opcode field always occupies bits [6:0], the destination register (`rd`) always occupies bits [11:7] when present, the first source register (`rs1`) always occupies bits [19:15], and the second source register (`rs2`) always occupies bits [24:20]. This positional consistency means that the register file read ports can be addressed in parallel with opcode decode, reducing the critical path through the decode stage and simplifying control logic. The following subsections describe each format in detail.

2.2.1 Instruction Field Definitions

Before examining each format individually, it is useful to define the standard fields that appear across all six formats. Each field occupies a fixed bit range within the 32-bit instruction word, and the hardware decode logic exploits this consistency to read multiple fields simultaneously.

- `opcode` **[6:0]**: The primary opcode field, present in every instruction format. Its 7 bits identify the broad instruction class, such as integer register-register operations, immediate operations, loads, stores, branches, or jumps. Not all 128 possible opcode values are used; the specification reserves certain encodings for future standard extensions and, critically, dedicates four encodings (`custom-0` through `custom-3`) to non-standard implementer-defined instructions. The least significant two bits of

a valid 32-bit base instruction are always `11`, which is how a fetch unit distinguishes 32-bit instructions from 16-bit compressed (`C` extension) encodings.

- `rd` **[11:7]**: The destination register field. When present, its 5-bit value indexes one of the 32 general-purpose integer registers (`x0`–`x31`) that will receive the instruction result. Store (S-type) and branch (B-type) instructions do not write to a register and therefore do not carry an `rd` field; the bits [11:7] are instead repurposed as part of the immediate encoding in those formats.

- `funct3` **[14:12]**: A 3-bit secondary function code that refines the operation within an opcode class. For example, within the branch opcode class, `funct3` selects among `BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, and `BGEU`. Within the load opcode class, it selects the access width and sign behaviour (`LB`, `LH`, `LW`, `LD`, `LBU`, `LHU`, `LWU`). The field is present in R, I, S, and B-type formats and is absent in U and J-type, where the upper bits encode a wider immediate instead.

- `funct7` **[31:25]**: A 7-bit tertiary function code used in R-type instructions to further disambiguate operations that share the same opcode and `funct3`. The most common use is bit 30 of `funct7`, which distinguishes `ADD` from `SUB` and logical right shift (`SRL`) from arithmetic right shift (`SRA`). For M-extension multiply and divide instructions, a fixed `funct7` value of `0000001` is used across all operations, with `funct3` selecting the specific instruction. In I-type shift instructions, bits [31:26] of the instruction word serve a role analogous to `funct7`, distinguishing `SRLI` from `SRAI`.

- `rs1` **[19:15]**: The first source register field. Its 5-bit value indexes the general-purpose register that provides the first read operand: the base address for loads and stores, the left-hand operand for arithmetic and logical operations, the comparand for branches, and the base address for `JALR`. This field is present in R, I, S, and B-type formats.

- `rs2` **[24:20]**: The second source register field. Its 5-bit value indexes the register providing the second read operand: the right-hand operand for register-register arithmetic, the data word to be written for stores, and the second comparand for branches. This field is present in R, S, and B-type formats. In I-type and U-type formats, these bits are absorbed into the immediate field.

- `imm`: The immediate field carries a compile-time constant embedded directly in the instruction word. Unlike the register fields, the immediate does not occupy a single contiguous bit range across all formats; its width, position, and bit ordering vary by format type (12 bits in I and S-type, 13 bits in B-type, 20 bits in U-type, and 21 bits in J-type). In all cases, the hardware sign-extends the immediate to the

full 64-bit datapath width before use, with the sign bit always sourced from bit 31 of the instruction word. The specific bit mappings for each format are described in the subsections that follow.

2.2.2 R-Type (Register-Register Operations)

The R-type format encodes arithmetic, logical, and shift instructions that operate entirely on register operands, with no immediate value. It is used by all RV64I register-register instructions as well as the integer multiply and divide instructions of the M extension.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
`funct7`	`rs2`	`rs1`	`funct3`	`rd`	`opcode`
7 bits	*5 bits*	*5 bits*	*3 bits*	*5 bits*	*7 bits*

The opcode field (bits [6:0]) identifies the broad instruction class. The `funct3` field (bits [14:12]) and the `funct7` field (bits [31:25]) together distinguish individual instructions within the same opcode class. For example, `ADD` and `SUB` share the same opcode and `funct3` value but are differentiated by bit 30 of `funct7` (0 for `ADD`, 1 for `SUB`). This two-level function encoding maximises the number of distinct operations encodable within a single opcode class without dedicating excessive bits to the opcode field alone. The five source and destination register fields (`rs1`, `rs2`, `rd`) each encode one of the 32 general-purpose registers using a 5-bit index. Representative R-type instructions include `ADD`, `SUB`, `AND`, `OR`, `XOR`, `SLL`, `SRL`, and `SRA`, as well as the M-extension instructions `MUL`, `MULH`, `DIV`, and `REM`.

2.2.3 I-Type (Immediate Operations and Loads)

The I-type format encodes instructions that combine a register operand with a 12-bit signed immediate value. It is used for immediate arithmetic and logical operations, load instructions, and the `JALR` control-flow instruction.

[31:20]	[19:15]	[14:12]	[11:7]	[6:0]
`imm[11:0]`	`rs1`	`funct3`	`rd`	`opcode`
12 bits	*5 bits*	*3 bits*	*5 bits*	*7 bits*

The 12-bit immediate is sign-extended to the full 64-bit register width before use, yielding a signed range of $[-2048, +2047]$. For load instructions, the immediate encodes the byte offset from the base address in `rs1`; for immediate arithmetic instructions such as `ADDI`, it is the second operand. For `JALR`, it encodes the offset added to `rs1` to compute the

jump target. Note that for shift instructions in RV64I (`SLLI`, `SRLI`, `SRAI`), only the lower 6 bits of the immediate field encode the shift amount; the upper bits serve as a secondary function code that distinguishes logical from arithmetic right shifts.

2.2.4 S-Type (Store Operations)

The S-type format is used exclusively for store instructions, which write a register value to a computed memory address. Unlike I-type, S-type has two source registers (`rs1` as the base address, `rs2` as the data to be stored) and no destination register. To maintain positional consistency of `rs1` and `rs2` at bits [19:15] and [24:20] respectively, the 12-bit store offset immediate is split across two non-contiguous fields.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
`imm[11:5]`	`rs2`	`rs1`	`funct3`	`imm[4:0]`	`opcode`
7 bits	*5 bits*	*5 bits*	*3 bits*	*5 bits*	*7 bits*

The 12-bit immediate is reconstructed by concatenating `imm[11:5]` (bits [31:25]) and `imm[4:0]` (bits [11:7]) in the decode stage, then sign-extending the result to 64 bits. The split arises directly from keeping `rs2` at bits [24:20] for decode consistency: placing the full immediate in the upper bits would have displaced the second source register. The `funct3` field encodes the access width: `SB` (store byte), `SH` (store halfword), `SW` (store word), and `SD` (store doubleword, RV64I only).

2.2.5 B-Type (Branch Operations)

The B-type format encodes conditional branch instructions. It is closely related to S-type in field layout, but the immediate encodes a PC-relative branch offset rather than a memory address offset, and the immediate bits are rearranged to share as much decode hardware as possible with S-type.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
`imm[12|10:5]`	`rs2`	`rs1`	`funct3`	`imm[4:1|11]`	`opcode`
7 bits	*5 bits*	*5 bits*	*3 bits*	*5 bits*	*7 bits*

The branch target is computed as PC + imm, where the 13-bit signed immediate (bit 0 is always implicitly zero, since all targets must be 2-byte aligned) is assembled from the scattered fields as:

$$\text{imm} = \{\,\text{inst}[31],\ \text{inst}[7],\ \text{inst}[30{:}25],\ \text{inst}[11{:}8],\ 1'\text{b0}\,\} \tag{2.1}$$

The sign bit is always sourced from bit 31. The effective branch offset range is $[-4096,\ +4094]$ bytes in steps of 2. The `funct3` field encodes the branch condition: `BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, and `BGEU`.

2.2.6 U-Type (Upper Immediate Operations)

The U-type format encodes instructions that place a 20-bit immediate into the upper bits of a destination register. Only two instructions use this format: `LUI` (Load Upper Immediate) and `AUIPC` (Add Upper Immediate to PC).

[31:12]	[11:7]	[6:0]
`imm[31:12]`	`rd`	`opcode`
20 bits	*5 bits*	*7 bits*

The 20-bit immediate occupies bits [31:12] of the instruction word directly and is placed into bits [31:12] of the destination register, with bits [11:0] zeroed and the result sign-extended to 64 bits. `LUI` writes this shifted immediate to `rd` without adding any base value. `AUIPC` instead adds the shifted immediate to the current PC before writing to `rd`. Together, these two instructions form the canonical two-instruction idiom for materialising arbitrary 32-bit constants or PC-relative addresses, combined with a subsequent I-type instruction (for example, `LUI` followed by `ADDI` constructs any 32-bit signed immediate in exactly two instructions).

2.2.7 J-Type (Jump Operations)

The J-type format is used by the single unconditional jump instruction `JAL` (Jump and Link). It encodes a 21-bit PC-relative offset, allowing a jump to a target within a ± 1 MiB range relative to the current instruction address.

[31:12]	[11:7]	[6:0]
`imm[20|10:1|11|19:12]`	`rd`	`opcode`
20 bits (scrambled)	*5 bits*	*7 bits*

As with B-type, the immediate bits are non-contiguously placed to maximise hardware sharing with U-type decode. The 21-bit signed jump offset is reassembled as:

$$\text{imm} = \{\,\text{inst}[31],\ \text{inst}[19{:}12],\ \text{inst}[20],\ \text{inst}[30{:}21],\ 1'\text{b0}\,\} \tag{2.2}$$

The least significant bit is implicitly zero, encoding only 2-byte-aligned targets. `JAL` writes the address of the instruction immediately following the jump (i.e., PC + 4) into `rd`, which serves as the return address in the standard function call convention. When `rd` is `x0`, the return address is discarded and the instruction acts as a plain unconditional branch.

2.3 RV64I Base Integer Instruction Set

The RV64I base integer instruction set forms the mandatory foundation of the processor implemented in this thesis. It defines 47 instructions covering integer arithmetic, logical operations, memory access, control flow, and environment interaction. Every instruction is 32 bits wide and follows one of the six formats described in Section 2.2. The following subsections group these instructions by functional category, describing their semantics, encoding, and relevance to the implemented microarchitecture.

2.3.1 Arithmetic and Logical Instructions

Arithmetic and logical instructions perform integer computations entirely within the register file, with no memory interaction. They divide into two groups: register-register instructions, which use R-type encoding and take both operands from registers, and register-immediate instructions, which use I-type encoding and combine one register operand with a sign-extended 12-bit immediate.

Integer Arithmetic

The core arithmetic instructions are `ADD` and `SUB` for register-register addition and subtraction, and `ADDI` for register-immediate addition. In RV64I, these operate on the full 64-bit register width. The `ADDIW`, `ADDW`, and `SUBW` variants operate on the lower 32 bits of their operands and sign-extend the 32-bit result to 64 bits, which is essential for maintaining compatibility with software that uses 32-bit integer types on a 64-bit machine. There is no `SUBI` instruction; subtracting an immediate is achieved by negating the constant and using `ADDI`, which avoids duplicating opcode space. Overflow is not trapped in hardware; it wraps silently modulo 2^{64}, and detection is left to software where required.

Logical Instructions

The logical instructions `AND`, `OR`, and `XOR` perform bitwise operations on two register operands (R-type), with immediate variants `ANDI`, `ORI`, and `XORI` available in I-type

encoding. A useful idiom arising from the immediate variants is `XORI rd, rs1, -1` (XOR with all-ones immediate), which performs a bitwise NOT on `rs1` and places the result in `rd`, eliminating the need for a dedicated NOT instruction.

Shift Instructions

RISC-V provides three shift operations: shift left logical (`SLL`), shift right logical (`SRL`), and shift right arithmetic (`SRA`). Left shifts fill vacated low bits with zeros. Logical right shifts fill vacated high bits with zeros. Arithmetic right shifts replicate the sign bit into vacated high bits, which is equivalent to a signed integer division by a power of two. Each has both a register-register form (R-type, shift amount from the lower 6 bits of `rs2` in RV64I) and an immediate form (`SLLI`, `SRLI`, `SRAI`, I-type, shift amount encoded in bits [25:20] of the instruction). Word-width variants (`SLLW`, `SRLW`, `SRAW` and their immediate counterparts) operate on 32-bit values and sign-extend the result, using only the lower 5 bits of the shift amount.

Comparison Instructions

Set-if-Less-Than (`SLT`) and its unsigned counterpart (`SLTU`) write 1 to `rd` if `rs1` is less than `rs2` (using signed and unsigned comparison respectively), and 0 otherwise. Immediate variants `SLTI` and `SLTIU` compare `rs1` against a sign-extended 12-bit immediate. A notable idiom is `SLTIU rd, rs1, 1`, which sets `rd` to 1 if `rs1` equals zero (and 0 otherwise), implementing a logical NOT without a dedicated instruction.

2.3.2 Load and Store Instructions

RISC-V is a load-store architecture: all data processing instructions operate exclusively on register operands, and memory is accessed only through explicit load and store instructions. This clean separation simplifies the pipeline design, as the memory access stage is only ever active for load and store instructions, and the ALU need never interact with memory directly.

Addressing Mode

RISC-V supports a single memory addressing mode: *base plus offset*. The effective address is computed as:

$$\mathrm{EA} = \mathrm{rs1} + \mathrm{sext}(\mathrm{imm}) \tag{2.3}$$

where `rs1` holds the base address and `imm` is a signed 12-bit immediate encoding the byte offset. This single addressing mode keeps the address generation unit (AGU) simple:

Table 2.1: RV64I Arithmetic and Logical Instructions

Instruction	Format	Operation	Notes
`ADD rd, rs1, rs2`	R	`rd = rs1 + rs2`	64-bit
`SUB rd, rs1, rs2`	R	`rd = rs1 - rs2`	64-bit
`ADDI rd, rs1, imm`	I	`rd = rs1 + sext(imm)`	No `SUBI`
`ADDW rd, rs1, rs2`	R	`rd = sext(rs1[31:0] + rs2[31:0])`	32-bit, sign-extended
`SUBW rd, rs1, rs2`	R	`rd = sext(rs1[31:0] - rs2[31:0])`	32-bit, sign-extended
`ADDIW rd, rs1, imm`	I	`rd = sext(rs1[31:0] + sext(imm))`	32-bit, sign-extended
`AND rd, rs1, rs2`	R	`rd = rs1 & rs2`	Bitwise AND
`OR rd, rs1, rs2`	R	`rd = rs1 | rs2`	Bitwise OR
`XOR rd, rs1, rs2`	R	`rd = rs1 ^ rs2`	Bitwise XOR
`ANDI rd, rs1, imm`	I	`rd = rs1 & sext(imm)`	
`ORI rd, rs1, imm`	I	`rd = rs1 | sext(imm)`	
`XORI rd, rs1, imm`	I	`rd = rs1 ^ sext(imm)`	`imm=-1` gives NOT
`SLL rd, rs1, rs2`	R	`rd = rs1 « rs2[5:0]`	Logical left
`SRL rd, rs1, rs2`	R	`rd = rs1 » rs2[5:0]`	Logical right
`SRA rd, rs1, rs2`	R	`rd = rs1 »> rs2[5:0]`	Arithmetic right
`SLLI rd, rs1, shamt`	I	`rd = rs1 « shamt`	`shamt` in [25:20]
`SRLI rd, rs1, shamt`	I	`rd = rs1 » shamt`	Logical
`SRAI rd, rs1, shamt`	I	`rd = rs1 »> shamt`	Arithmetic
`SLT rd, rs1, rs2`	R	`rd = (rs1 < rs2) ? 1 : 0`	Signed
`SLTU rd, rs1, rs2`	R	`rd = (rs1 <u rs2) ? 1 : 0`	Unsigned
`SLTI rd, rs1, imm`	I	`rd = (rs1 < sext(imm)) ? 1 : 0`	Signed
`SLTIU rd, rs1, imm`	I	`rd = (rs1 <u sext(imm)) ? 1 : 0`	Unsigned

one adder and one sign extender, with no need for scaled-index, pre-decrement, or post-increment modes found in more complex ISAs. The 12-bit offset covers a $[-2048, +2047]$ byte range from the base register, which is sufficient for structure field access and stack frame indexing in the vast majority of compiled code.

Load Instructions

Load instructions use I-type encoding. They read a value from the computed effective address in memory and write the result to the destination register `rd`, sign-extending or zero-extending as specified. RV64I provides loads of five widths: byte (8-bit), halfword (16-bit), word (32-bit), doubleword (64-bit), and unsigned variants for the narrower widths. Signed loads (`LB`, `LH`, `LW`) sign-extend the loaded value to 64 bits; unsigned loads

(`LBU`, `LHU`, `LWU`) zero-extend instead. The doubleword load `LD` always fills the full 64-bit register.

Unaligned memory accesses are permitted by the RISC-V specification but may be handled in hardware or trapped to a software handler depending on the implementation. The processor implemented in this thesis requires naturally aligned accesses (i.e., an n-byte load must be issued to an address that is a multiple of n), consistent with the constraints of the memory architectures evaluated in later chapters.

Store Instructions

Store instructions use S-type encoding. They write the value in `rs2` to the effective address computed from `rs1` and the split 12-bit immediate. The width of the write is again selected by `funct3`: `SB` writes the least significant byte, `SH` the least significant halfword, `SW` the least significant word, and `SD` all 64 bits. Note that stores have no destination register and produce no register result; their only architectural effect is the memory write.

Table 2.2: RV64I Load and Store Instructions

Instruction	Format	Operation	Notes
`LB rd, imm(rs1)`	I	`rd = sext(Mem[rs1+imm][7:0])`	Load byte, signed
`LH rd, imm(rs1)`	I	`rd = sext(Mem[rs1+imm][15:0])`	Load halfword, signed
`LW rd, imm(rs1)`	I	`rd = sext(Mem[rs1+imm][31:0])`	Load word, signed
`LD rd, imm(rs1)`	I	`rd = Mem[rs1+imm][63:0]`	Load doubleword
`LBU rd, imm(rs1)`	I	`rd = zext(Mem[rs1+imm][7:0])`	Load byte, unsigned
`LHU rd, imm(rs1)`	I	`rd = zext(Mem[rs1+imm][15:0])`	Load halfword, unsigned
`LWU rd, imm(rs1)`	I	`rd = zext(Mem[rs1+imm][31:0])`	Load word, unsigned
`SB rs2, imm(rs1)`	S	`Mem[rs1+imm][7:0] = rs2[7:0]`	Store byte
`SH rs2, imm(rs1)`	S	`Mem[rs1+imm][15:0] = rs2[15:0]`	Store halfword
`SW rs2, imm(rs1)`	S	`Mem[rs1+imm][31:0] = rs2[31:0]`	Store word
`SD rs2, imm(rs1)`	S	`Mem[rs1+imm][63:0] = rs2[63:0]`	Store doubleword

2.3.3 Branch and Jump Instructions

Control flow instructions alter the program counter from its default sequential increment of 4 bytes. RISC-V provides two categories: conditional branches, which transfer control only when a tested condition is true, and unconditional jumps, which always redirect the PC.

Conditional Branches

All six conditional branch instructions use B-type encoding and compute a PC-relative target address. The processor compares two source registers (`rs1` and `rs2`) and, if the condition is satisfied, sets the PC to PC + sext(imm); otherwise execution continues at PC + 4. The branch offset is a 13-bit signed value, giving a reach of ±4 KiB from the branch instruction. For targets beyond this range, a compiler generates a negated short branch over an unconditional `JAL` to the distant target.

The six branch conditions cover signed and unsigned less-than and equality comparisons. Signed greater-than and greater-than-or-equal comparisons are obtained by swapping the operand registers of `BLT` and `BGE` respectively, so no additional opcodes are needed. Similarly, branch-if-zero and branch-if-nonzero idioms use `BEQ` or `BNE` with `x0` as one operand.

Unconditional Jumps

`JAL` (Jump and Link) uses J-type encoding and encodes a 21-bit PC-relative offset, giving a reach of ±1 MiB. It unconditionally sets the PC to PC + sext(imm) and writes PC + 4 (the return address) to `rd`. Using `rd = ra` (x1) implements a function call; using `rd = x0` discards the return address and produces a plain unconditional branch with a 1 MiB range.

`JALR` (Jump and Link Register) uses I-type encoding and computes its target as (rs1 + sext(imm)) & ∼1, where the final AND clears the least significant bit to enforce 2-byte alignment. It writes PC + 4 to `rd`. `JALR` is the key instruction for indirect control flow: function returns are implemented as `JALR x0, ra, 0` (jump to the address in the return-address register, discarding the new return address), and calls through function pointers use `JALR ra, rs1, 0`.

Together, `JAL` and `JALR` cover the four canonical control-flow patterns in compiled code: direct calls (`JAL ra, offset`), indirect calls (`JALR ra, rs1, 0`), returns (`JALR x0, ra, 0`), and long-range unconditional branches (`JAL x0, offset`).

2.3.4 Upper Immediate Instructions

Upper Immediate Instructions

The two U-type instructions `LUI` and `AUIPC` exist to solve a practical encoding constraint: the I-type immediate is only 12 bits wide, which is insufficient to directly load an arbitrary 32-bit constant or construct a PC-relative address to an arbitrary location in the address space.

Table 2.3: RV64I Branch and Jump Instructions

Instruction	Format	Operation	Notes
`BEQ rs1, rs2, imm`	B	Branch if `rs1 == rs2`	Signed/unsigned same
`BNE rs1, rs2, imm`	B	Branch if `rs1 != rs2`	Signed/unsigned same
`BLT rs1, rs2, imm`	B	Branch if `rs1 < rs2`	Signed comparison
`BGE rs1, rs2, imm`	B	Branch if `rs1 >= rs2`	Signed comparison
`BLTU rs1, rs2, imm`	B	Branch if `rs1` $<_u$ `rs2`	Unsigned comparison
`BGEU rs1, rs2, imm`	B	Branch if `rs1` $\geq_u$ `rs2`	Unsigned comparison
`JAL rd, imm`	J	`rd = PC+4; PC = PC+sext(imm)`	±1 MiB range
`JALR rd, rs1, imm`	I	`rd = PC+4; PC = (rs1+imm)&~1`	Indirect jump

`LUI` (Load Upper Immediate) writes a 20-bit immediate into bits [31:12] of `rd`, zeroing bits [11:0] and sign-extending the result to 64 bits. It is the first instruction of the standard two-instruction sequence for loading a 32-bit constant: `LUI` sets the upper 20 bits, and a subsequent `ADDI` adds the lower 12 bits. Because `ADDI` sign- extends its 12-bit immediate, the upper 20-bit value loaded by `LUI` must be adjusted by +1 whenever the lower 12-bit immediate has its sign bit set (bit 11 set), to compensate for the sign extension that would otherwise corrupt the upper portion of the intended constant.

`AUIPC` (Add Upper Immediate to PC) adds the shifted 20-bit immediate to the current PC and writes the result to `rd`. This produces a PC-relative base address that is within ±2 GiB of the instruction's own address. Combined with a subsequent I-type instruction such as `ADDI` or a load, `AUIPC` enables position-independent addressing of any location in the virtual address space reachable within a ±2 GiB window. The pair `AUIPC` + `JALR` is also the standard long-range function call sequence when the target is beyond the ±1 MiB reach of `JAL`.

Table 2.4: RV64I Upper Immediate Instructions

Instruction	Format	Operation	Notes
`LUI rd, imm`	U	`rd = sext(imm « 12)`	Bits [11:0] zeroed
`AUIPC rd, imm`	U	`rd = PC + sext(imm « 12)`	PC-relative base

2.4 Privilege Model, CSRs, and Exception-Handling Instructions

The processor implements a machine-mode–only privilege model, which is the minimal privileged execution environment defined by RISC-V. Machine mode (M-mode) has unrestricted access to processor state, memory-mapped resources, and exception-control registers, and it is the first mode entered after reset. This model is sufficient for the targeted bare-metal edge-AI runtime because firmware can initialise the hardware, run inference kernels, and handle traps without requiring supervisor or user modes.

Two instruction groups support this privilege model. Control and Status Register (CSR) instructions read and modify per-hart architectural state, while SYSTEM instructions (`ECALL`, `EBREAK`, and `MRET`) enter or leave the trap-handling flow. The following subsections describe their encoding, behaviour, and the implemented Machine-mode CSRs.

2.4.1 CSR Instruction Format

All six CSR instructions share the `SYSTEM` opcode (`0x73`) and use the I-type instruction format, differentiated from one another solely by the `funct3` field.

[31:20]	[19:15]	[14:12]	[11:7]	[6:0]
`csr[11:0]`	`rs1 / uimm`	`funct3`	`rd`	`0x73`
12 bits	*5 bits*	*3 bits*	*5 bits*	*7 bits*

The field definitions are as follows. `csr` (bits [31:20]) is the 12-bit address of the target CSR. `rs1 / uimm` (bits [19:15]) provides either the source register (register variants) or a 5-bit zero-extended immediate (immediate variants). `funct3` (bits [14:12]) selects the operation and operand source. `rd` (bits [11:7]) receives the *old* value of the CSR before any modification; if `rd = x0` the read is suppressed.

The `funct3` encoding is summarised in Table 2.5. Bit 2 of `funct3` selects between a register source (`rs1`) and a zero-extended 5-bit immediate (`uimm`); bits [1:0] select the operation.

Two special cases affect write behaviour. For `CSRRS`/`CSRRSI` and `CSRRC`/`CSRRCI`, if the source operand is zero (i.e. `rs1 = x0` or `uimm = 0`), no write to the CSR is performed and the instruction acts as a pure read. `CSRRW`/`CSRRWI` always perform the write unconditionally.

TABLE 2.5: CSR instruction `funct3` encoding.

Instruction	funct3	Imm bit [2]	Op bits [1:0]	Operation
CSRRW	001	0 (reg)	01(write)	CSR ← rs1; rd ← old CSR
CSRRS	010	0 (reg)	10(set)	CSR ← CSR | rs1
CSRRC	011	0 (reg)	11(clear)	CSR ← CSR & ~rs1
CSRRWI	101	1 (imm)	01(write)	CSR ← uimm
CSRRSI	110	1 (imm)	10(set)	CSR ← CSR | uimm
CSRRCI	111	1 (imm)	11(clear)	CSR ← CSR & ~uimm

2.4.2 SYSTEM Instructions: ECALL, EBREAK, and MRET

`ECALL`, `EBREAK`, and `MRET` all share the `SYSTEM` opcode and `funct3 = 000` with `rs1 = 0` and `rd = 0`. They are distinguished exclusively by the 12-bit `funct12` field in bits [31:20], making comparison of the full field mandatory in the decoder.

[31:20] funct12	[19:15]	[14:12]	[11:7]	[6:0]
000000000000 (ECALL)	00000	000	00000	0x73
000000000001 (EBREAK)	00000	000	00000	0x73
001100000010 (MRET)	00000	000	00000	0x73

TABLE 2.6: SYSTEM instruction encodings and behaviour.

Instruction	funct12	Format	Behaviour
ECALL	0x000	I	Raises an environment-call exception. mcause ← 11 (M-mode call); mepc ← PC of ECALL; PC ← mtvec.
EBREAK	0x001	I	Raises a breakpoint exception. mcause ← 3; mepc ← PC of EBREAK; PC ← mtvec.
MRET	0x302	I	Returns from M-mode trap. PC ← mepc; pipeline is flushed.

2.4.3 Implemented CSR Registers

Only the minimum set of Machine-mode CSRs required for exception handling is implemented. Each register is written automatically on exception entry, writable by CSR

instructions, and readable by software at any time.

Table 2.7: Implemented Machine-mode CSR registers.

Register	Address	Purpose
`MISA`	`0x301`	Read-only. Reports the supported ISA extensions and base architecture width (`RV64IM` + custom MAC).
`MTVEC`	`0x305`	Trap-handler base address. The PC is redirected here whenever any exception is taken.
`MEPC`	`0x341`	Stores the PC of the instruction that caused the trap, allowing execution to resume via `MRET`.
`MCAUSE`	`0x342`	Encodes the cause of the most recent trap (see Table 3.5 in Chapter 3).
`MTVAL`	`0x343`	Holds exception-specific auxiliary information: the faulting address for memory exceptions, or the instruction bits for an illegal-instruction fault.

The exception and CSR subsystem is described in full in Chapter 3 (Section 3.7).

2.5 M Extension: Integer Multiplication and Division

The RISC-V M extension introduces standard integer multiplication and division instructions, encoded in the R-type format using `funct7 = 0000001` to distinguish them from the base integer instruction set. All M-extension instructions share the same general encoding structure shown in Table 2.8.

Table 2.8: M-Extension R-Type Encoding

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
funct7	rs2	rs1	funct3	rd	opcode
`0000001`	src2	src1	varies	dest	`0110011`/`0111011`

The opcode `0110011` (OP) is used for 64-bit operations, while `0111011` (OP-32) is used for 32-bit word operations that sign-extend their result into the 64-bit register.

Multiplication Instructions

Four multiplication instructions are provided, differing in operand signedness and which half of the 128-bit product is written to `rd`:

TABLE 2.9: M-Extension Multiplication Instructions

Instruction	funct3	Operation	Description
`MUL rd, rs1, rs2`	`000`	`rd = (rs1 × rs2)`$_{63:0}$	Signed × Signed, lower 64 bits
`MULH rd, rs1, rs2`	`001`	`rd = (rs1 × rs2)`$_{127:64}$	Signed × Signed, upper 64 bits
`MULHU rd, rs1, rs2`	`011`	`rd = (rs1 × rs2)`$_{127:64}$	Unsigned × Unsigned, upper 64 bits
`MULHSU rd, rs1, rs2`	`010`	`rd = (rs1 × rs2)`$_{127:64}$	Signed × Unsigned, upper 64 bits

`MUL` returns the lower 64 bits of the full product, which is identical regardless of whether the operands are treated as signed or unsigned. `MULH`, `MULHU`, and `MULHSU` return the upper 64 bits and are distinguished by operand signedness, allowing software to recover the full 128-bit result when needed.

The 32-bit word variant `MULW` (funct3 = `000`, opcode = `0111011`) multiplies the lower 32 bits of `rs1` and `rs2` and sign-extends the lower 32 bits of the product into `rd`.

In the EdgeCore-V implementation, all multiplication operations complete in a **single clock cycle** using a Wallace tree reduction network, avoiding pipeline stalls for this class of instruction.

Division and Remainder Instructions

Four division instructions are defined, with signed and unsigned variants for both quotient and remainder:

TABLE 2.10: M-Extension Division Instructions

Instruction	funct3	Operation	Description
`DIV rd, rs1, rs2`	`100`	`rd = rs1 / rs2`	Signed division
`DIVU rd, rs1, rs2`	`101`	`rd = rs1 / rs2`	Unsigned division
`REM rd, rs1, rs2`	`110`	`rd = rs1 % rs2`	Signed remainder
`REMU rd, rs1, rs2`	`111`	`rd = rs1 % rs2`	Unsigned remainder

The 32-bit word variants `DIVW`, `DIVUW`, `REMW`, and `REMUW` operate on the lower 32 bits of both source registers and sign-extend the result into the full 64-bit destination register.

The RISC-V specification defines explicit semantics for exceptional cases to avoid undefined behavior:

TABLE 2.11: Semantics for Division by Zero and Division Overflow

Condition	Dividend	Divisor	DIVU	DIVUW	REMU[W]	DIV	DIVW	REM[W]
Division by zero	x	0	$2^{64}-1$	$2^{32}-1$	x	-1	-1	x
Overflow (signed only)	-2^{63}	-1	—	—	—	-2^{63}	-2^{31}	0

Signed overflow occurs only when the most-negative integer (-2^{63}) is divided by -1; in that case the quotient is defined as the dividend and the remainder as zero. Division by zero never raises a hardware exception in RISC-V; instead, the results above are returned by the hardware, and trap behavior is left to software.

In EdgeCore-V, division uses a restoring algorithm that completes in **64 clock cycles**, stalling the pipeline until the result is ready.

2.6 MAC Extension (Custom)

The Multiply-Accumulate (MAC) operation computes $rd \mathrel{+}= rs1 \times rs2$ in a single instruction, combining a multiplication with an addition into the destination register. This operation is fundamental to dot-product and convolution computations that dominate neural network inference workloads.

EdgeCore-V introduces a custom MAC instruction as a non-standard extension, using a reserved opcode outside the standard RISC-V encoding space to avoid conflicts with future standard extensions.

MAC Instruction Encoding

The MAC instruction follows the R-type format:

Table 2.12: MAC Instruction Encoding

31:25	**24:20**	**19:15**	**14:12**	**11:7**	**6:0**
funct7	rs2	rs1	funct3	rd	opcode
`0000000` / `0000001`	src2	src1	`000`	dest	`0001011`

The `funct7` field is used to distinguish between the unsigned and signed variants of the MAC instruction, both of which share the same opcode (`0001011`) and `funct3` (`000`):

- `funct7 = 0000000` → **Unsigned MAC**: `src1` and `src2` are treated as unsigned operands.
- `funct7 = 0000001` → **Signed MAC**: `src1` and `src2` are treated as signed (two's complement) operands.

The custom opcode `0001011` is in the reserved custom-0 opcode space defined by the RISC-V specification, ensuring the encoding does not conflict with any ratified or draft standard extension.

MAC Operation

The MAC instruction performs:

$$rd \leftarrow rd + (rs1 \times rs2) \tag{2.4}$$

The interpretation of `rs1` and `rs2` depends on the `funct7` field selected at encoding time:

- If `funct7 = 0000000` (**unsigned MAC**), `rs1` and `rs2` are treated as **unsigned 64-bit integers**, and the multiplication is performed using the unsigned datapath of the Wallace tree multiplier.
- If `funct7 = 0000001` (**signed MAC**), `rs1` and `rs2` are treated as **signed (two's complement) 64-bit integers**, and the multiplier's sign-extension logic is enabled to correctly compute the signed product.

In both cases, the lower 64 bits of the resulting product are added to the current value of `rd`, and the result is written back to `rd`. This two-source, accumulate-into-`rd` semantics allows iterative dot products to be expressed as a tight loop with no separate accumulation instruction, regardless of operand signedness.

Assembler Syntax

```
mac  rd, rs1, rs2      # signed:   rd = rd + (rs1 * rs2)[63:0]
macu rd, rs1, rs2      # unsigned: rd = rd + (rs1 * rs2)[63:0]
```

Like the standard `MUL` instruction, the MAC instruction completes in a **single clock cycle** by reusing the Wallace tree multiplier and appending a final addition stage before the writeback. The assembler selects the appropriate `funct7` encoding (`0000001` for `mac`, `0000000` for `macu`) based on the mnemonic used.

2.7 Summary

This chapter presented the instruction set architecture implemented by the proposed processor, beginning with the design principles and modular structure of RISC-V and the selected RV64IM ISA profile. It described the six standard instruction formats, the organization of the RV64I base integer instruction set, the machine-mode privilege model and CSR instructions, and the standard M extension for hardware multiplication and division. Finally, the chapter introduced the custom MAC extension developed in this work, which provides single-cycle multiply–accumulate instructions to accelerate neural-network inference. Together, these ISA features establish the software-visible architecture upon which the processor microarchitecture described in the following chapter is built.

Chapter 3

Processor Architecture

3.1 Design Overview

Building a processor from the ground up is fundamentally an exercise in managing complexity. The temptation in any hardware project is to reach immediately for the most optimised design, but doing so collapses the verification problem: if a feature-complete processor does not run a program correctly, there is no principled way to isolate the fault. The architecture of this processor therefore developed as a sequence of deliberate, verifiable steps, each one building directly on the last.

The journey begins with a single-cycle RV64I core: every instruction completes in exactly one clock cycle, the datapath is acyclic, and correctness can be confirmed instruction by instruction. That foundation is then restructured into a five-stage pipeline to improve throughput, at the cost of introducing hazards that must be resolved explicitly. A forwarding and hazard-detection unit eliminates most pipeline stalls, while a two-bit dynamic branch predictor reduces the penalty of control-flow changes. An exception and CSR subsystem then adds the minimum machine-mode support required for a compliant RISC-V hart, handling illegal instructions, misaligned accesses, environment calls, and breakpoints. Finally, the M-extension multiplier/divider and a fused Multiply-Accumulate (MAC) unit are integrated, completing the integer execution core before any specific memory system is attached.

The guiding principle throughout is *correctness first, performance second.* Every architectural addition was verified in simulation against the previous baseline before being committed, so the chapter that follows can trace not only what each component does, but why it was designed the way it was.

3.2 Single-Cycle Implementation

The single-cycle core constitutes the foundational implementation of the RV64I processor presented in this thesis. It provides a complete implementation of the RV64I base integer instruction set while maintaining a straightforward datapath and control structure. Since every instruction is fetched, decoded, executed, accesses memory if required, and writes back its result within a single clock cycle, the design contains no pipeline registers, forwarding logic, or hazard handling mechanisms. Instead, the datapath forms a single combinational path from instruction fetch to architectural state update. The processor adopts a Harvard memory architecture, employing separate instruction and data memories to allow independent instruction fetch and data access while eliminating structural conflicts between them. The simplicity of the single-cycle organization makes the processor easier to design, analyze, and functionally verify, while also providing a natural architectural baseline for the more advanced pipelined implementations developed later in this chapter.

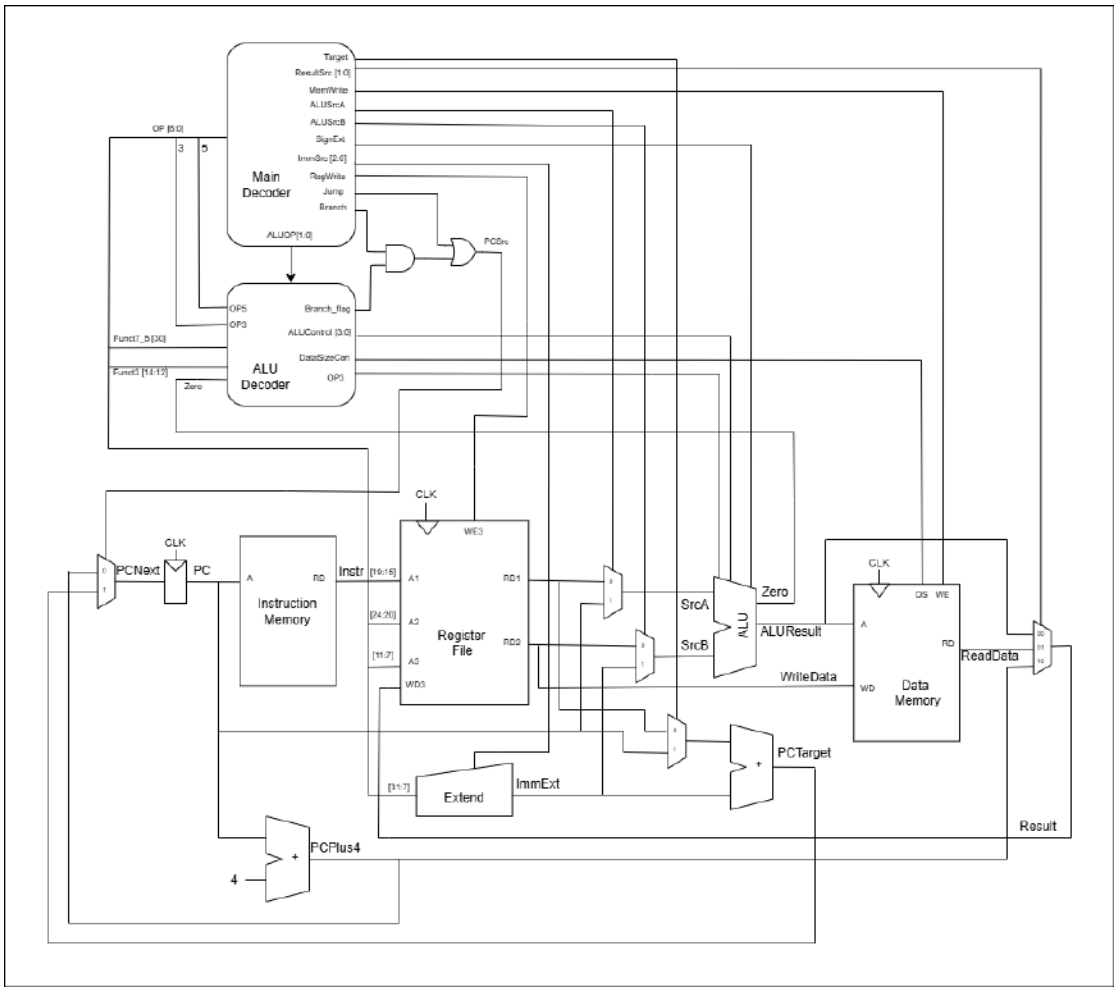


FIGURE 3.1: Complete single-cycle RV64I datapath.

The design implements the complete RV64I base integer instruction set across all six instruction formats (R, I, S, B, U, J). The datapath is divided into five functional blocks that mirror the classical RISC pipeline stages, but here they operate in a single unbroken combinational chain within each clock cycle.

3.2.1 Instruction Fetch Unit

The Instruction Fetch unit is the entry point of the datapath. It is built around a 64-bit Program Counter (PC) register that holds the address of the instruction currently being executed. On every rising clock edge the PC is updated; in the absence of any control-flow instruction the update is simply $PC = PC + 4$, since all RISC-V instructions are fixed at 32 bits and therefore occupy four bytes.

For control-flow instructions the update is more involved. A branch instruction computes a target address as PC + SignExt(imm) and replaces the normal PC + 4 path when the branch condition is satisfied. JAL computes its target the same way but is unconditional. JALR adds a register value to a 12-bit immediate, masking the least-significant bit to zero to guarantee alignment. A priority multiplexer in front of the PC register selects among these candidates based on the control signals generated by the control unit.

The instruction memory is modelled as a synchronous read-only memory addressed by the current PC. Its output, the 32-bit instruction word, feeds every downstream block simultaneously.

3.2.2 Instruction Decode Unit

The Instruction Decode unit extracts the information encoded in the 32-bit instruction word and presents it to the rest of the datapath in a form that is convenient to use.

The **register file** holds 32 general-purpose registers, each 64 bits wide, numbered `x0`–`x31`. Register `x0` is hardwired to zero: any write to it is silently discarded, and any read from it returns zero unconditionally. The register file provides two independent read ports — addressed by the `rs1` and `rs2` fields of the instruction — and one write port, addressed by `rd` and gated by the `RegWrite` control signal. In the single-cycle design the write happens at the end of the same clock cycle in which the instruction is fetched, so the register file is written and read within the same cycle; the write-before-read convention on the write port means that a read of the same address as a concurrent write correctly returns the new value.

The ImmSrc[2:0] signal generated by the main decoder selects the appropriate immediate reconstruction, supporting I-type (12-bit signed), S-type (12-bit split), B-type (13-bit signed branch offset, aligned to 2-byte boundaries), U-type (20-bit upper immediate

placed in bits [31:12] with the lower 12 bits cleared), and J-type (21-bit signed jump offset, aligned to 2-byte boundaries).

3.2.3 Execution Unit

The Execution unit is where the instruction's semantics are realised arithmetically. It contains the ALU and its auxiliary control logic.

The ALU accepts two 64-bit operands. The first operand is sourced from `rs1` or from the PC (selected by `ALUSrcA`), to support AUIPC which adds the upper immediate to the current instruction address. The second operand is sourced from `rs2` or from the sign-extended immediate (selected by `ALUSrcB`), covering both register-register and register-immediate instructions.

The supported operations include addition, subtraction, bitwise AND/OR/XOR, logical and arithmetic shifts (both 64-bit and 32-bit word variants controlled by `OP3` and `SignExtension`), signed and unsigned less-than comparisons (SLT, SLTU), and the LUI pass-through. Two outputs emerge: `ALUResult`, which carries the computed value, and `Zero`, a single-bit flag used by the branch logic.

The ALU decoder sits between the main decoder and the ALU itself. It receives `ALUOp[1:0]` from the main decoder together with `funct3`, `funct7[5]`, and `OP[5]` from the instruction word, and produces the four-bit `ALUControl` signal. It also evaluates branch conditions: for each branch type it inspects the `Zero` flag (or its complement) and asserts `BranchFlag` when the condition is met. This evaluation is local and combinational in the single-cycle design, which works correctly here but will need to be restructured when the design is pipelined, since the ALU result will then not be available until the Execute stage.

3.2.4 Memory Access Unit

The Memory Access unit handles all interaction with the data memory. In the single-cycle design this is a simple synchronous memory whose address input is driven by `ALUResult` and whose write-data input is driven by `rs2`. The `MemWrite` control signal gates write operations; `DataSizeCon[2:0]` (derived from `funct3`) selects the transfer width — byte, half-word, word, or double-word — and controls sign extension on loads. For store instructions the memory is written and the result path is irrelevant (`ResultSrc` is set to don't-care). For load instructions the memory output is the value that will be written back to the register file.

3.2.5 Write-Back Unit

The Write-Back unit is the final selector in the data path. A multiplexer controlled by `ResultSrc[1:0]` chooses among three candidates for the value to be written into `rd`:

- `ALUResult` — for arithmetic, logical, shift, and address-computation instructions.
- Memory read data — for load instructions.
- PC + 4 — for JAL and JALR, which store the return address in `rd`.

This result is presented to the register file's write port simultaneously with the `rd` address and the `RegWrite` enable signal.

3.2.6 Control Unit

The control unit is responsible for decoding each instruction and generating the control signals required to configure the datapath. As shown in Figure 3.2, the control logic is implemented as two combinational blocks: the *Main Decoder* and the *ALU Decoder*. Separating the control logic into these two stages simplifies the decoder design by assigning instruction-level control to the main decoder while delegating arithmetic and logical operation selection to the ALU decoder.

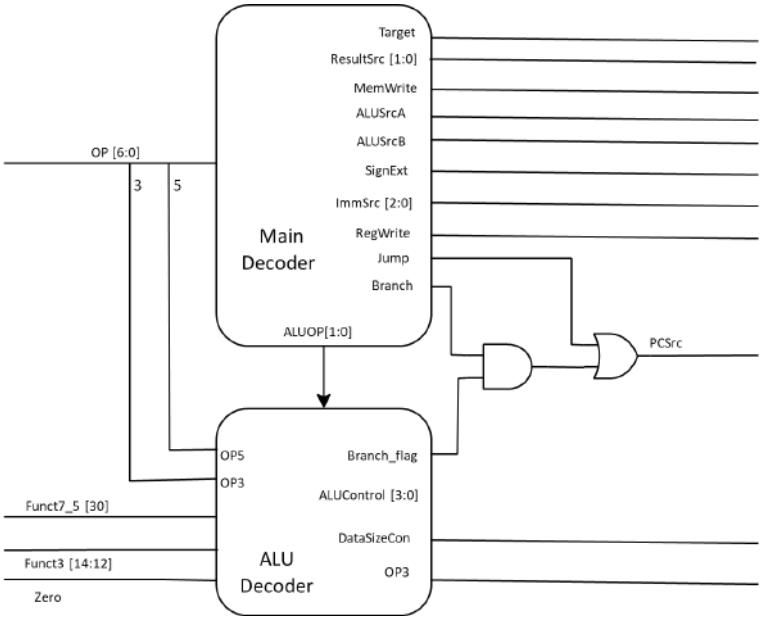


FIGURE 3.2: Complete Single-Cycle Control Unit

3.2.6.1 Main Decoder

The Main Decoder receives the instruction opcode (`OP[6:0]`) and produces the primary control signals that configure the datapath. These include `RegWrite`, `MemWrite`, `ResultSrc[1:0]`, `ALUSrcA`, `ALUSrcB`, `ImmSrc[2:0]`, `SignExt`, `Target`, `Jump`, `Branch`, and the two-bit `ALUOp` signal.

The `ImmSrc` signal selects the immediate format corresponding to the instruction type, while `ResultSrc` selects the value written back to the register file. The `ALUSrcA` and `ALUSrcB` signals determine the two operands supplied to the ALU, enabling register-register operations, immediate arithmetic, address calculation, and PC-relative instructions.

Rather than directly generating the ALU control signals, the Main Decoder produces the intermediate `ALUOp` field. This reduces the complexity of the opcode decoder by separating instruction decoding from ALU operation selection. Table **??** summarizes the complete Main Decoder truth table.

TABLE 3.1: Main decoder control signal truth table.

Instruction	RW	Imm	SrcA	SrcB	MW	Res	Br	ALU	Jmp	Sgn	Tgt
LW	1	000	0	1	0	01	0	00	0	0	1
SW	0	001	0	1	1	xx	0	00	0	0	1
R-Type	1	xxx	0	0	0	00	0	10	0	0	1
BEQ/B-Type	0	010	0	0	0	xx	1	01	0	0	1
I-Type ALU	1	000	0	1	0	00	0	10	0	0	1
Shift Imm.	1	100	0	1	0	00	0	10	0	0	1
I-Type Word	1	000	0	1	0	00	0	10	0	1	1
Shift Word	1	100	0	1	0	00	0	10	0	1	1
R-Type Word	1	xxx	0	0	0	00	0	10	0	1	1
JAL	1	011	x	x	0	10	0	xx	1	0	1
JALR	1	000	x	x	0	10	0	xx	1	0	0
AUIPC	1	101	1	1	0	00	0	00	0	0	1
LUI	1	101	1	1	0	00	0	11	0	0	1

Note: `x` denotes a don't-care value. Header abbreviations are RW = RegWrite, MW = MemWrite, Res = ResultSrc, Br = Branch, ALU = ALUOp, Jmp = Jump, Sgn = SignExt, and Tgt = Target.

3.2.6.2 ALU Decoder

The ALU Decoder receives the `ALUOp` field from the Main Decoder together with the instruction's `funct3`, `funct7[5]`, and `OP[5]` bits. Using these inputs, it generates the four-bit `ALUControl` signal that selects the required ALU operation.

For arithmetic and logical instructions, the ALU Decoder distinguishes between operations such as addition, subtraction, logical AND, OR, XOR, comparisons, and logical or arithmetic shifts. The decoder also supports the dedicated control required for instructions such as `LUI`, whose result is formed without performing a conventional arithmetic operation.

For branch instructions, the ALU Decoder selects the appropriate comparison operation according to the branch type. The resulting comparison flag is then combined with the `Branch` signal generated by the Main Decoder to determine whether the branch condition has been satisfied. Jump instructions assert the `Jump` signal, causing the Program Counter to load the computed target address unconditionally. The final Program Counter select signal (`PCSrc`) is therefore generated from the combination of the `Branch`, `Jump`, and ALU comparison signals, as illustrated in Figure 3.2.

Table **??** presents the complete ALU Decoder truth table.

TABLE 3.2: ALU decoder truth table.

ALU	funct3	OP/f7	Control	Branch	Instruction
00	x	xx	0000 (ADD)	x	Load, Store
11	x	xx	1010 (LUI)	x	LUI, U-type
01	000	xx	0001 (SUB)	Zero	BEQ
01	001	xx	0001 (SUB)	¬Zero	BNE
01	100	xx	0110 (SLT)	¬Zero	BLT
01	101	xx	0110 (SLT)	Zero	BGE
01	110	xx	0101 (SLTU)	¬Zero	BLTU
01	111	xx	0101 (SLTU)	Zero	BGEU
10	000	00,01,10	0000 (ADD)	x	ADD, ADDI
10	000	11	0001 (SUB)	x	SUB
10	010	xx	0110 (SLT)	x	SLT, SLTI
10	011	xx	0101 (SLTU)	x	SLTU, SLTIU
10	110	xx	0011 (OR)	x	OR, ORI
10	111	xx	0010 (AND)	x	AND, ANDI
10	100	xx	0100 (XOR)	x	XOR, XORI
10	001	xx	0111 (SLL)	x	SLL, SLLI
10	101	x0	1000 (SRL)	x	SRL, SRLI
10	101	x1	1001 (SRA)	x	SRA, SRAI

Note: `x` denotes a don't-care value. OP/f7 denotes `{OP5, funct7[5]}`.

3.3 From Single-Cycle to Pipelined: The Motivation

The single-cycle design is correct but slow. The clock period is set by the longest combinational path through the entire datapath — in an RV64I implementation this is typically a load instruction, which must traverse the PC logic, instruction memory, register file read, ALU, data memory, and finally the write-back multiplexer before the clock edge can safely latch the result. Every other instruction class is faster, but the clock must accommodate the worst case. The result is that the large majority of the silicon is idle during every cycle: while a load is crawling through the data memory, the instruction memory and ALU are doing nothing.

Pipelining addresses this by decomposing the single long path into five shorter stages separated by pipeline registers. Each stage takes roughly one-fifth of the original clock period, so the clock frequency rises proportionally. More importantly, five instructions can be in flight simultaneously — one in each stage — so the ideal throughput approaches one instruction completed per cycle, compared to the single-cycle design's one instruction per full datapath traversal.

The cost is that instructions in different stages may interact in ways that produce incorrect results if left unmanaged. These interactions are the *hazards* of pipelined execution, and they fall into three categories.

A **data hazard** occurs when an instruction needs a value that a preceding instruction has computed but not yet written back to the register file. Because the register file is only updated in the Write-Back (WB) stage, an instruction that reads a register in the Decode (ID) stage may obtain a stale value if the instruction that writes that register is still in the Execute (EX), Memory (MEM), or Write-Back (WB) stage.

Example 3.1 (Read After Write (RAW) Hazard)**.** *Consider the following instruction sequence:*

```
ADD x5, x1, x2      // x5 = x1 + x2
SUB x6, x5, x3      // Uses x5
```

The `SUB` *instruction depends on the value of register* `x5` *produced by the preceding* `ADD` *instruction. The* `SUB` *instruction reads* `x5` *during its Decode stage before the* `ADD` *instruction has written the updated value back to the register file. Without additional hardware support, the* `SUB` *instruction would therefore read an incorrect value, resulting in a Read After Write (RAW) data hazard.*

TABLE 3.3: RAW data hazard without forwarding.

Instruction	C1	C2	C3	C4	C5
ADD x5,x1,x2	IF	ID	EX	MEM	WB
SUB x6,x5,x3		IF	ID	EX	MEM

As shown in Table 3.3, the `SUB` instruction reads register `x5` in Cycle 3, while the `ADD` instruction does not update the register file until Cycle 5. Consequently, the operand read during the Decode stage is stale.

Data hazards are resolved primarily through data forwarding (also referred to as bypassing). Instead of waiting for the producing instruction to complete the Write-Back stage, the required result is taken directly from the pipeline stage in which it becomes available and forwarded to the Execute stage inputs of the dependent instruction. This eliminates most Read After Write (RAW) hazards without introducing pipeline stalls.

An important exception is the load-use hazard. When a load instruction is immediately followed by an instruction that consumes the loaded value, forwarding alone is insufficient because the requested data is not available until the end of the Memory stage. However, the dependent instruction requires the operand at the beginning of its Execute stage, one clock cycle earlier. Since the data has not yet been produced, the processor must insert a one-cycle pipeline stall before the dependent instruction can proceed.

Example 3.2 (Load-Use Hazard)**.** *Consider the following instruction sequence:*

```
LW  x5, 0(x1)       // Load word into x5
ADD x6, x5, x2      // Uses x5
```

The value loaded by the `LW` *instruction becomes available only at the end of the Memory stage. Since the following* `ADD` *instruction requires this operand at the beginning of its Execute stage, forwarding cannot satisfy the dependency. The hazard detection unit therefore inserts a one-cycle stall, allowing the loaded value to become available before execution continues.*

TABLE 3.4: Load-use hazard requiring one pipeline stall.

Instruction	C1	C2	C3	C4	C5	C6
LW x5,0(x1)	IF	ID	EX	MEM	WB	
STALL				ID		
ADD x6,x5,x2		IF	ID	ID	EX	MEM

Table 3.4 illustrates the timing of the load-use hazard. The loaded value is produced only at the end of the Memory stage, making it impossible to forward the operand to the dependent instruction during the following cycle. Consequently, a single pipeline bubble is inserted before the dependent instruction resumes execution.

A **control hazard** occurs because the processor must fetch the next instruction before it knows where the next instruction is. For unconditional jumps and taken branches, the instructions fetched in the cycles after the branch instruction are from the wrong address and must be discarded. The naive resolution is to assume the branch is not taken and flush incorrectly fetched instructions when the assumption proves wrong. The branch predictor described in Section 3.6 improves on this by making an informed guess about the outcome.

A **structural hazard** would arise if two instructions in the pipeline needed the same physical resource at the same time. The five-stage design avoids structural hazards between instructions by using separate instruction and data memories, but structural hazards can still arise from external stalls — for example, when the instruction cache needs multiple cycles to return an instruction, all earlier stages must be held in place until the cache responds.

The five stages of the pipeline map naturally from the single-cycle datapath:

- **IF** — Instruction Fetch: read the instruction from instruction memory.
- **ID** — Instruction Decode: decode the instruction, read registers, detect hazards.
- **EX** — Execute: perform the ALU operation, evaluate branch conditions, compute targets.
- **MEM** — Memory: access data memory for loads and stores.
- **WB** — Write-Back: write the result to the register file.

Between each pair of adjacent stages sits a pipeline register that captures the outputs of the upstream stage on the rising clock edge and holds them stable for the downstream stage to consume during the next cycle.

3.4 5-Stage Pipeline Architecture

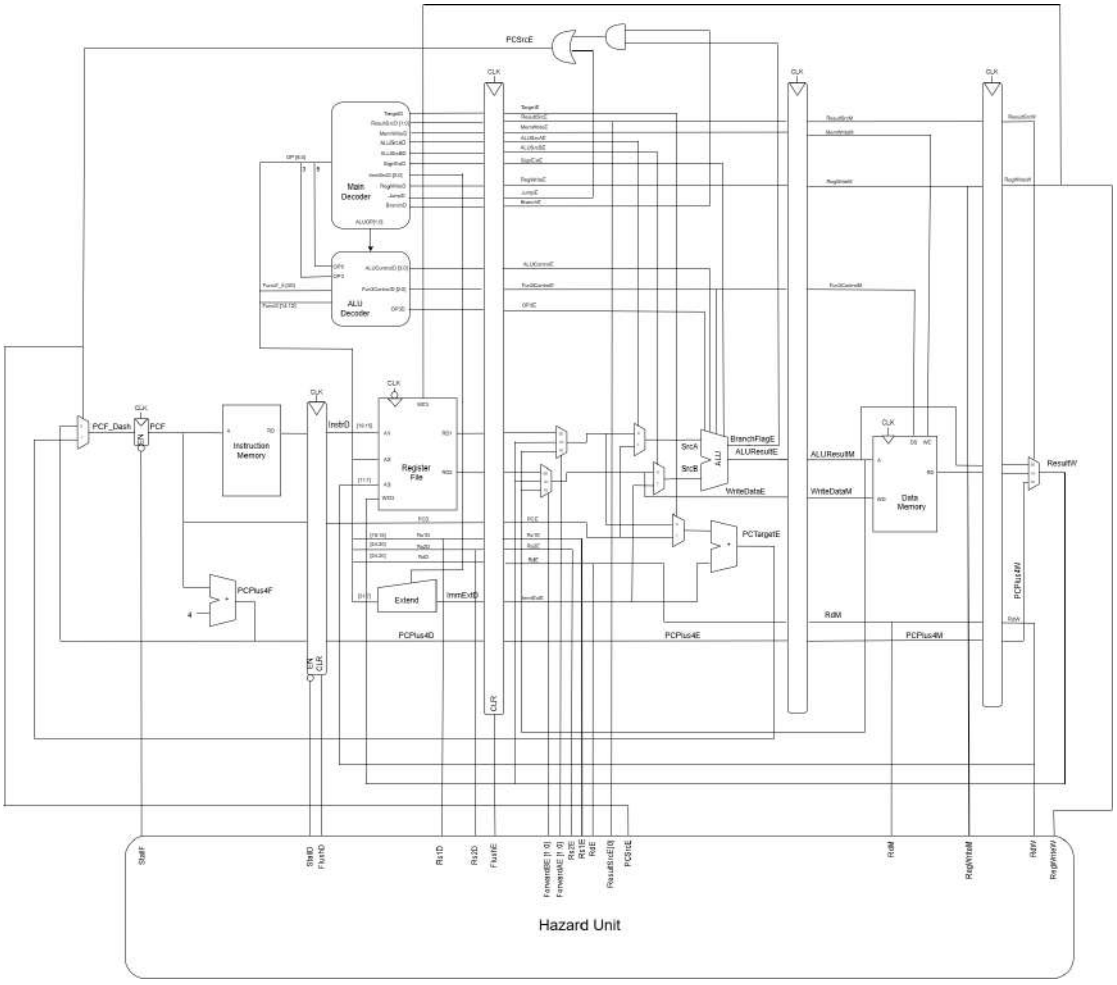


FIGURE 3.3: Five-stage pipelined RV64I.

The pipeline registers — `IF/ID`, `ID/EX`, `EX/MEM`, and `MEM/WB` — carry not only the data values that each stage produces, but also all the control signals that downstream stages need. The instruction word itself is threaded through `IF/ID` and `ID/EX` so that the exception unit can inspect it in later stages. The PC is propagated through all four registers so that branch targets, `PC+4` return addresses, and exception `mepc` values are always associated with the correct instruction regardless of which stage they are consumed in.

3.4.1 Fetch Stage (IF)

The Fetch stage drives the PC and presents it to the instruction memory. In the absence of any redirect, the PC advances to `PC + 4`. Four sources can override this: a branch or jump target resolved in the Execute stage (`PCTargetE`), the branch predictor's predicted target (`PredictTarget`), the trap handler address (`mtvec`) on an exception, and the saved exception PC (`mepc`) on an `MRET`. These are arbitrated by a priority multiplexer in which exception redirects take precedence over branch resolution, which takes precedence over prediction, which takes precedence over the sequential increment.The branch prediction mechanism and the exception/CSR subsystem are described in dedicated sections later in this chapter. Their interaction with the fetch stage is introduced here for completeness and explained in detail in the corresponding sections.

When the pipeline control logic asserts `StallF`, the program counter is held constant and no new instruction is fetched until the stall condition is cleared. Pipeline stalls may occur for several reasons, including hazard resolution and instruction cache misses, as discussed in Chapter 3.5.

3.4.2 Decode Stage (ID)

The Decode stage interprets the fetched instruction and prepares all information required by the downstream pipeline stages. It reads the source operands from the register file, generates the instruction immediate, produces the control signals, and performs early instruction classification.

Register operands are read using the `rs1` and `rs2` fields stored in the `IF/ID` pipeline register. In parallel, the Immediate Generator reconstructs the appropriate sign-extended immediate according to the instruction format selected by the `ImmSrc` control signal.

The main control unit decodes the opcode and generates the control signals required by the remaining pipeline stages. These signals are stored in the `ID/EX` pipeline register together with the decoded operands, immediate value, instruction word, and program counter.

This stage also interfaces with the Hazard Detection Unit, which may temporarily stall instruction decoding when required. The detailed operation of hazard detection and pipeline stalling is presented in Section 3.5.

Finally, the Decode stage identifies instructions that generate synchronous exceptions, including illegal instructions, `ECALL`, and `EBREAK`. The corresponding exception information is propagated through the pipeline for handling by the exception subsystem.

3.4.3 Execute Stage (EX)

The Execute stage performs the core computation of the pipeline. Depending on the instruction type, it executes arithmetic and logical operations, computes effective addresses for memory accesses, evaluates branch conditions, and generates branch or jump target addresses. The ALU receives its operands from the `ID/EX` pipeline register, with the second operand selected between the register value and the sign-extended immediate according to the `ALUSrc` control signal.

Since register values may not yet have reached the Write-Back stage, the Execute stage obtains the most recent operand values through the forwarding network whenever necessary. This allows dependent instructions to execute without waiting for the register file to be updated. The forwarding logic operates transparently before the ALU performs its computation, while its implementation and selection logic are described in detail in Section 3.5.1.

For branch and jump instructions, the Execute stage also determines the next program counter. The branch comparison logic evaluates the branch condition using the ALU result together with the instruction's `funct3` field, while the target address is computed in parallel from the current program counter and the decoded immediate value. If the instruction changes the control flow, the computed target address is supplied to the Fetch stage and the pipeline control logic redirects execution accordingly. The recovery mechanism for control-flow changes, including flushing incorrectly fetched instructions and branch prediction, is presented in Section 3.5.2.

3.4.4 Memory Stage (MEM)

The Memory stage issues load and store operations to the data memory. The address is `ALUResultM` (computed in the preceding Execute stage), the write data is the forwarded value of `rs2` (held in `WriteDataM`), and the access width is encoded in `DataSizeCon[2:0]`. For store instructions, `MemWrite` enables the write; for load instructions, the memory output is captured and forwarded to the Write-Back stage. Misaligned address exceptions are detected here by checking the low-order address bits against the access width, and the fault address is recorded for `mtval`.

3.4.5 Write-Back Stage (WB)

The Write-Back stage selects the final register-file write value. The `ResultSrc[1:0]` multiplexer chooses among the ALU result, the memory read data, `PC+4` (for JAL/JALR return addresses), and — once CSR instructions are added — the CSR read value. The selected value is written to `rdW` in the register file when `RegWriteW` is asserted.

3.5 Hazard Handling

3.5.1 Data Hazards and Forwarding

The forwarding unit eliminates most read-after-write (RAW) data hazards without introducing pipeline stalls. Operating in the Execute stage, it selects each ALU source operand from one of three candidates: the register file output (no forwarding), the value in the EX/MEM pipeline register (EX-to-EX forwarding), or the value in the MEM/WB pipeline register (MEM-to-EX forwarding).The selection conditions are:

- **EX-to-EX forward (ForwardAE/ForwardBE = 10):** `rdM == rs1E/rs2E` and `RegWriteM = 1` and `rdM` $\neq$ `0`.
- **MEM-to-EX forward (ForwardAE/ForwardBE = 01):** `rdW == rs1E/rs2E` and `RegWriteW = 1` and `rdW` $\neq$ `0` and the EX-to-EX condition is not active.
- **No forward (ForwardAE/ForwardBE = 00):** no matching producer in the pipeline.

The conditions `rdM` $\neq 0$ and `rdW` $\neq 0$ prevent forwarding from instructions whose destination register is `x0`. Since `x0` is hardwired to zero, writes to it have no architectural effect and therefore must not be forwarded.

The one hazard forwarding cannot resolve is the *load-use* case: a load instruction in Execute does not produce its result until the end of the Memory stage, but the dependent instruction would need that result at the start of its own Execute stage — one cycle before it is available. The hazard detection unit in the Decode stage catches this case with the condition:

`ResultSrcE[0] = 1` **AND** (`rdE = rs1D` **OR** `rdE = rs2D`)

When the condition is true, the unit asserts `StallF` and `StallD` to freeze the Fetch and Decode stages, and asserts `FlushE` to inject a bubble (NOP with all control signals zeroed) into the Execute stage. The inserted bubble delays the dependent instruction by one cycle, allowing the load to complete the Memory stage. On the following cycle,

the loaded value is forwarded through the MEM-to-EX path, and execution resumes correctly.

3.5.2 Control Hazards and Branch Handling

The base pipeline resolves branches in the Execute stage. Because branch outcomes are determined in the Execute stage, the processor has already fetched the instructions at `PC+4` and `PC+8`. If the branch is taken, both instructions belong to the wrong execution path and must be discarded.

`JALR` requires special handling because its target address depends on a register operand, which itself may require forwarding. The Execute stage computes the target using the forwarded operand, allowing the existing forwarding network to resolve data dependencies without additional hardware. The `Target` control signal (asserted for all control-transfer instructions except `JALR`) distinguishes between PC-relative and register-based targets.

For unconditional JAL instructions, the target is known at the Decode stage (it depends only on the PC and the immediate), but the implementation resolves JAL in Execute alongside branches, accepting a consistent two-cycle redirect penalty in exchange for a simpler control path. The branch predictor described in the next section eliminates this penalty in the common case.

3.5.3 Structural Hazards

Unlike a classical pipeline, this design does not experience structural hazards caused by competing hardware resources, since instruction and data accesses are separated. Instead, structural stalls occur only when the instruction cach. In this case, the hazard unit freezes the pipeline and no flush is required because the instructions already in Execute, Memory, and Write-Back remain valid and must complete.

3.6 Branch Prediction

Conditional branch instructions introduce *control hazards* into a pipelined processor because the correct next program counter (PC) cannot be determined immediately after the branch instruction is fetched. Unlike arithmetic instructions, whose execution proceeds sequentially, a branch instruction may alter the control flow depending on the outcome of a comparison. Consequently, the Fetch stage must decide which instruction to fetch next before the branch outcome is available.

In the five-stage pipeline presented in this work, branch conditions are evaluated in the Execute stage. During the intervening cycles, the processor continues fetching instructions assuming that execution will proceed sequentially. If the branch is ultimately taken,

the fetched instructions belong to the wrong execution path and must be discarded. The pipeline must then be flushed and instruction fetching restarted from the correct target address, introducing a performance penalty.

As discussed in Section 3.5, the baseline pipeline incurs a two-cycle penalty whenever a branch is taken. Although this penalty may appear modest for an isolated branch, its cumulative effect becomes significant in applications containing frequent control-flow changes. Loop-intensive workloads are particularly affected because the terminating branch is executed repeatedly throughout program execution. In a typical loop, the branch is taken during every iteration except the final one, causing a sequential fetch policy to follow the incorrect execution path for nearly every dynamic branch.

This behavior is especially detrimental in embedded and machine-learning applications, where a substantial portion of the execution time is spent inside nested loops. Repeated pipeline flushes reduce the effective instruction throughput and limit the performance benefits provided by pipelining. Consequently, modern processors employ branch prediction mechanisms that attempt to determine the correct execution path before the branch instruction reaches the Execute stage.

Branch prediction attempts to reduce or eliminate control hazard penalties by predicting the future behavior of branch instructions. Rather than waiting until the branch condition is evaluated, the processor predicts whether the branch will be taken and, if necessary, predicts the destination address as well. When the prediction is correct, instruction fetching continues uninterrupted and the pipeline experiences no control-hazard penalty. Only incorrect predictions require flushing the incorrectly fetched instructions and restarting execution from the correct target address.

3.6.1 Branch Prediction Techniques

Branch prediction algorithms are commonly classified into two categories: *static* predictors and *dynamic* predictors.

Static predictors make their decision without considering the execution history of a branch instruction. Their prediction is determined solely by a fixed rule or compiler-generated information. The simplest static predictor always assumes that branches are not taken, allowing the processor to continue fetching sequential instructions without additional hardware. Although attractive because of its simplicity, this approach performs poorly for loop branches, which are typically taken during every iteration except the last.

An alternative strategy always predicts branches as taken. While this improves the prediction accuracy for loop branches, it performs worse for forward conditional branches

that are frequently not taken. More sophisticated static schemes, such as the Backward Taken Forward Not Taken (BTFNT) heuristic, exploit the observation that backward branches usually correspond to loops whereas forward branches commonly implement conditional statements. Although these techniques improve prediction accuracy compared to fixed policies, they remain unable to adapt to the runtime behavior of individual branches.

Dynamic branch predictors overcome this limitation by observing the execution history of each branch instruction. Instead of making a fixed prediction, the processor records previous branch outcomes and uses this information to predict future executions. This approach exploits the temporal locality exhibited by most programs, where a branch that was recently taken is likely to behave similarly during subsequent executions. As a result, dynamic predictors generally achieve substantially higher prediction accuracy than static approaches while requiring only modest additional hardware.

3.6.2 One-Bit Dynamic Predictor

The simplest dynamic predictor associates a single prediction bit with each branch instruction. This bit records the outcome of the previous execution, with one state representing "Taken" and the other representing "Not Taken." Whenever the branch executes, the predictor updates the stored bit to reflect the new outcome. Future executions simply repeat the previous behavior.

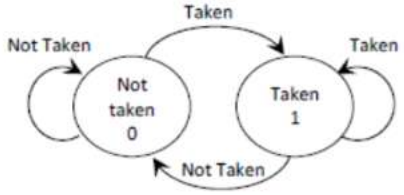


FIGURE 3.4: One Bit Branch Predictors State Machine

Although conceptually simple and inexpensive to implement, the one-bit predictor suffers from poor performance when executing loops. Consider a loop that iterates one hundred times. The branch is taken during the first ninety-nine iterations and is not taken only when the loop terminates. During the final iteration, the predictor changes its state from Taken to Not Taken. When the loop executes again, the first iteration is immediately mispredicted because the predictor still expects the branch to be not taken. Consequently, every loop execution produces two mispredictions: one when the loop exits and another when it begins again.

This behavior significantly reduces the effectiveness of the predictor for workloads dominated by repetitive loops, making a more robust prediction mechanism desirable.

3.6.3 Two-Bit Saturating Predictor

To improve prediction accuracy, many modern processors employ a two-bit saturating counter instead of a single prediction bit. Rather than immediately reversing its prediction after one unexpected branch outcome, the predictor changes its prediction only after two consecutive outcomes disagree with the current prediction. This characteristic, commonly referred to as *hysteresis*, prevents temporary deviations from permanently altering the prediction state.

The predictor consists of four states:

- Strongly Not Taken (00)
- Weakly Not Taken (01)
- Weakly Taken (10)
- Strongly Taken (11)

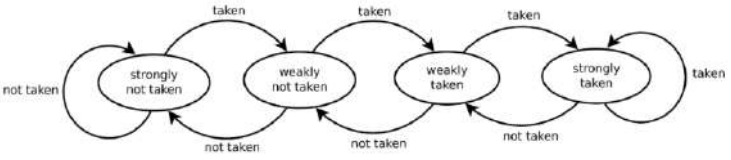


FIGURE 3.5: Two Bit Branch Predictors State Machine

Whenever a branch is taken, the counter increments toward the Strongly Taken state. Conversely, a not-taken branch decrements the counter toward the Strongly Not Taken state. The counter saturates at both extremes, preventing overflow beyond the two terminal states.

Unlike the one-bit predictor, a branch that has consistently been taken remains predicted as taken even after a single unexpected not-taken outcome. Only a second consecutive not-taken execution changes the prediction direction. Consequently, a loop typically incurs only one misprediction when it terminates instead of two, substantially improving prediction accuracy for repetitive control-flow patterns.

The additional hardware required by the two-bit predictor is minimal, consisting of only one extra state bit per branch. Nevertheless, this small increase in storage significantly improves prediction accuracy, making the two-bit saturating counter one of the most widely adopted branch prediction algorithms in modern processors.

3.6.4 Selection of the Prediction Algorithm

Considering the trade-off between hardware complexity and prediction accuracy, this processor adopts the two-bit saturating predictor. More advanced predictors, such as

two-level adaptive predictors, tournament predictors, perceptron predictors, and TAGE predictors, can achieve higher prediction accuracy by exploiting longer branch histories and more sophisticated learning mechanisms. However, these techniques require considerably larger storage structures, more complex update logic, and longer prediction latency.

The target platform of this processor is an FPGA-based embedded system where hardware resources are constrained and deterministic timing is desirable. Under these design constraints, the two-bit predictor provides an effective compromise between implementation cost and performance improvement. It requires only two bits of prediction state per branch while eliminating the majority of loop-related mispredictions encountered by the baseline pipeline.

The implemented Branch Prediction Unit (BPU) therefore combines a Branch Target Buffer (BTB), responsible for storing branch target addresses, with a Branch State Machine (BSM), which implements the two-bit saturating prediction algorithm. Together, these structures enable branch prediction during the Fetch stage while maintaining a compact hardware implementation suitable for FPGA deployment.

3.6.5 Branch Prediction Unit Architecture

The Branch Prediction Unit (BPU) implemented in this processor is illustrated in Figure 3.6. The predictor operates entirely during the Fetch stage, allowing the next program counter to be selected before the branch instruction reaches the Execute stage. The BPU consists of two cooperating structures:

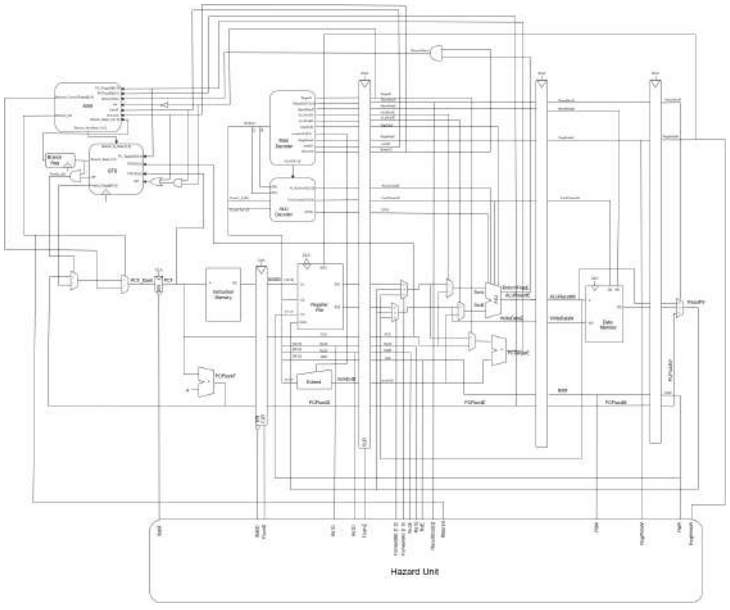


FIGURE 3.6: Pipeline with Branch Prediction Unit.

- A *Branch Target Buffer (BTB)*, which stores previously observed branch target addresses together with branch identification information.
- A *Branch State Machine (BSM)*, which implements the two-bit saturating prediction algorithm used to determine whether a branch should be predicted as taken or not taken.

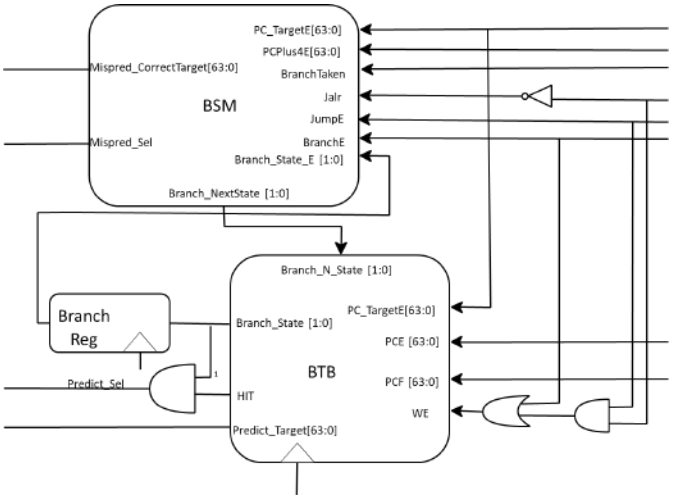


FIGURE 3.7: Branch Prediction Unit.

During instruction fetch, the current program counter is presented to the BTB. If a matching branch entry is found and the corresponding prediction state indicates that the branch is likely to be taken, the predictor redirects instruction fetch to the stored target address. Otherwise, fetching proceeds sequentially from `PC+4`. Once the branch instruction reaches the Execute stage, its actual outcome is compared with the prediction. The prediction state is then updated and, if necessary, the pipeline is redirected to the correct execution path.

This organization separates branch direction prediction from target address storage while maintaining a compact implementation suitable for FPGA deployment.

3.6.6 Branch Target Buffer

The Branch Target Buffer is a direct-mapped cache-like structure that stores information about previously encountered control-transfer instructions. Each BTB entry contains

- the upper address bits of the branch instruction (tag),
- the predicted target address,
- and the associated two-bit prediction state.

The BTB contains 64 entries and is indexed using bits `PCF[7:2]` of the Fetch-stage program counter. These six index bits uniquely select one entry within the table, while the remaining upper bits (`PCF[63:8]`) are stored as the tag.

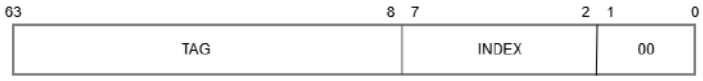


FIGURE 3.8: BTB Entries

Using a tag is necessary because multiple program counters may map to the same BTB index. Whenever the Fetch stage accesses the BTB, the stored tag is compared against the upper bits of the current program counter. A prediction is considered valid only if both the index and the tag match. This comparison prevents unrelated branch instructions from incorrectly sharing prediction information.

If both the tag comparison succeeds and the prediction state indicates a taken branch, the BTB asserts `Predict_Sel` and supplies the stored target address through `Predict_Target`. The program counter multiplexer therefore selects the predicted target as the next fetch address.

If either the tag comparison fails or the prediction state indicates that the branch is not taken, the BTB produces no prediction and the processor continues fetching sequentially using `PC+4`.

The BTB is updated only after a branch or jump instruction has been resolved in the Execute stage. At this point, the actual target address is known and the branch outcome has been determined. The BTB entry indexed by `PCE[7:2]` is updated with

- the new tag (`PCE[63:8]`),
- the resolved branch target (`PC_TargetE`),
- and the updated prediction state generated by the BSM.

Updating the BTB only after branch resolution guarantees that every stored entry corresponds to the architecturally correct execution path.

3.6.7 Branch State Machine

Each BTB entry contains a two-bit saturating counter implementing the Branch State Machine (BSM). The state machine records the recent execution behavior of the associated branch instruction and determines the prediction presented during instruction fetch.

The predictor supports four states:

State	Prediction
00	Strongly Not Taken
01	Weakly Not Taken
10	Weakly Taken
11	Strongly Taken

Following reset, all prediction entries are initialized to the Weakly Not Taken state. This conservative initialization avoids strongly biasing predictions before sufficient execution history has been collected.

Whenever a branch instruction reaches the Execute stage, its actual outcome is determined by the branch comparison logic. If the branch is taken, the corresponding counter is incremented toward the Weakly Taken state. Conversely, if the branch is not taken, the counter is decremented toward the Strongly Not Taken state. Since the counter saturates at both extremes, additional increments or decrements do not move beyond the terminal states.

The prediction used during instruction fetch depends solely on the most significant bit of the counter. States `10` and `11` predict that the branch will be taken, whereas states `00` and `01` predict that execution will continue sequentially.

This implementation introduces hysteresis into the prediction process. A branch that has repeatedly been taken remains predicted as taken even after a single unexpected not-taken outcome. Likewise, a strongly not-taken branch requires two consecutive taken outcomes before reversing its prediction. This behavior significantly improves prediction accuracy for repetitive execution patterns such as loops.

3.6.8 Prediction and Update Sequence

The operation of the Branch Prediction Unit can be divided into two phases: prediction and update.

During the prediction phase, the Fetch stage presents the current program counter to the BTB. The corresponding entry is selected using the index bits of the program counter, after which the stored tag is compared with the incoming address. If the comparison succeeds and the prediction state indicates a taken branch, the BTB supplies the predicted target address and asserts `Predict_Sel`. The program counter multiplexer then redirects instruction fetch to the predicted destination. Otherwise, the Fetch stage continues with the sequential address `PC+4`.

The update phase begins when the branch instruction reaches the Execute stage. At this point, the branch condition has been evaluated and the correct target address has been

computed. The actual branch outcome is compared with the prediction that was made during the Fetch stage.

If the prediction was correct, execution proceeds without interruption. The BSM simply updates the saturating counter according to the branch outcome, reinforcing the existing prediction.

If the prediction was incorrect, the BSM asserts `Mispred_Sel`. The correct destination address is simultaneously supplied through `Mispred_CorrectTarget`. This signal selects either

- `PC_TargetE`, when the branch was taken but predicted not taken, or
- `PCPlus4E`, when the branch was predicted taken but actually was not taken.

The hazard unit responds by asserting `FlushD` and `FlushE`, removing the incorrectly fetched instructions from the pipeline. The program counter is redirected to the correct address, after which instruction fetching resumes normally. Finally, the BTB entry and its corresponding prediction state are updated to reflect the actual execution outcome.

3.6.9 Handling Unconditional Jumps

The prediction mechanism also supports unconditional `JAL` instructions. Since these instructions always transfer control, they are treated as permanently taken branches. During execution, the corresponding BTB entry is updated with the computed destination address while the BSM is forced into the Strongly Taken state. Future executions of the same `JAL` instruction therefore redirect the Fetch stage immediately without waiting for branch resolution.

The handling of `JALR` instructions is fundamentally different. Unlike `JAL`, the destination address of a `JALR` instruction depends on the value stored in a general-purpose register. This value is not available during the Fetch stage and may itself require forwarding from later pipeline stages. Consequently, the destination cannot be determined using a predictor indexed solely by the program counter.

In the implemented processor, `JALR` instructions bypass the branch predictor and are resolved in the Execute stage together with the remaining control-transfer logic. Although this approach introduces the normal control-hazard penalty, it significantly simplifies the prediction hardware while maintaining correct program execution.

More sophisticated processors often employ indirect branch predictors or return-address stacks (RAS) to improve the prediction accuracy of indirect jumps and procedure returns. These techniques increase hardware complexity and were therefore considered beyond the scope of the present implementation.

3.7 Exception and CSR Subsystem

The exception and CSR subsystem gives the pipelined core the minimum Machine-mode support required for deterministic trap handling. It detects architectural faults, records the fault context in CSRs, redirects the PC to the trap handler, and later resumes execution when software executes `MRET`. Figure 3.9 shows the main structure: exception sources from the pipeline feed a priority encoder, whose output controls CSR updates, pipeline flushing, and PC redirection.

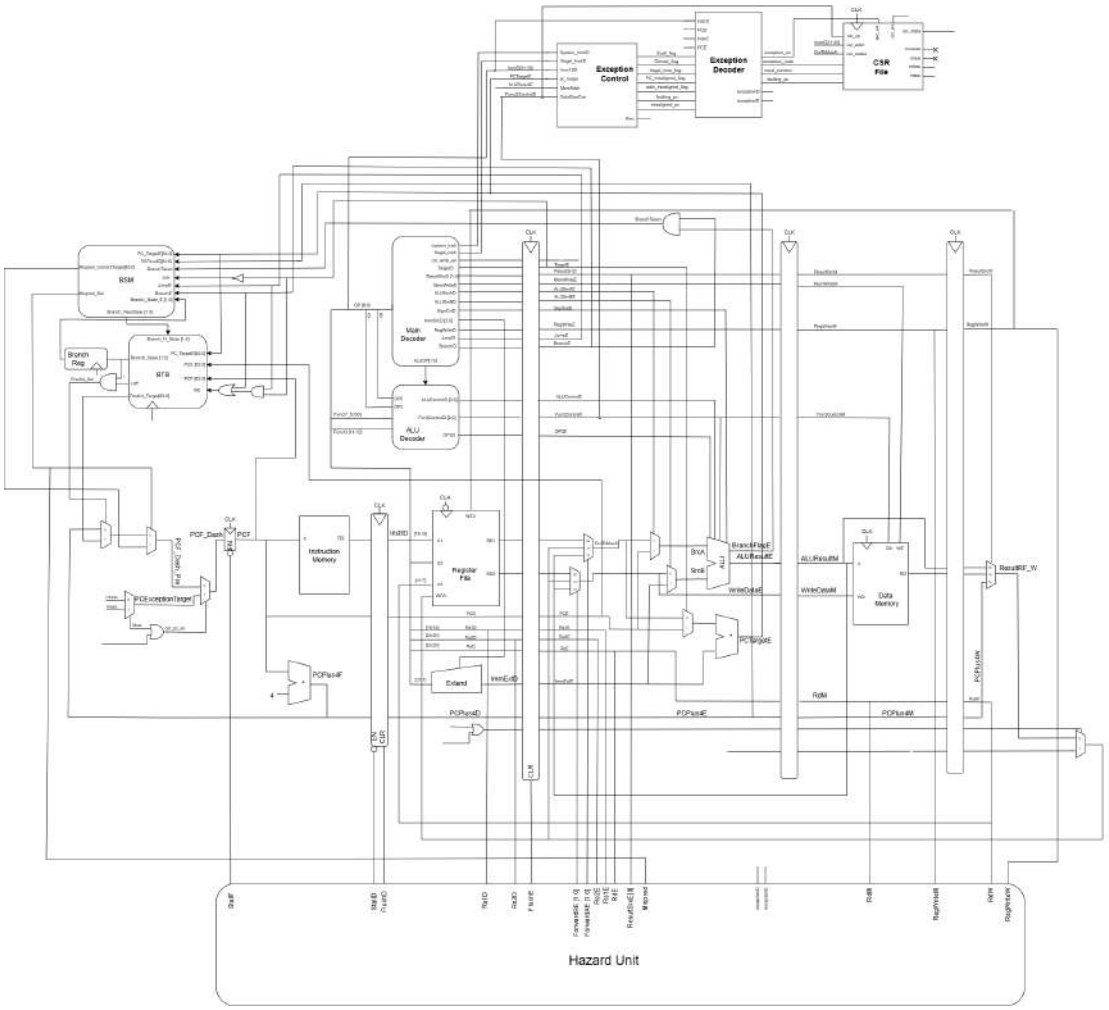


FIGURE 3.9: Exception and CSR unit: detection sources from each pipeline stage feed the priority and CSR-update logic.

3.7.1 Exception Sources and Precise Trap Priority

Seven exception conditions are supported, as listed in Table 3.5. Because the pipeline contains several in-flight instructions, more than one stage can raise an exception signal in the same cycle. The unit still maintains *precise exceptions*: the oldest faulting instruction is handled first, older completed instructions remain visible, and all younger instructions

are flushed before they can update architectural state. The priority column is therefore a tie-breaker among exception causes belonging to the selected faulting instruction, not a mechanism for allowing a younger instruction to override an older one.

TABLE 3.5: Supported exception types and reported trap information.

Priority	Exception	mcause	mtval	Detection Stage	Description
1	Instruction access fault	1	Faulting PC	Fetch	Bus error or permission violation during instruction fetch.
2	Illegal instruction	2	Instruction bits	Decode	Opcode or encoding not recognised by the decoder.
3	Instruction address misaligned	0	Faulting PC	Fetch	PC not 4-byte aligned; typically caused by a bad jump target.
4	ECALL	11	0	Decode	Synchronous environment call from M-mode software.
5	EBREAK	3	0	Decode	Software breakpoint; transfers control to the debug handler.
6	Load/store address misaligned	4	Faulting address	Execute	Data address not naturally aligned to the access width.
7	Load/store access fault	5, 7	Memory address	Memory	Bus error or permission violation during data memory access.

3.7.2 Trap Entry, CSR Update, and Pipeline Flush

On exception entry, the core performs the trap update atomically on the clock edge. The PC of the selected faulting instruction is written to `mepc`, the encoded cause is written to `mcause`, and auxiliary information is written to `mtval`. The PC is then redirected to `mtvec`. At the same time, pipeline registers are flushed selectively: younger instructions are removed, while older instructions that are already beyond the faulting point are allowed to complete if they are architecturally valid. CSR writes from ordinary CSR instructions are suppressed during exception entry so that `mepc`, `mcause`, and `mtval` cannot be overwritten by an in-flight instruction in the same cycle.

The detection points follow the pipeline naturally. Fetch detects instruction access faults and PC misalignment; Decode detects illegal instructions, `ECALL`, and `EBREAK`; Execute detects load/store address misalignment; and Memory detects load/store access faults. If a stall and an exception occur together, the exception has priority over the stall because the architectural trap state must not be delayed or lost. Control-flow redirection uses the same ordering: reset has the highest priority, followed by exception entry, `MRET`, branch-misprediction correction, and finally the normal sequential `PC`+4 update.

3.7.3 Branch Misprediction and Illegal-Instruction Interaction

A subtle case appears when branch prediction and illegal-instruction detection interact. During a misprediction, the Fetch stage may already hold a wrong-path instruction. If that instruction has an unrecognised opcode, the decoder can raise `illegal_instr_flag` even though the instruction should be flushed and must never become architecturally

visible. Figure 3.10 shows this failure mode: the wrong-path illegal-instruction signal asserts `exception_en`, causing a spurious trap before the branch correction finishes.

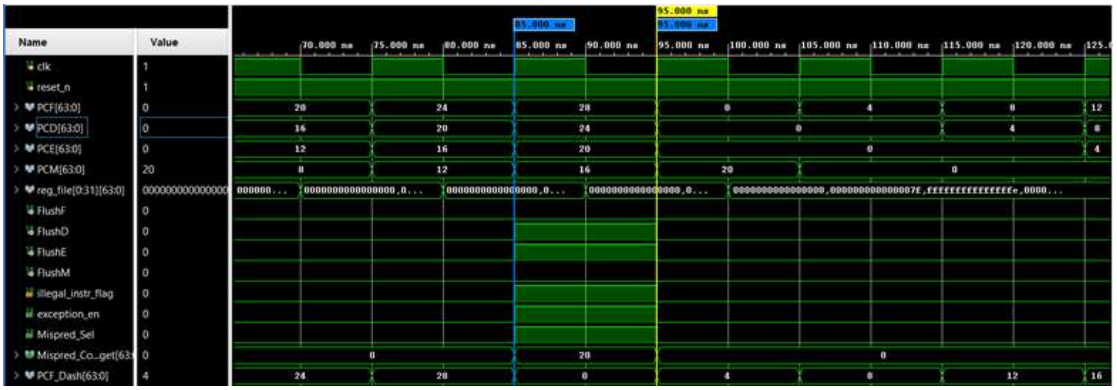


FIGURE 3.10: Simulation waveform before the fix: a wrong-path instruction raises `illegal_instr_flag` during a branch misprediction and incorrectly enables exception entry.

The fix is to suppress the illegal-instruction exception whenever it coincides with a branch misprediction. Other exception sources such as `ECALL`, `EBREAK`, misaligned accesses, and bus errors remain unaffected because they originate from instructions that have reached later stages and are already on the confirmed architectural path. After the fix, Figure 3.11 shows that `exception_en` remains de-asserted during the misprediction window and the pipeline correctly redirects to the intended target.

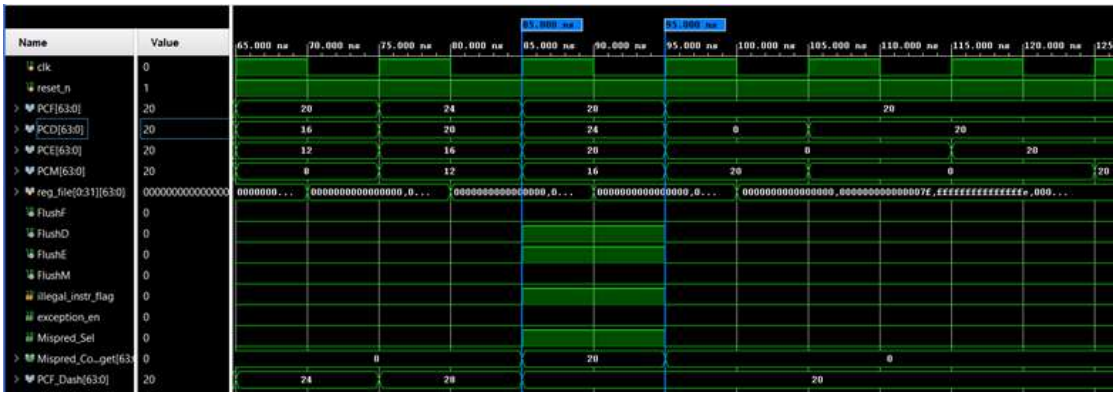


FIGURE 3.11: Simulation waveform after the fix during the misprediction window.

3.7.4 Software Trap Handling

Software observes traps through three CSRs. `mcause` identifies the trap cause, `mepc` stores the address to resume from or skip, and `mtval` provides cause-specific detail such as a faulting memory address or illegal instruction bits. A trap handler can therefore dispatch on `mcause`, use `mepc` to emulate or skip the faulting instruction, and then execute `MRET`. The `MRET` instruction flushes any younger speculative work and reloads the PC from

`mepc`; it does not itself generate an exception. Listing **??** gives a compact example of this software flow.

3.8 M-Extension and MAC Instructions

3.8.1 Multiplier Architecture

The M-extension multiplier is implemented as a combinational 64×64-bit Wallace-tree structure capable of producing a full 128-bit product in a single clock cycle. The design supports all four multiplication instructions defined by the RISC-V M extension — `MUL`, `MULH`, `MULHSU`, and `MULHU` — as well as the word-width variant `MULW`, and is shared with the MAC unit described in Section 3.8.1.8. A 3-bit `Mul_Control` signal selects the active instruction, while a 2-bit `mode` input allows the same hardware to operate on narrower operand widths (8, 16, 32, or 64 bits) for SIMD multiplication paths.

3.8.1.1 Sign Handling

Each multiplication variant imposes a different sign convention on its two operands. Rather than maintaining separate signed and unsigned datapaths, the multiplier adopts a sign-magnitude approach: the magnitude of each operand is computed unconditionally, the product of the two magnitudes is formed by the Wallace tree, and the result is negated at the output if and only if exactly one operand was negative. This reduces the Wallace tree itself to a purely unsigned structure and confines all sign logic to two small combinational blocks at the input and output.

The per-operand sign bits are derived as follows. For `MULH` (signed×signed), both `sign_a` and `sign_b` are taken directly from the most-significant bits of the respective 64-bit inputs. For `MULHSU` (signed×unsigned), `sign_a` is taken from `a[63]` while `sign_b` is forced to zero, making the second operand unconditionally non-negative. For `MULHU` (unsigned×unsigned), both sign bits are forced to zero. For `MUL` and the MAC path, signedness is controlled externally by the `signed_en` qualifier, so `sign_a = signed_en & a[63]` and `sign_b = signed_en & b[63]`. In all cases the absolute value of each operand is formed by a conditional two's-complement negation:

$$\text{abs}(x) = \begin{cases} \sim x + 1 & \text{if } \texttt{sign_x} = 1 \\ x & \text{otherwise} \end{cases} \tag{3.1}$$

The sign of the 128-bit product is then `result_neg = sign_a ^ sign_b`, and the final output is negated in two's complement when `result_neg` is asserted.

3.8.1.2 Wallace Tree Structure

The Wallace tree is organised as a two-stage hierarchy. In the first stage, the 64-bit absolute-value operands are each partitioned into eight 8-bit slices. An 8×8 grid of `Wallace_tree_8` sub-modules computes all 64 partial products $pp[i][j] = \text{abs_a}[8i{+}7 : 8i] \times \text{abs_b}[8j{+}7 : 8j]$, each yielding a 16-bit result. This decomposition exploits the fact that an n-bit multiplier can be constructed hierarchically from smaller multipliers, and the 8×8 granularity keeps each leaf instance small enough to map efficiently to the carry-save adder trees available in the target FPGA fabric.

In the second stage, each 16-bit partial product $pp[i][j]$ is zero-extended to 128 bits and shifted left by $8(i + j)$ bit positions to align it with its correct weight in the full 128-bit product:

$$sh[i][j] = pp[i][j] \ll 8(i + j) \tag{3.2}$$

The shifted partial products are then summed. To support the variable-width SIMD modes, the summation is performed over four subsets of the grid:

- **8-bit mode:** only $sh[0][0]$ is used.
- **16-bit mode:** the 2×2 block $sh[i][j],\ i, j \in \{0, 1\}$ is summed.
- **32-bit mode:** the 4×4 block $sh[i][j],\ i, j \in \{0, \ldots, 3\}$ is summed.
- **64-bit mode:** the full 8×8 grid is summed, producing the complete 128-bit product.

Within each mode the partial-product rows are accumulated using tree-structured additions, with the final row sums combined pairwise to minimise the critical path. The `mode[1:0]` signal selects among the four raw sums before the sign-correction step is applied.

3.8.1.3 Result Selection

After sign correction, a final multiplexer controlled by `Mul_Control` selects the portion of the 128-bit product presented to the pipeline:

Table 3.6: Multiplier output selection by instruction.

Mul_Control	Instruction	Operand signs	Result
`3'b100`	`MUL`	controlled by `signed_en`	bits [63:0]
`3'b101`	`MULH`	signed × signed	bits [127:64]
`3'b110`	`MULHSU`	signed × unsigned	bits [127:64]
`3'b111`	`MULHU`	unsigned × unsigned	bits [127:64]

`MUL` discards the upper half of the product; the lower 64 bits are identical regardless of whether the operands are interpreted as signed or unsigned, which is why the RISC-V specification provides only one low-half instruction. `MULHU` bypasses the sign-correction negation entirely — since `result_neg` is structurally zero for an unsigned×unsigned multiply, `raw_sum[127:64]` is used directly to make the intent explicit in the RTL.

3.8.1.4 Why a Wallace Tree?

The fundamental challenge in building a fast integer multiplier is reducing a large number of partial products into a single sum as quickly as possible. For a 64×64-bit multiplier, a naïve approach generates 64 partial products — one for each bit of the multiplier operand — each 64 bits wide and shifted by one position relative to its neighbour. Summing these sequentially with a ripple-carry adder chain would require 63 additions executed one after another, producing a critical path that grows linearly with the operand width. This is unacceptable for a processor targeting high clock frequencies.

Three architectural families are commonly considered for this reduction problem:

Array Multiplier: An array multiplier arranges the partial products in a two-dimensional array and reduces them column by column using a grid of half-adders and full-adders. The structure is regular and maps well to standard-cell layouts, but its critical path is proportional to the number of partial product rows: each row must wait for the carry produced by the row above it, giving an $O(n)$ delay for an n-bit operand. For 64-bit operands this results in a long carry-propagation chain that limits the achievable clock frequency.

Wallace Tree Multiplier: The Wallace tree eliminates the sequential carry-propagation problem by reducing partial products in parallel. Instead of processing columns one at a time, a Wallace tree applies carry-save adders (CSAs) simultaneously across all columns at every reduction stage. A carry-save adder takes three input bits of the same weight and produces a sum bit and a carry bit of the next weight, without waiting for an incoming carry—the carry is saved and passed laterally rather than propagated upward. By applying CSA layers in parallel, the number of partial-product rows is reduced by a factor of $3/2$ at each stage. Starting from n rows, the number of stages required to reach two rows is $O(\log_{1.5} n)$, giving a critical path that grows logarithmically rather than linearly with n and offering a fundamental speed advantage over array structures at wide operand widths. For a 64×64-bit multiplier the improvement is significant: an array multiplier requires a critical path passing through approximately 64 full-adder stages, whereas a Wallace tree reaches two rows in roughly $\lceil \log_{1.5} 64 \rceil = 10$ CSA stages. This logarithmic reduction is the primary reason the Wallace tree was chosen for EdgeCore-V.

3.8.1.5 How the Wallace Tree Reduction Works

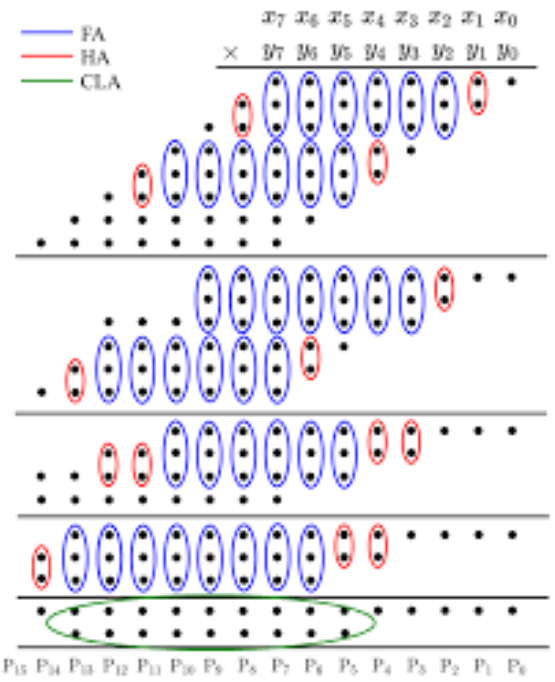


FIGURE 3.12: wallace_dot_reduction_8bit $P_{15} \dots P_0$.

Figure 3.12 illustrates this process concretely for $n = 8$. The top row shows the initial partial-product matrix: column x_7 (the centre of the triangle) holds the maximum of 8 dots, tapering to a single dot at each edge, exactly as predicted by Step 1. Each subsequent row of the figure applies one CSA layer — full adders wherever three dots share a column, half adders wherever only two remain — and the column heights visibly contract from 8, to 5, to 3, to 2 dots, matching the $\lfloor \frac{2r}{3} \rfloor + (r \bmod 3)$ recurrence from Step 2. By the final layer, each column's partial sums have been reduced to the point where the last set of half/full adders produces the finished product bits directly, without requiring a separate carry-propagate addition stage.

The operation of a Wallace tree multiplier proceeds in three conceptual steps: partial product generation, carry-save reduction, and final addition.

Step 1 — Partial Product Generation. For two n-bit unsigned operands A and B, the partial products are formed as:

$$PP_i = A \cdot b_i \cdot 2^i, \quad i = 0, 1, \dots, n-1 \tag{3.3}$$

where b_i is the i-th bit of B. Each partial product PP_i is simply A shifted left by i positions if $b_i = 1$, or zero otherwise. This gives n binary numbers, each up to $2n$ bits wide, whose sum equals the full product $A \times B$.

Step 2 — Carry-Save Reduction. The Wallace tree reduces these n partial products to exactly two numbers using carry-save adders arranged in a tree. At each reduction layer, every group of three rows at the same bit-weight is fed into a full-adder: the full-adder produces a *sum* output at the same weight and a *carry* output at the next higher weight. Crucially, neither output waits for a carry from a neighbouring column — all columns are processed in parallel within the same CSA layer. The row count after each layer is $\lfloor \frac{2r}{3} \rfloor + (r \bmod 3)$, where r is the current row count, converging to two rows after $O(\log_{1.5} n)$ layers.

Step 3 — Final Addition. Once the tree has reduced the partial products down through successive CSA layers, the final reduction level yields the completed 2n-bit product directly, with carry resolution distributed across the reduction stages rather than deferred to a single addition at the end

Hierarchical Decomposition in this Design Because implementing a flat 64×64 Wallace tree directly would generate an extremely large number of partial products and wires, the multiplier in EdgeCore-V decomposes the 64-bit operands into eight 8-bit slices each. An 8×8 grid of `Wallace_tree_8` sub-modules computes all 64 partial products of the form abs_a$[8i{+}7 : 8i]$ $\times$ abs_b$[8j{+}7 : 8j]$, producing 16-bit results. Each 16-bit partial product is then zero-extended to 128 bits and shifted by $8(i + j)$ bit positions to restore the correct positional weight before the final summation. This hierarchical approach retains the logarithmic depth advantage of the Wallace tree while keeping each leaf instance small and regular, which aids both synthesis and physical implementation on the target FPGA fabric. Each `Wallace_tree_8` sub-module is itself an instance of exactly the 8-bit reduction shown in Figure 3.12, so the full 64-bit multiplier can be understood as 64 copies of this structure feeding into the second-stage shift-and-sum network.

3.8.1.6 Division Unit

Algorithm. The division unit implements the classical *restoring division* algorithm in hardware. Restoring division is an iterative, bit-serial algorithm that computes one quotient bit per clock cycle by the following procedure. Let A be a 65-bit accumulator register, Q hold the developing quotient (initialised with the dividend magnitude), and M hold the divisor magnitude. At each of the 64 iterations:

1. Shift A and Q left by one bit as a combined 129-bit register:

$$A \leftarrow \{A[63:0],\ Q[63]\}, \quad Q \leftarrow \{Q[62:0],\ 0\}$$

2. Perform a trial subtraction: $A_{\text{sub}} \leftarrow A - M$.
3. If $A_{\text{sub}}[64] = 0$ (no borrow, subtraction did not go negative), *accept*: set $A \leftarrow A_{\text{sub}}$ and $Q[0] \leftarrow 1$.
4. Otherwise, *restore*: keep A unchanged and leave $Q[0] = 0$.
5. Decrement the iteration counter N.

After 64 iterations Q holds the quotient magnitude and $A[63:0]$ holds the remainder magnitude. The algorithm is called *restoring* because in the non-accept case the accumulator is restored to its value before the trial subtraction, in contrast to non-restoring variants that accept negative partial remainders.

Sign Handling. Signed division (`DIV`, `REM`) is handled by converting both operands to their absolute values before the algorithm begins, running the unsigned restoring algorithm on the magnitudes, and then applying sign correction to the results. The sign of the quotient is negative when exactly one operand is negative (`Divisor[63] ^ Dividend[63]`); the sign of the remainder follows the dividend. This conversion is performed at the input using two's-complement negation muxes controlled by `Div_Control[0]`: when bit 0 is 0 the operation is signed and the muxes apply $\sim x + 1$ to any negative operand; when bit 0 is 1 the operation is unsigned and both operands pass through unchanged.

Word Operations. For 32-bit word instructions (`DIVW`, `REMW`), `Div_Control[2]` is asserted and the final output is sign-extended from bit 31, identical to the approach used for the word-width ALU instructions.

Corner Cases. The RISC-V specification mandates specific results for two degenerate inputs. Division by zero must return -1 (all ones) for the quotient and the dividend for the remainder, regardless of signedness. Signed overflow — the most-negative integer (`0x8000000000000000`) divided by -1 — must return the most-negative integer for the quotient and zero for the remainder. Both cases are detected in the `Start` state before any iteration begins: the `flag_error` register is asserted and the pre-computed `ResultError`/`RemainderError` values are selected by the error mux, with `Valid` asserted immediately so the pipeline is not stalled.

FSM. The division unit is controlled by a three-state FSM:

- **Start** — detects corner cases (producing an immediate result) or initialises A, Q, M, and N for a normal division, then transitions to `Computing`.

- **Computing** — executes one restoring-division step per clock cycle, decrementing N until $N = 1$, then transitions to `Stop`.
- **Stop** — latches Q and $A[63 : 0]$ into `result_reg` and `remainder_reg`, asserts `Valid`, and returns to `Start`.

The unit is held in reset as long as `Div_Control[3] = 0`: any instruction that is not a division keeps the FSM in the `Start` state and clears all datapath registers, ensuring the unit is ready for the next division without requiring an explicit flush.

3.8.1.7 Multi-Cycle Hazard and Stall Resolution

Why Division Creates a Hazard. The five-stage pipeline assumes that every instruction in the Execute stage produces its result by the end of that cycle, so that the result can be forwarded to a dependent instruction entering Execute in the next cycle. The Wallace-tree multiplier satisfies this assumption because it is purely combinational. The restoring divider does not: it requires up to 66 clock cycles to assert `Valid`, during which the pipeline continues to fetch and decode subsequent instructions. If those instructions attempt to read the destination register of the division before `Valid` is asserted, they will read a stale value from the register file. This is a *multi-cycle data hazard* that forwarding alone cannot resolve, because the correct value does not exist anywhere in the pipeline until the divider finishes.

Detection. The hazard is detected in the `Stalling_Unit` by a single condition:

$$\text{ICU_Busy} = \overline{\text{ICU_Valid}} \;\&\; (\text{Function_SelD} = \texttt{2'b10}) \tag{3.4}$$

`Function_SelD` carries the `Function_Sel` value of the instruction currently in the Decode stage — that is, the instruction immediately behind the dividing instruction in the pipeline. `ICU_Valid` is the `Valid` output of the divider, which remains low throughout the computation. The conjunction therefore asserts `ICU_Busy` for every cycle from the moment a division instruction enters Execute until its `Valid` signal is finally asserted.

Note that `Function_SelD` is checked in the Decode stage rather than the Execute stage. This is intentional: because the divider takes many cycles to complete, the instruction behind the division will have already advanced past Decode by the time the stall is needed. Using the Decode-stage signal ensures the stall is asserted early enough to prevent any subsequent instruction from consuming a stale register value.

Stall Response. When `ICU_Busy` is asserted the stalling unit freezes the entire front end of the pipeline:

$$\texttt{StallF} = 1 \quad \text{(PC held constant)}$$
$$\texttt{StallD} = 1 \quad \text{(IF/ID register held)}$$

`StallE`, `StallM`, and `StallW` are not asserted by `ICU_Busy` (they are only asserted for D-cache stalls), so the division instruction itself continues to advance through Execute, Memory, and Write-Back normally. The pipeline therefore stalls only the stages behind the division; the division instruction and all instructions ahead of it proceed unaffected. When the divider finally asserts `Valid`, `ICU_Busy` falls, the stall signals are de-asserted, and the pipeline resumes normal operation. The division result is then available in the Write-Back stage and can be forwarded via the standard MEM-to-EX or WB-to-EX forwarding paths to any immediately following instruction that depends on it.

Interaction with Other Stall Sources. The stalling unit combines four independent stall sources with a logical OR:

$$\text{StallF} = \text{lwStall} \mid \text{ICU_Busy} \mid \text{icache_stall_F} \mid \text{dcache_stall_M} \tag{3.5}$$

This ensures that a division stall does not mask a simultaneously occurring load-use stall or cache miss, and vice versa. Each source independently contributes to the combined stall signal, and the pipeline remains frozen until all active stall conditions are cleared.

3.8.1.8 MAC Unit

What is a MAC Operation? A Multiply-Accumulate (MAC) operation computes the expression

$$rd \leftarrow rd + rs1 \times rs2 \tag{3.6}$$

in a single instruction. Rather than issuing a separate multiply followed by an add and writing the intermediate product to a temporary register, the MAC instruction reads the current value of the destination register rd, multiplies the two source operands $rs1$ and $rs2$, and accumulates the product directly into rd. The result is written back to rd at the end of the same cycle.

Why a Dedicated MAC Instruction? The MAC operation is the innermost computation of a large class of practically important algorithms. Digital filters, matrix multiplication, dot products, and neural-network inference all reduce to repeated sequences of the form accumulator $+= a_i \times b_i$. On a processor that provides only separate `MUL`

and `ADD` instructions, each such iteration requires two instructions, two register writes, and a temporary register to hold the intermediate product. For a workload such as the MNIST inference benchmark used in this project — which consists almost entirely of multiply-accumulate operations over weight and activation vectors — this overhead is significant.

A fused MAC instruction eliminates the temporary register, reduces instruction count by up to half for accumulation-dominated loops, and removes a register-file write that would otherwise occur every other instruction. The motivation is therefore both performance (fewer instructions, fewer pipeline cycles) and code density (smaller binary, better instruction-cache utilisation). This is particularly relevant for an edge-inference target such as EdgeCore-V, where the dominant workload is fixed-point neural-network computation.

Instruction Encoding. The MAC instruction uses the custom opcode `7'b0001011`, which is reserved by the RISC-V specification for non-standard extensions and does not conflict with any ratified instruction. The encoding is:

Table 3.7: MAC instruction encoding.

Instruction	funct7	rs2	rs1	funct3	rd	opcode
MAC (unsigned)	0000000	rs2	rs1	width	rd	0001011
MAC (signed)	0000001	rs2	rs1	width	rd	0001011

The `funct7` field selects signedness: `0000000` produces an unsigned MAC (`Signed_en = 0`) and `0000001` produces a signed MAC (`Signed_en = 1`). The `funct3[1:0]` field selects the operand width via the `Mode` signal: `00` for 8-bit, `01` for 16-bit, `10` for 32-bit, and `11` for 64-bit. This allows the same instruction to serve SIMD-style accumulation at sub-word granularities without additional opcodes. The `rd` field serves a dual role: it identifies both the destination register and the accumulator source, so three architectural registers ($rs1$, $rs2$, rd) are read from the register file on every MAC instruction.

Control Signals from the Main Decoder. When the main decoder detects opcode `0001011` it asserts `MAC_Sel = 1` and sets `Function_Sel = 2'b00`, directing the ICU to use the ALU adder as the final accumulation stage. `RegWrite` is asserted so that the result is written back to rd. No memory access is performed (`MemWrite = 0`, `MemRead = 0`), and no branch or jump signals are asserted. The `Signed_en` and `Mode` signals are passed through the pipeline registers to the ICU alongside `MAC_Sel`.

Datapath Implementation inside the ICU. The MAC operation is implemented entirely within the ICU without adding any new functional units. Two multiplexers —

`MAC_MUX_1` and `MAC_MUX_2` — are inserted between the forwarding network outputs and the ALU inputs, controlled by `MAC_SelE`:

- **`MAC_MUX_1`**: when `MAC_SelE = 0`, passes `SrcA` (the value of $rs1$) to the ALU's first input. When `MAC_SelE = 1`, replaces `SrcA` with `MulResult` — the product $rs1 \times rs2$ just computed by the Wallace-tree multiplier.
- **`MAC_MUX_2`**: when `MAC_SelE = 0`, passes `SrcB` (the value of $rs2$ or an immediate) to the ALU's second input. When `MAC_SelE = 1`, replaces `SrcB` with `OutToMuxC` — the forwarded value of rd, the accumulator operand.

With both muxes selecting their MAC paths, the ALU adder computes `MulResult` + `OutToMuxC`, which is exactly $rs1 \times rs2 + rd$ as required by Equation 3.6. The Wallace-tree multiplier and the ALU adder therefore operate in the same clock cycle, making the MAC instruction single-cycle despite performing both a multiplication and an addition.

The Third-Register Hazard. Standard R-type and I-type instructions read at most two source registers ($rs1$ and $rs2$). The MAC instruction reads a third register — the current value of rd — as the accumulator input. This creates a hazard scenario that the standard two-operand forwarding network (`ForwardAE`, `ForwardBE`) does not cover: if a preceding instruction writes to the same register that the MAC instruction uses as its accumulator, the value of rd read from the register file at the Decode stage may be stale.

Consider the sequence:

```
ADD  x5, x1, x2       # writes x5
MAC  x5, x3, x4       # reads x5 as accumulator --- hazard!
```

When the MAC instruction is in the Execute stage, the `ADD` result for `x5` is either still in the MEM stage (EX-to-EX forward needed) or in the WB stage (MEM-to-EX forward needed). In either case `OutToMuxC` must be sourced from the in-flight result rather than from the register file.

Resolution via ForwardCE. A dedicated 2-bit forwarding signal `ForwardCE` is added to the Forwarding Unit specifically to handle the MAC accumulator operand. It follows the same priority logic as `ForwardAE` and `ForwardBE`, comparing the MAC instruction's rd address (carried through the pipeline as `RdE`) against the destination registers of the instructions in the Memory and Write-Back stages:

$$\texttt{ForwardCE} = \texttt{2'b10} \quad \text{if} \quad \texttt{RdM} = \texttt{RdE} \;\&\; \texttt{RegWriteM} = 1 \;\&\; \texttt{RdM} \neq 0 \quad \text{(MEM-stage forward)} \tag{3.7}$$

$$\texttt{ForwardCE} = \texttt{2'b01} \quad \text{if} \quad \texttt{RdW} = \texttt{RdE} \;\&\; \texttt{RegWriteW} = 1 \;\&\; \texttt{RdW} \neq 0 \quad \text{(WB-stage forward)} \tag{3.8}$$

$$\texttt{ForwardCE} = \texttt{2'b00} \quad \text{otherwise} \quad \text{(no hazard, use register file value)} \tag{3.9}$$

`ForwardCE` drives the select input of the `OutToMuxC` multiplexer. When `ForwardCE = 2'b10`, `OutToMuxC` is sourced from the EX/MEM pipeline register (the result of the preceding instruction currently in the Memory stage). When `ForwardCE = 2'b01`, it is sourced from the MEM/WB register (the result currently in Write-Back). When `ForwardCE = 2'b00`, the register file output for *rd* is used directly. In all three cases the correct accumulator value arrives at `MAC_MUX_2` without stalling the pipeline, preserving the single-cycle throughput of the MAC instruction.

3.9 Summary

The processor architecture of EdgeCore-V was developed as four successive layers, each adding a well-defined capability on top of a verified base.

The **single-cycle core** established functional correctness for the full RV64I ISA. Every instruction was validated in simulation against an ISA reference model before the pipeline was attempted. The clean separation between the main decoder (opcode → control signals) and the ALU decoder (ALUOp + funct fields → ALUControl) proved important: when porting to the pipeline, these two blocks required no modification.

The **five-stage pipeline** increased instruction throughput by overlapping execution. The hazard unit resolves data hazards through forwarding — EX-to-EX and MEM-to-EX paths cover all cases except load-use, which requires a one-cycle stall. Control hazards from taken branches are resolved by flushing two pipeline stages; the consistent two-cycle penalty motivated the addition of the branch predictor.

The **branch predictor** combines a 64-entry direct-mapped BTB with a per-entry 2-bit saturating counter. Predictions are made in the Fetch stage and corrections are applied in the Execute stage. For loop-dominated workloads the predictor quickly reaches Strongly Taken and incurs no misprediction penalty during the body of the loop. The discovery and resolution of the misprediction–illegal-instruction conflict — where a wrong-path instruction could trigger a spurious exception — is a concrete example of the subtle

interactions that only become visible when prediction and exception handling are integrated.

The **exception and CSR subsystem** adds machine-mode trap infrastructure: seven exception types, five CSRs, full CSR read-modify-write semantics, ECALL/EBREAK/MRET support, and a priority encoder that enforces precise exception semantics across pipeline stages.

Together these components constitute the base integer execution core of EdgeCore-V. The memory hierarchy — instruction cache, data cache, scratchpad memory, DMA, and AXI interconnect — is built on top of this core and is described in the following chapter.

Chapter 4

Memory Subsystem

4.1 Overview

In any pipelined processor, the memory subsystem – not the datapath – is ultimately what determines sustained performance. A single-cycle ALU operation is of little value if the instruction that feeds it, or the operand it consumes, has to wait tens or hundreds of cycles to arrive from external memory. This is the classical *Von Neumann bottleneck*: a single shared path between the processor and memory, whose bandwidth and latency place a hard ceiling on achievable throughput regardless of how aggressively the compute pipeline itself is optimized. EdgeCore-V's core pipeline was designed and verified first (Chapter 2–Chapter 3), but its real-world performance is governed almost entirely by how instructions and data are supplied to it – which is the subject of this chapter.

Rather than committing to a single memory architecture up front, EdgeCore-V was developed through three successive, deliberately incremental versions, each one addressing a specific weakness exposed by the version before it:

- **Version 1 – Direct AXI Access.** The CPU fetches instructions through a dedicated instruction cache, but all data accesses go directly to main memory over AXI, with no data caching at all. This establishes a correct, simple baseline and isolates the cost of *not* exploiting data locality.
- **Version 2 – I-Cache + D-Cache.** A data cache is added alongside the instruction cache, both sharing a single AXI master interface through an internal arbitration scheme. This version quantifies how much performance is recoverable simply by exploiting temporal and spatial locality in data accesses, using a conventional cache hierarchy.

- **Version 3 – Hybrid (I-Cache + D-Cache + SPM + DMA).** A software-managed scratchpad memory (SPM) and a DMA engine are added on top of Version 2. This targets exactly the class of access pattern that caches handle poorly: large, predictable, streaming data transfers (such as the weight and activation tensors of the MNIST inference workload used in Chapter 7), which benefit from being moved in bulk, deterministically, and *off* the critical path of the CPU pipeline entirely.

The remainder of this chapter is organized bottom-up rather than version-by-version. Section 4.2 first lays out the general theory of cache organization – placement, replacement, and write policies – independent of any specific implementation choice, since these are standard design axes with well-understood trade-offs regardless of which point on each axis a particular cache picks. Section 4.3 then covers the AXI protocol used throughout the design at the signal level. Only after this shared foundation is in place do Sections 4.4–4.6 walk through the three versions themselves, at which point each version simply states which point on each axis it adopts and why, rather than re-deriving the underlying algorithm from scratch. This structure is intentional: it is easier to judge *why* EdgeCore-V's D-cache uses write-back once write-back has already been contrasted against write-through in the abstract, than to have that contrast folded into the description of one specific cache.

4.2 Memory Hierarchy Fundamentals

4.2.1 The Locality Principle and the Memory Hierarchy

Caches exist because of a large and persistent gap between processor cycle time and DRAM access latency, and because real programs do not access memory uniformly at random. Two empirical regularities – collectively, the *principle of locality* – make caching effective in practice:

- **Temporal locality:** a memory location that has been accessed recently is likely to be accessed again soon (e.g. a loop counter, or a weight value reused across every row of a matrix multiplication).
- **Spatial locality:** a memory location near one that was just accessed is likely to be accessed soon as well (e.g. the next instruction in sequence, or the next element of an array).

A cache exploits both forms simultaneously: it retains recently-used data (temporal locality) and, by fetching an entire aligned block around the requested address rather than a single word (spatial locality), it front-loads the cost of nearby future accesses into the single access that caused the miss. Every design decision discussed in this section –

how a block is placed, what is evicted to make room for a new one, and when a modified block is written back to memory – exists to make that exploitation as effective as possible for a given amount of hardware.

4.2.2 Placement Policy: How a Block Maps to the Cache

When a memory block is brought into the cache, it must be placed in one of the cache's storage locations, and a rule is needed both to place it and to later find it again. Three placement strategies exist, forming a spectrum:

- **Direct-mapped.** Each memory block maps to exactly one possible location in the cache (typically via the low-order bits of its address). This is the cheapest option – only one tag comparator is needed per access – but it is also the most vulnerable to *conflict misses*: if two frequently-used addresses happen to map to the same location, they will repeatedly evict each other even though the cache as a whole may be far from full.
- **Fully associative.** A block may be placed in *any* location in the cache, with every location's tag compared against the incoming address in parallel. This eliminates conflict misses entirely, since only genuine capacity exhaustion can force an eviction, but the cost of the parallel tag comparison grows with the number of locations, making it impractical for anything but very small caches.
- **Set-associative.** A middle ground: the cache is divided into sets, each memory block maps to exactly one set (as in the direct-mapped case), but within that set the block may occupy any one of N *ways* (as in the fully-associative case, but scoped to just that set). This bounds the tag-comparison cost to N comparators while still absorbing the common case of two or three addresses aliasing to the same set – N-way associativity tolerates up to N simultaneously "hot" addresses per set before conflict misses reappear.

In practice, increasing associativity yields diminishing returns: moving from 1-way to 2-way typically eliminates the majority of conflict misses for a given workload, and further increases toward 4-way or 8-way continue to help but by successively smaller margins, while the comparator and replacement-state cost continues to grow linearly (or worse) with N. Section 4.4 states where EdgeCore-V's caches sit on this spectrum and why.

4.2.3 Replacement Policy: What to Evict on a Miss

In any placement scheme other than direct-mapped, a miss that must be placed into an already-full set requires choosing a victim to evict. Three replacement policies are commonly used:

- **Random.** A way is chosen arbitrarily. This is trivial to implement (no state at all is needed per set) but ignores locality completely, and can evict a way that was, in fact, just used.

- **FIFO (first-in, first-out).** The way that has resided in the set the longest, regardless of how recently it was *accessed*, is evicted. This requires only a per-set pointer or counter, but – like random replacement – it can evict a heavily-reused way simply because it happens to be the oldest resident, since FIFO tracks residency time, not access recency.

- **Least-Recently-Used (LRU).** The way that has gone the longest *without being accessed* is evicted. This directly targets temporal locality – the empirical regularity described in Section 4.2.1 – and is the standard choice when a workload is expected to exhibit meaningful reuse. Exact LRU requires tracking a full recency ordering across all N ways of a set (an $N!$-permutation, i.e. $\lceil \log_2 N! \rceil$ bits per set for exact tracking), which becomes expensive as N grows; in practice, most designs – including this one – instead use an **approximate LRU** scheme that captures the same recency intuition with substantially less per-set state. One common approximation, used in this design (Section 4.4), is a small saturating counter per way that is bumped up on every access to that way and decremented on every access to a *different* way in the same set, with the way holding the lowest counter value chosen as the victim; this tracks relative recency closely without requiring an exact ordering to be maintained.

For instruction and data streams with strong loop-driven reuse – the common case for the RISC-V test programs and MNIST inference workload used throughout this thesis (Chapter 5–Chapter 7) – LRU or an approximation of it consistently outperforms random and FIFO replacement, at the cost of the extra per-way state needed to track recency.

4.2.4 Write Policy: Handling Stores

A data cache (unlike a read-only instruction cache) must additionally decide how a store instruction is reflected to the backing memory. Two independent decisions are involved.

On a write hit: write-through vs. write-back.

- **Write-through** propagates every store immediately to main memory, in addition to updating the cache line. The cache and memory are therefore never out of sync, which simplifies eviction (a line can always simply be discarded, since memory is already current) and simplifies any downstream consumer of memory contents. The cost is that every single store generates external memory traffic, regardless of whether that same location is about to be written again.

- **Write-back** updates only the cache line on a store, and marks it *dirty*; the write is deferred and only actually reaches memory when the line is eventually evicted (or explicitly flushed). This can substantially reduce memory traffic for addresses that are written repeatedly before eviction – exactly the pattern of an accumulator in a tight loop – at the cost of needing a per-line dirty bit and a write-back step inserted into the eviction sequence.

On a write miss: write-allocate vs. no-write-allocate.

- **Write-allocate** brings the missed line into the cache (as if it were a read) before applying the store. This is the natural pairing with write-back, since it means subsequent accesses to the same line – reads or writes – can now hit.
- **No-write-allocate** sends the store directly to memory without allocating a line for it. This is the natural pairing with write-through, for workloads where a written address is unlikely to be read again soon.

Section 4.5 states which combination EdgeCore-V's D-cache uses and the reasoning behind that choice in the context of the AXI interconnect it shares with the I-cache.

4.2.5 Hardware-Managed vs. Software-Managed Memory

Everything described above is *hardware-managed*: the cache controller decides, transparently to software, what is resident and what is evicted, based on recency heuristics applied to whatever addresses happen to be touched. This is the right model when access patterns are irregular or not known ahead of time. An alternative model, *software-managed* memory (a scratchpad, or SPM), instead exposes a region of on-chip memory directly into the address space with no tag comparison, no hit/miss logic, and no automatic eviction at all: software (or, as in this design, a DMA engine acting on software's behalf) decides explicitly what resides there and when. The trade-off is the mirror image of the hardware-managed case: an SPM guarantees fixed, predictable access latency and immunity to eviction by an unrelated access, but only for data that is placed there deliberately, and it offers no benefit at all for data whose access pattern was not anticipated in advance. Section 4.6 discusses why EdgeCore-V adds an SPM alongside, rather than instead of, its caches.

4.3 AXI Bus Interface

All three versions of the memory subsystem communicate with main memory through the AMBA AXI protocol. AXI was chosen over a custom or simplified memory interface for three reasons. First, it is the interconnect standard used by the Zynq Processing System

(PS7)'s AXI ports, meaning any IP integrated onto the target platform – including third-party memory controllers – speaks AXI natively, so no protocol-translation layer is needed at the system boundary. Second, AXI supports burst transactions, which is essential once cache-line fills are introduced in Version 2 and becomes even more valuable for the bulk SPM transfers of Version 3, since an entire cache line or SPM tile can be moved as one addressed burst rather than as a sequence of individually-addressed single-word transfers. Third, AXI's split address/data channel structure decouples the request and response paths, allowing multiple masters (I-cache, D-cache, and, from Version 3 onward, the DMA engine) to eventually share a single physical memory port without the protocol itself forcing unrelated transfers to serialize at the electrical level.

4.3.1 AXI Channels and Signals

AXI separates read and write into entirely independent channel groups, and further separates address from data within each direction, giving five channels in total:

Write Address Channel (AW) – master to slave.

- `AWADDR` – the address of the first byte of the write burst.
- `AWLEN` – burst length (number of data beats minus one).
- `AWSIZE` – the width of each beat (e.g. $2^{\texttt{AWSIZE}}$ bytes).
- `AWBURST` – burst type: `FIXED` (same address every beat, used for e.g. a FIFO-mapped peripheral), `INCR` (address increments by the beat size each beat – the type used for cache-line fills and SPM transfers in this design), or `WRAP` (increments then wraps at a boundary).
- `AWVALID` / `AWREADY` – the handshake pair: the master asserts `AWVALID` once address and control fields are stable, and the transfer is considered accepted only on a cycle where both `AWVALID` and `AWREADY` are high simultaneously.

Write Data Channel (W) – master to slave.

- `WDATA` – the data beat itself (128 bits wide in this design).
- `WSTRB` – a byte-enable mask, one bit per byte of `WDATA`, allowing a beat to update only a subset of its bytes.
- `WLAST` – asserted on the final beat of the burst.
- `WVALID` / `WREADY` – handshake pair, same rule as above: a beat transfers only when both are high on the same cycle.

Write Response Channel (B) – slave to master.

- `BRESP` – completion status of the write (e.g. `OKAY`, or an error response).
- `BVALID` / `BREADY` – handshake pair confirming the master has accepted the response.

Read Address Channel (AR) – master to slave. Mirrors the Write Address Channel exactly: `ARADDR`, `ARLEN`, `ARSIZE`, `ARBURST`, `ARVALID`/`ARREADY`.

Read Data Channel (R) – slave to master.

- `RDATA` – the returned data beat.
- `RRESP` – completion status for that beat.
- `RLAST` – asserted on the final beat of the read burst.
- `RVALID` / `RREADY` – handshake pair.

The VALID/READY handshake rule. Every AXI channel follows the same rule regardless of direction: the source of a channel asserts `VALID` once its outgoing signals are stable and holds them stable until the transfer is accepted; the destination asserts `READY` whenever it is able to accept a transfer; and the transfer itself occurs only on a clock edge where both signals are simultaneously high. Neither side may wait for the other to assert its own signal first – a `VALID` may be asserted independently of `READY`, and vice versa – which is what allows AXI masters and slaves to be designed and verified independently of each other's internal timing.

4.3.2 Burst Sizing in This Design

The masters in this design use a 128-bit (16-byte) AXI data width and a 4-bit burst-length encoding (AXI3-style, capping a single burst at 16 beats, i.e. 256 bytes). Two consequences of this follow directly and recur throughout this chapter:

- A 64-byte cache line (Section 4.4) fills or writes back in exactly **4 beats** of a single `INCR` burst.
- A DMA transfer (Section 4.6) whose requested size exceeds 256 bytes cannot be issued as one AXI burst, and must instead be split into multiple sequential bursts – a consequence handled explicitly by the DMA controller's burst-splitting logic (Section 4.6).

4.3.3 Two-Level Arbitration

Because multiple independent requesters (I-cache, D-cache, and, from Version 3 onward, the DMA engine) need access to a single physical memory port, EdgeCore-V uses a **two-level arbitration hierarchy** rather than a single flat arbiter across all requesters:

1. At the lower level, the `Caches_Axi_Bridge` module multiplexes the I-cache and D-cache into one combined AXI master (Section 4.5).

2. At the upper level, a separate `Arbiter` module multiplexes that combined cache master against the DMA engine's own AXI master, presenting a single, final master interface to main memory (Section 4.6).

A third, independent arbitration decision exists entirely outside this AXI hierarchy, local to the SPM itself, and is discussed separately in Section 4.6.5, since it governs a different resource (the SPM's internal memory array) with a different – and, notably, opposite – priority order to either of the two levels above.

4.4 Version 1: I-Cache with Direct AXI Memory Access

In this first version, instruction fetch is served by a dedicated instruction cache, while every data memory access (every load and store issued from the pipeline's memory stage) bypasses caching entirely and is sent directly to main memory over AXI. This asymmetry is deliberate: instruction fetch is overwhelmingly sequential and highly reusable within loops, so even a simple cache yields an immediate, low-risk win, whereas introducing data caching brings with it the write-policy and eviction complexity described in Section 4.2 – complexity this version intentionally defers. Version 1 therefore exists to answer a specific question in isolation: *how much does an uncached data path cost*, independent of any confound from an imperfectly-tuned D-cache.

4.4.1 I-Cache Architecture

Placed against the placement-policy spectrum of Section 4.2, the instruction cache is **4-way set-associative**, with **64 sets** and a **64-byte (512-bit) line size**, giving a total capacity of $64 \times 4 \times 64\text{B} = 16\text{KB}$. Each parameter reflects a specific point on that spectrum, chosen for concrete reasons rather than convention:

- **Why 4-way associativity.** A cache's associativity is a trade-off between conflict misses and hardware complexity. A direct-mapped cache suffers from frequent conflict misses because multiple active instruction blocks that map to the same set continually evict one another. Increasing associativity allows several frequently used blocks to coexist within the same set, substantially reducing these conflicts.

However, each additional way requires another parallel tag comparator and more complex replacement logic, increasing both area and timing cost. A 4-way organization provides most of the achievable reduction in conflict misses while keeping the comparison and replacement hardware relatively small, making it a widely adopted balance between performance and implementation cost.

- **Why a 64-byte cache line.** Cache line size balances spatial locality against refill overhead. Larger lines exploit sequential instruction execution by fetching neighboring instructions together, but excessively large lines waste memory bandwidth and cache capacity when execution branches before the entire line is used. A 64-byte line stores sixteen 32-bit RISC-V instructions, which is sufficient to cover most sequential execution regions encountered in the benchmark programs used throughout this thesis. It also aligns naturally with the 128-bit AXI interface, requiring exactly four data beats per cache-line refill and allowing each refill to be completed using a single AXI burst.

- **Why 64 sets (16KB capacity).** With a fixed 4-way associativity and 64-byte line size, the number of sets is determined by the total cache capacity. The primary design choice is therefore the 16KB capacity, which was selected as a balance between cache performance and FPGA resource usage. During development, this capacity was sufficient to avoid noticeable thrashing for the target workloads while keeping Block RAM consumption modest, leaving resources available for the D-cache and the scratchpad memory in the other processor configurations.

Tag array, data array, and valid bits. Each of the 64 sets holds 4 ways, and each way carries a tag field, a valid bit, and its 64-byte data array. On a fetch, the set index selects one of the 64 sets, and the tag of the incoming address is compared in parallel against all 4 ways' stored tags; a match on a valid way constitutes a hit.

Replacement policy chosen: counter-based LRU. Of the three replacement policies introduced in Section 4.2, this design uses the approximate-LRU scheme: a **2-bit saturating counter per way, per set** (00 = least-recently-used, 11 = most-recently-used). On every access to a way, that way's counter is set to 11, and every other way in the same set whose counter is strictly lower is decremented by one (saturating at 0); on a miss, the way whose counter reads 00 is selected as the eviction victim. LRU (rather than random or FIFO) was chosen because it directly targets the temporal locality that instruction fetch exhibits most strongly – loop bodies are revisited repeatedly – and the counter-based approximation was chosen over an exact recency stack because, at only 4 ways, it tracks recency almost as precisely as an exact ordering while requiring far less per-set state than an explicit permutation encoding. The update rule is implemented

once, as a reusable `update_lru_counters` task invoked from the controller's clocked state-machine logic on both the hit path and the miss/refill path, so the same recency-tracking behavior applies consistently regardless of which path triggered the update.

4.4.2 I-Cache Controller FSM

The I-cache controller is a 3-state finite state machine: `IDLE` (awaiting a fetch request), `LOOKUP` (tag comparison against the 4 ways of the indexed set), and `FETCH` (issuing an AXI burst read on miss and waiting for the refill to complete, signaled by `r_valid` from the cache memory array). Because the I-cache is read-only, a missed line can always be overwritten immediately regardless of the victim's prior contents – there is nothing to write back – so `FETCH` transitions directly back to `IDLE` once the refill completes, with no intermediate eviction step.

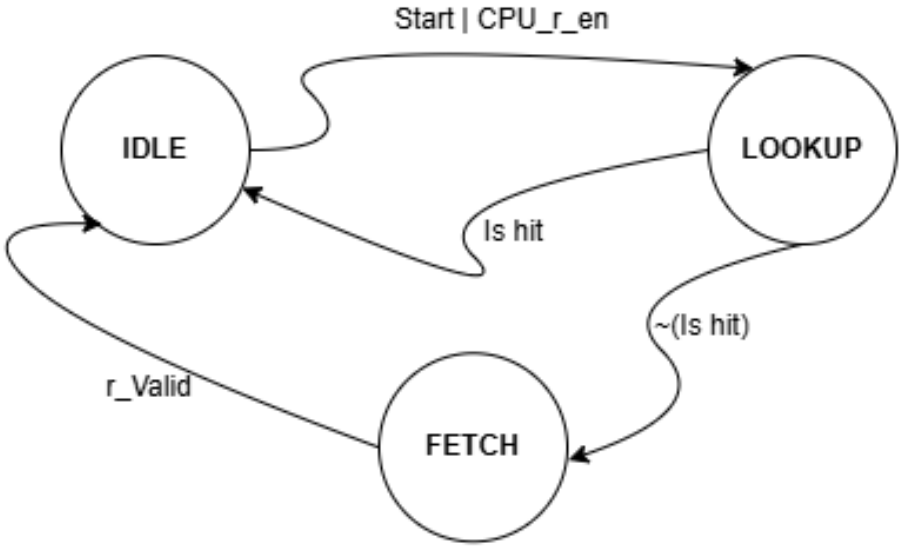


FIGURE 4.1: I-Cache controller FSM

The controller's `stall` output is asserted whenever the next state is not `IDLE`, holding the pipeline's fetch stage until the requested word is either found on a `LOOKUP` hit or supplied by a completed `FETCH`. A watchdog timeout (1000 cycles) additionally guards the `FETCH` state: if an outstanding AXI transaction fails to complete within this window, the controller raises a `bus_error_exception` and returns to `IDLE` rather than stalling the pipeline indefinitely on an unresponsive memory system.

4.4.3 Limitations of Version 1

While Version 1 establishes a correctly-functioning baseline, every single load and store in the program – regardless of whether the same address was just accessed a few cycles prior – incurs the full AXI round-trip latency to main memory. No temporal or spatial

locality in the data stream is exploited at all. For any workload with non-trivial data reuse (which includes essentially all realistic compute kernels, and specifically the MNIST inference benchmark of Chapter 7, where the same weight and activation values are read repeatedly across the inner loops of matrix multiplication), this is expected to dominate the achievable IPC and motivates the addition of a data cache in Version 2.

4.5 Version 2: I-Cache and D-Cache with AXI

Version 2 adds a data cache alongside the existing instruction cache. Both caches now independently issue requests, which are combined into a single AXI master interface by the `Caches_Axi_Bridge` module rather than each cache driving its own independent AXI port.

4.5.1 D-Cache Architecture

The D-cache shares the same placement and replacement choices as the I-cache – 4-way set-associative, 64 sets, 64-byte lines, 16KB total capacity, counter-based LRU – for the same reasons given in Section 4.4. Keeping the I-cache and D-cache geometrically identical also simplified verification, since the same testbench infrastructure and a structurally similar controller FSM could be reused for both.

Write policy chosen: Of the write-policy combinations introduced in Section 4.2, this design uses write-back with per-way dirty bits, paired with write-allocate on a write miss. Write-back was chosen specifically because it reduces contention on the AXI interconnect that this version introduces: a store to the same address performed repeatedly (characteristic of accumulator-style inner loops, as seen in the MAC-based MNIST inference kernel of Chapter 7) only needs to reach main memory once, on eventual eviction, rather than once per store instruction as write-through would require. Write-allocate is the natural pairing, since it ensures a line just written to also becomes available for the reads that typically follow a write to the same working set.

4.5.2 D-Cache Controller FSM

The D-cache controller extends the I-cache's 3-state structure with a fourth state, as the `VICTIM_WRITE` is inserted between `LOOKUP` and `FETCH`: `IDLE` → `LOOKUP` → (`VICTIM_WRITE`, only if the selected victim way is dirty) → `FETCH` → `IDLE`. Unlike the I-cache, a D-cache miss may need to evict a line that has been modified since it was filled, and that modified data must be written back to main memory before the line can be safely overwritten; `VICTIM_WRITE` is skipped entirely when the selected victim is clean, so a clean-line miss costs no more than an I-cache miss. On allocation, the new line's dirty

bit is set directly from whether the triggering access was itself a write, so a line that is filled specifically to service a store is correctly marked dirty from the moment it is installed rather than requiring a separate write-back cycle immediately afterward. The same 1000-cycle watchdog timeout used by the I-cache guards both the `VICTIM_WRITE` and `FETCH` states.

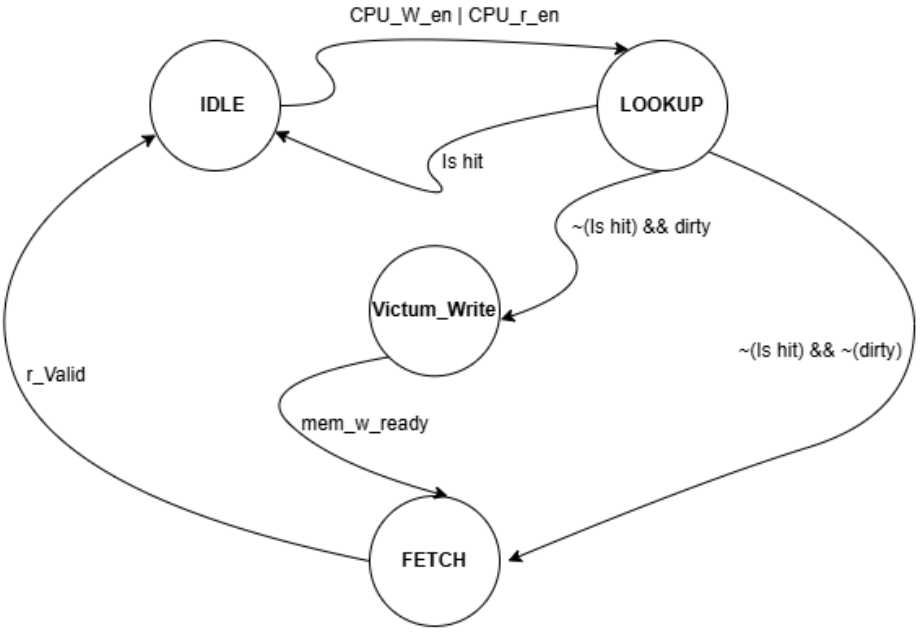


FIGURE 4.2: D-ache controller FSM

4.5.3 Cache Arbitration

Because the I-cache and D-cache issue independent requests toward the same AXI master interface, `Caches_Axi_Bridge` multiplexes the two into a single AXI master, servicing one cache's request to completion before evaluating the other – the two are mutually exclusive at the bridge, with no interleaving of beats from concurrent transactions. Arbitration uses a **fixed priority, evaluated at the bridge's IDLE state**: a pending **D-cache write** is granted first, followed by a pending **D-cache read miss**, with an **I-cache read miss** serviced only if neither D-cache request is pending.

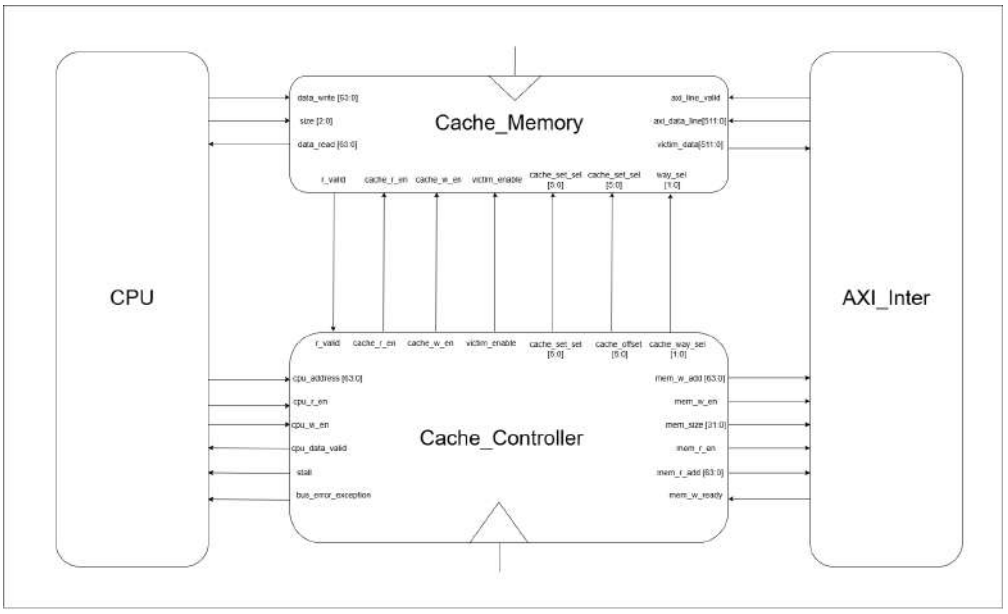


FIGURE 4.3: Caches AXI Bridge.

This ordering favors data requests over instruction requests when both miss in the same cycle. The rationale is that a D-cache miss blocks an instruction that has already been fetched and has already progressed partway through the pipeline, whereas an I-cache miss blocks an instruction that has not yet entered the pipeline at all; resolving the D-cache miss first lets already-in-flight work drain sooner rather than leaving a partially-completed instruction stalled in the memory stage behind an earlier-stage fetch. Within the D-cache's own priority, a pending write is serviced ahead of a pending read miss because, in the case of a dirty eviction, the write-back is already committed as part of the ongoing miss sequence – the victim line has been selected and its data is ready to be pushed out – and the read miss that triggered the eviction cannot complete until that write-back finishes regardless, so delaying it further only prolongs the sequence with no benefit.

Timeout-based bus fault detection. As noted in Section 4.4, both controllers guard every state that awaits an external AXI response with a 1000-cycle timeout, asserting `bus_error_exception` on expiry rather than stalling the pipeline indefinitely. This gives the processor a defined recovery path for a genuinely unresponsive or misconfigured memory system, independent of the ordinary tag-miss handling exercised in Chapter 5.

4.5.4 Limitations of Version 2

Version 2 substantially reduces AXI traffic for data with strong temporal or spatial locality, but a conventional cache is still fundamentally *reactive*, per Section 4.2: it

only holds data after that data has already been accessed once and caused a miss, and it has no way to distinguish data that will predictably be reused (such as a fixed set of neural-network weights known in advance to be needed for the entire duration of an inference pass) from data that is accessed once and never again. Large, regular, streaming transfers – exactly the pattern of loading MNIST weights and activation tensors – gain comparatively little from a cache whose capacity (16KB) may be smaller than the working set, and every first-touch access still pays the full cache-miss AXI latency with no way to move it out of the CPU's critical path in advance. This motivates Version 3.

4.6 Version 3: Hybrid System — I-Cache, D-Cache, SPM, and DMA

Version 3 retains the I-cache and D-cache of Version 2 unchanged, and adds two new components specifically targeted at the class of access pattern identified as Version 2's weakness: a software-managed scratchpad memory (SPM), and a DMA engine capable of moving data into and out of that SPM without occupying the CPU pipeline.

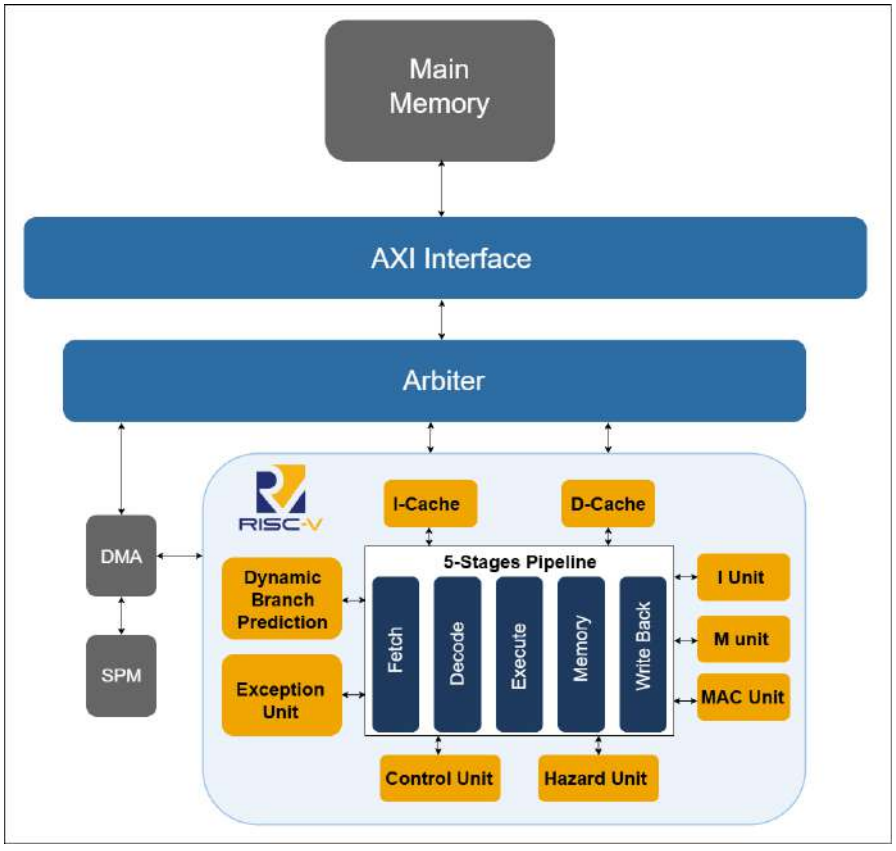


FIGURE 4.4: Full System Diagram

4.6.1 Scratchpad Memory (SPM)

Following the hardware-managed/software-managed distinction introduced in Section 4.2, the SPM is a **software-managed, byte-addressable 8KB** memory (`depth = 8192` bytes in the RTL). It is mapped at the fixed base address `0x8000_0000` (Section 4.6.6) and is accessed by the CPU through ordinary load and store instructions, with no tag comparison, replacement policy, or hit/miss logic. Consequently, every access has deterministic latency, making the SPM well suited for predictable workloads such as tiled neural-network inference.

For the MNIST accelerator presented in this work, weights are transferred explicitly by the DMA engine and remain resident in the SPM until the software replaces them. This deterministic placement eliminates the uncertainty associated with cache replacement and ensures that data required by the current computation remains available throughout tile execution.

Why 8KB. The SPM capacity was selected as a balance between transfer granularity and FPGA resource utilization. Because the processor executes sequentially while the DMA engine operates independently, computation can begin as soon as the first DMA transaction has completed. Increasing the SPM size would encourage larger DMA transfers, reducing opportunities to overlap communication with computation and increasing processor idle time. At the same time, the available BRAM resources on the target FPGA (Chapter 6) limit the practical SPM size. An 8KB SPM is sufficient for the tiled MNIST model used in this work, allowing the largest weight matrix to be processed over multiple DMA transfers while maintaining efficient overlap between data movement and computation.

Dual-port structure. The SPM exposes two independent ports onto the same underlying byte array: a CPU port supporting byte-, halfword-, word-, and doubleword-sized accesses with the same semantics as the D-cache interface, and a dedicated 128-bit DMA port for bulk transfers. Matching the DMA port width to the AXI data width (Section 4.3.2) allows each received AXI beat to be written into the SPM in a single cycle without additional width conversion.

4.6.2 CPU Memory-Request Handshake

Accesses from the CPU to either the SPM or the DMA control registers pass through a single request/valid/ready handshake, implemented by the `CPU_Mem_Interface` module, rather than being issued directly. This module decouples the CPU's raw `outside_re`/`outside_we` signals from the target-specific response latency of the SPM or DMA register file, and

is intentionally simple: a three-state finite state machine with no combinational request path.

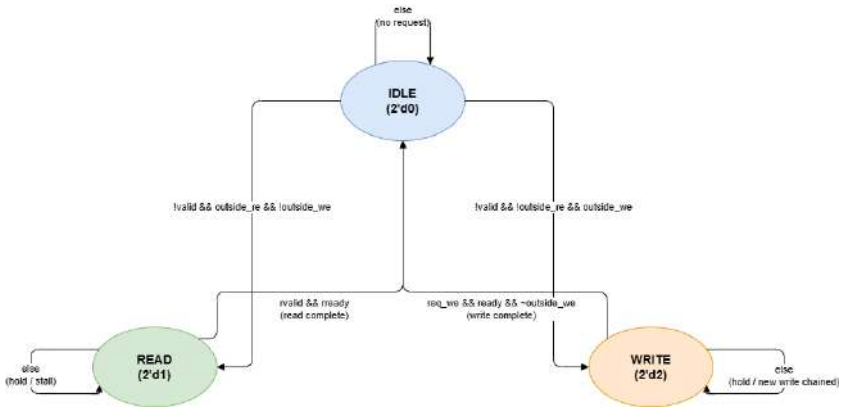


FIGURE 4.5: Finite state machine of the CPU memory-request handshake.

States. The machine idles in IDLE until the CPU issues a read or write. A read request moves it to READ, where it remains until the target returns `rvalid` together with `rready`, at which point the requested data is latched into `cpu_rdata` and the machine returns to IDLE. A write request moves it to WRITE instead, where it remains until the target asserts `ready` for the pending write; if a further write is already pending when the current one completes, it is latched directly into the same WRITE state rather than returning to IDLE first, allowing back-to-back writes to proceed without an idle cycle between them.

Why this belongs to the CPU side, not the target. Placing this handshake in front of both the SPM and the DMA registers, rather than duplicating equivalent logic inside each target, keeps the stall condition the CPU pipeline observes uniform regardless of which memory-mapped region is being accessed. It is this same handshake – specifically, its READ state remaining asserted until `rvalid` – that produces the `dma_read_stall` signal referenced in the DMA discussion below: the CPU pipeline is only stalled while this handshake is actively waiting on a response, not for the entire duration of an unrelated DMA transfer.

4.6.3 DMA Engine

The DMA engine is programmed by the CPU through a small set of memory-mapped control registers (Section 4.6.6) rather than through a dedicated instruction, which keeps the ISA itself unmodified and lets the DMA be driven entirely from ordinary store instructions – as demonstrated in the assembly test program of Chapter 5.

Descriptor fields. Each DMA transfer is described by four software-writable registers: `SRC`, `DST`, `SIZE`, and `CTRL`. A read-only `STATUS` register reports the current DMA state (Section 4.6.6). A transfer is initiated by writing the `START` bit in `CTRL`. This bit is self-clearing: once the DMA controller accepts the request, the register file automatically clears `CTRL[0]`, allowing one store instruction to launch a transfer without requiring software to clear the bit explicitly. While a transfer is in progress (`dma_busy` asserted), writes to the descriptor registers are ignored, preventing modification of an in-flight descriptor; meanwhile, the `STATUS` register continues to reflect the current state of the DMA engine.

Transfer modes and direction encoding. Two directions are supported, selected by `CTRL[1]`: `CTRL[1]`=**1** means memory-to-SPM, used to preload weight/activation tiles into the SPM ahead of the compute phase that will consume them; `CTRL[1]`=**0** means SPM-to-memory, used to write results (or a modified tile, as exercised in the Chapter 5 test program) back out once the CPU has finished operating on the SPM-resident data.

Configuration-error checking. Before starting a transfer, the DMA controller independently validates the descriptor against two conditions, rather than trusting software to have programmed it correctly: the declared direction must be consistent with which of SRC or DST actually falls inside the SPM's address range (`0x8000_0000`–`0x8000_1FFF`) – a memory-to-SPM transfer whose declared destination is *not* in SPM space, or a SPM-to-memory transfer whose declared source is *not* in SPM space, is rejected – and the transfer size must be non-zero. A descriptor failing either check is refused, the controller reports `dma_confg_error` via the STATUS register and never asserts `dma_busy`, and no AXI activity is generated at all. This gives software (and the verification test programs of Chapter 5) an explicit, inspectable failure mode for a misprogrammed descriptor, rather than the DMA silently transferring data to or from the wrong address range.

Burst splitting for arbitrary transfer sizes. As described in Section 4.3.2, a single AXI3 burst on the 128-bit interface is limited to 16 beats (256 bytes). The DMA controller therefore partitions larger transfers into multiple bursts automatically. For each burst, it computes the required number of beats from the remaining byte count: 16 beats whenever at least 256 bytes remain, otherwise the ceiling-divided number of beats required to cover the remaining bytes. After each burst completes, the remaining byte count is updated, and additional bursts are issued until the transfer finishes. This mechanism also supports transfer sizes that are not multiples of the 16-byte AXI beat width, with the final burst sized to cover only the remaining bytes.

CPU/DMA concurrency. DMA transfers do not stall the processor for their entire duration. As described in the handshake of Section 4.6.2, the CPU is only stalled while its own request is actively pending against the DMA registers or the SPM; ...

4.6.4 CPU/DMA Arbitration at the AXI Level

As introduced in Section 4.3, the DMA engine adds a second AXI master alongside the cache subsystem, whose instruction and data caches are already multiplexed by `Caches_Axi_Bridge` (Section 4.5). An `Arbiter` module sits between these two masters and the single AXI memory interface.

The arbiter employs a fixed-priority policy in which the CPU is always favored over the DMA. Read and write channels are arbitrated independently, and a pending CPU request is granted before a pending DMA request. Once a burst has been granted, however, it is allowed to complete without preemption; if a CPU request arrives while a DMA burst is in progress, it is serviced immediately after that burst finishes. Since each DMA burst is limited to 16 beats (Section 4.3.2), the maximum delay imposed on CPU memory requests is bounded by the duration of a single DMA burst.

4.6.5 SPM Port Arbitration: DMA Priority Over the CPU

Arbitration within the SPM is independent of the AXI-level arbiter and follows the opposite priority policy: the DMA port has priority over the CPU port. While `dma_busy` or `dma_start` is asserted, the CPU-facing handshake is held inactive, preventing the CPU from issuing loads or stores to the SPM until the transfer completes.

Unlike the AXI arbiter, which prioritizes overall system responsiveness, the SPM arbiter protects the integrity of data being transferred. Granting the CPU access while the DMA is populating or draining the SPM could expose partially updated data or interfere with an ongoing transfer. Giving the DMA exclusive access therefore guarantees that each tile is transferred atomically before the CPU consumes it. Since DMA transfers are bounded by the tiling strategy described in Chapter 7, the resulting CPU stall is also bounded and predictable.

4.6.6 Memory Map

TABLE 4.1: Memory map of the Hybrid (Version 3) configuration.

Region	Address Range	Notes
Main Memory (via AXI)	`0x0000_0000` – `0x7FFF_FFFF`	Instructions and uncached/D-cached data
Scratchpad Memory (SPM)	`0x8000_0000` – `0x8000_1FFF`	8KB, software-managed, no caching
DMA Control Registers	`0xFFFF_0000` – `0xFFFF_0027`	Memory-mapped, within peripheral space

TABLE 4.2: DMA control register offsets, relative to `0xFFFF_0000`.

Offset	Register	Description
+0x00	SRC	Source address of the transfer
+0x08	DST	Destination address of the transfer
+0x10	SIZE	Transfer size, in bytes
+0x18	CTRL	Bit 0: START (self-clearing); Bit 1: DIRECTION (1 = MEM→SPM, 0 = SPM→MEM)
+0x20	STATUS (read-only)	Bit 0: DONE; Bit 1: BUSY; Bit 2: BUS_ERROR; Bit 3: CONFIG_ERROR

4.6.7 Why the Hybrid Approach?

The Hybrid configuration is not a replacement for caching, but a complement to it: the I-cache and D-cache remain in place precisely because instruction fetch and irregular, hard-to-predict data accesses are still handled better by the hardware-managed model of Section 4.2 than by requiring software to explicitly manage every access. The SPM and DMA engine are added specifically for the subset of the workload – large, regular, known-in-advance data movement – that a cache handles inefficiently by design. This is what makes the MNIST tiling strategy of Chapter 7 possible in the first place: weight tiles are DMA'd into the SPM ahead of the computation that needs them, while the surrounding, less predictable control-flow and intermediate data accesses continue to be served by the D-cache exactly as in Version 2. Chapter 7 quantifies the degree to which this combination outperforms Versions 1 and 2 on the identical workload.

4.7 Summary and Version Comparison

TABLE 4.3: Feature comparison across the three memory subsystem versions.

Feature	V1	V2	V3 (Hybrid)
I-Cache (16KB, 4-way, 64 sets, counter-LRU)	Yes	Yes	Yes
D-Cache (16KB, 4-way, 64 sets, counter-LRU)	No	Yes	Yes
Write policy	–	Write-back, write-allocate	Write-back, write-allocate
SPM (8KB, software-managed)	No	No	Yes
DMA Engine (with config-error checking)	No	No	Yes
AXI arbitration levels	0	1 (I$/D$)	2 (I$/D$, then CPU/DMA)
SPM-local arbitration	N/A	N/A	DMA over CPU
Data bandwidth for irregular access	Low	Medium	Medium
Data bandwidth for regular/streaming access	Low	Medium	High

Each version was a deliberate, minimal increment over the last, built on the shared theoretical foundation of Section 4.2: Version 1 established correctness and quantified the cost of a fully-uncached data path; Version 2 added conventional caching – a write-back, write-allocate D-cache and a fixed-priority arbitration scheme favoring already-in-flight data requests – to recover locality-driven performance; and Version 3 added a software-managed SPM and a self-validating DMA engine, arbitrated onto the shared AXI port via a second, CPU-priority arbitration level, while the SPM's own local port instead favors the DMA, reflecting the different kind of contention each arbitration point is resolving. Of the three, the **Hybrid configuration (Version 3)** is recommended as

the target architecture for workloads resembling the MNIST inference benchmark used throughout this thesis, since it is the only version able to combine deterministic, cache-miss-free access for known data movement with the continued benefit of conventional caching for everything else. The extent of this advantage, measured in IPC, cycle count, and stall breakdown, is presented in Chapter 7.

Chapter 5

Verification and Testing

5.1 Verification Strategy

The verification of EdgeCore-V followed a bottom-up methodology: individual RTL blocks (ALU, register file, cache controllers, AXI bridge, DMA engine) were verified in isolation before being integrated into the full pipeline, and the full pipeline was in turn verified across its three configurations (Direct, Caches, Hybrid Caches+SPM). This staged approach made it possible to localize bugs to a specific block before they could propagate and manifest as confusing symptoms several stages downstream — a distinction that mattered in practice, since several of the hardest bugs encountered during development (e.g. I-cache stall propagation, DMA addressing mismatches) would have been far harder to isolate had unit-level correctness not already been established first.

Throughout the project, the Vivado XSIM simulator was used for both functional verification and performance evaluation. During development, XSIM enabled waveform-level inspection of individual modules as well as integrated system behavior, facilitating the verification and debugging of the processor at each design stage. The same simulation environment was also used for full-system simulations and benchmarking, providing a consistent verification workflow from module-level testing to complete processor evaluation.

Rather than rely solely on directed unit testbenches, the main validation vehicle for the pipeline as a whole was a set of hand-written RISC-V assembly programs, each purpose-built to exercise a specific mechanism (an instruction class, a hazard, a cache behavior, an exception path, etc.) rather than to represent realistic application code. This is an important distinction to make explicit: none of the programs in this chapter compute anything meaningful in a functional sense — their sole purpose is to place the processor into a specific, known state (a particular forwarding condition, a particular

cache line layout, a particular DMA handshake, a particular trap cause) so that the resulting waveform can be checked against a hand-computed expected outcome. Where a program does resemble something like a "real" workload (e.g. the MNIST inference benchmark), that is reserved for Chapter 7 and is not part of this verification chapter.

For every test described below, the same evidence pattern is used: the expected result is stated first (derived by hand from the instruction semantics), the RTL is simulated, and a waveform screenshot showing the relevant signals (register file, memory, CSRs, or control signals as appropriate) is included as visual confirmation that the observed behavior matches the expected one.

5.2 RISC-V Assembly Test Programs

As noted in Section 5.1, each program below is a synthetic stress test rather than a meaningful computation: it exists to put the processor into one specific, checkable condition. The full source of each program is included in Appendix A; only the relevant excerpts and expected/actual outcomes are discussed here.

5.2.1 Arithmetic, Logic, and Custom Instruction Tests

Base ISA arithmetic/logic/shift (`RV64I.asm`). The opening section of `RV64I.asm` stresses the full arithmetic and logical instruction set in a single linear sequence: R-type and I-type arithmetic (`add`, `sub`, `addi`), signed and unsigned set-less-than (`slt`, `sltu`, `slti`, `sltiu`), bitwise logic (`and`/`or`/`xor` and their immediate forms), and all six shift variants including the RV64-specific 32-bit `"w"` shifts (`slliw`, `srliw`, `sraiw`). Expected values for every instruction are annotated inline in the source (Appendix A) and were cross-checked against the resulting register-file waveform.

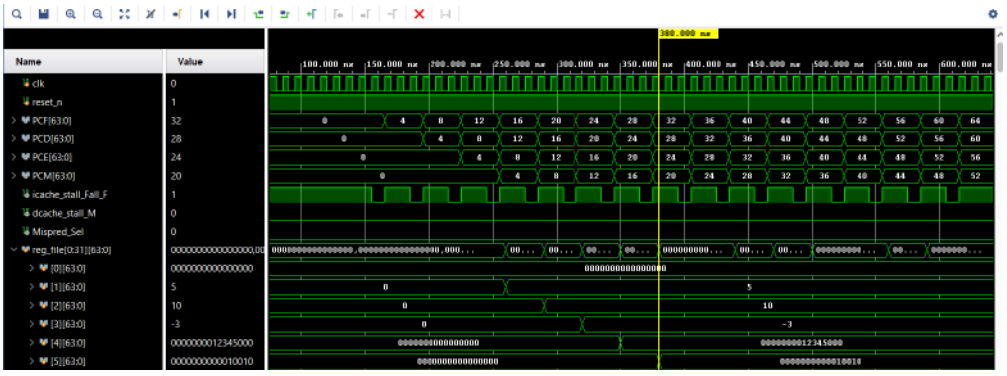


FIGURE 5.1: Setup Base Values, Section Of `RV64I.asm`.

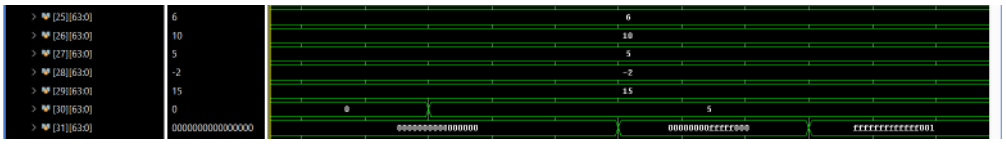


FIGURE 5.2: Arithmetic Operations, Section Of `RV64I.asm`.

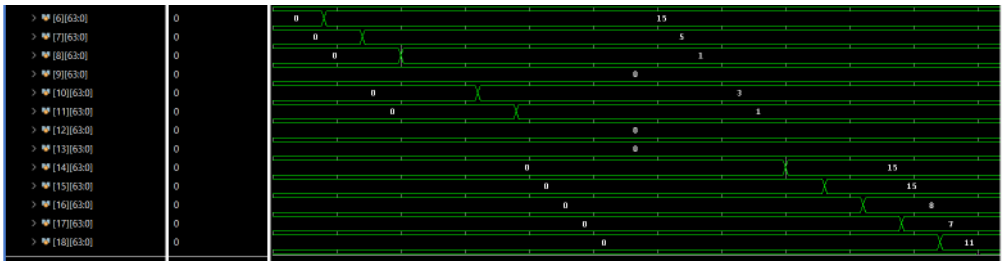


FIGURE 5.3: Logical operations, section of `RV64I.asm`.

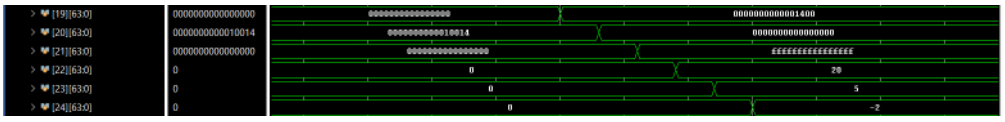


FIGURE 5.4: Shift operations (32-bit and 64-bit), section of `RV64I.asm`.



FIGURE 5.5: Memory Test (Base address = 0x800), Section Of `RV64I.asm`.

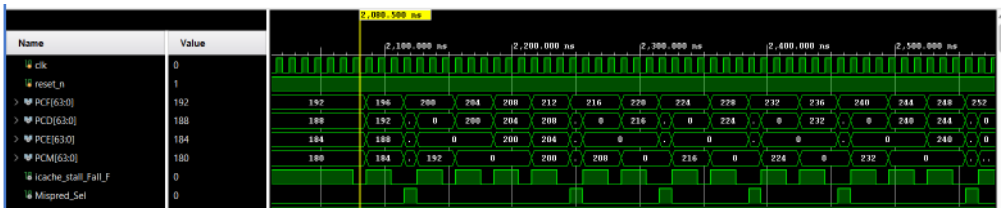


FIGURE 5.6: Branch Tests Section Of `RV64I.asm`.

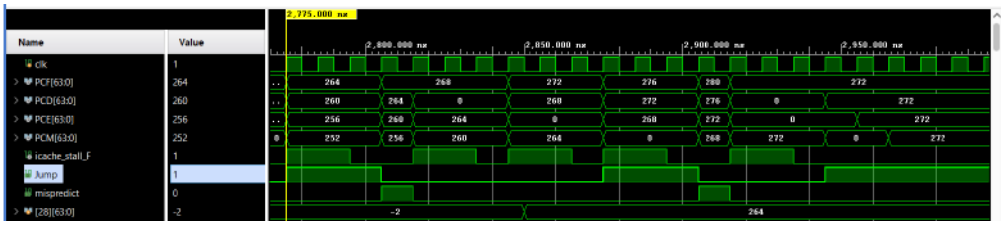


FIGURE 5.7: Jump Tests, Section Of `RV64I.asm`.

Custom MAC instruction (`mac.s`). EdgeCore-V's custom multiply-accumulate extension was verified separately using `mac.s`, which exercises both the unsigned (`macwu`) and signed (`macw`) variants. The signed case is the more interesting of the two: it accumulates the product of `x5 = 8` and `x4 = -9` into `x6 = 5`, deliberately mixing a negative operand into the multiply so that an incorrect sign-handling implementation in the custom MAC datapath (e.g. treating the multiply as unsigned internally) would produce a visibly wrong accumulator value rather than one that happens to match by coincidence.



FIGURE 5.8: MAC Unsigned Tests, Section Of `MAC.asm`.

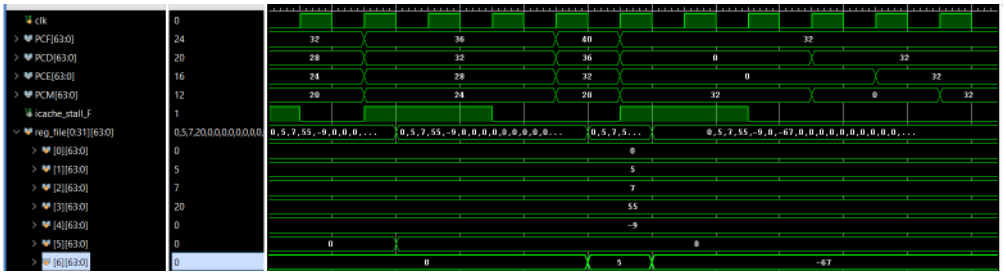


FIGURE 5.9: MAC Signed Tests, Section Of `MAC.asm`.

5.2.2 Memory Access Tests

D-cache line packing and hazard stress (`cache.s`). `cache.s` stresses the D-cache specifically rather than the ISA in general. All four store widths are packed into consecutive offsets of a single 64-byte line (base address `0x800`, offsets `0`, `8`, `16`, `24`), so a single cache-line fill/write-back must correctly hold byte-, half-, word-, and doubleword-granularity data simultaneously without one write corrupting an adjacent field. Immediately after each store, the corresponding value is read back through both its signed

and unsigned load form (e.g. the halfword store of `-2` is read back as both `-2` via `lh` and `65534` via `lhu`), and a doubleword loaded from the cache is fed directly into an `addi` on the next instruction, which additionally stresses the load-to-use forwarding/stall path discussed in Section 3.3 in the specific context of a cache hit rather than a raw register dependency.



FIGURE 5.10: D-Cache Test Output, Section Of `caches.s`.

5.2.3 Hybrid Test

Hybrid Caches+SPM round trip (`hybrid_test.asm`). This program is the primary integration test for the Hybrid configuration's D-cache, DMA engine and scratchpad memory. It is structured as a full round trip:

1. Two ordinary D-cache loads establish a baseline (confirming the cache path still functions normally in the Hybrid configuration).

2. A DMA descriptor is programmed (source, destination, transfer size) and the transfer is kicked off by writing the control register – this is the point at which the pipeline is expected to stall until the DMA engine reports completion.

3. Once the stall releases, two further D-cache loads confirm the CPU can continue issuing memory accesses immediately after the DMA completes, and the DMA status register is read back explicitly.

4. The CPU then reads the SPM directly to confirm the transferred data matches the source memory contents exactly, byte-for-byte.

5. The SPM contents are then overwritten using a deliberate mix of widths (doubleword, halfword, word, byte) and read back to confirm the SPM itself handles mixed-width access correctly, independent of the cache.

6. Finally, a second DMA transfer moves the modified SPM contents back to main memory, and the DMA status is polled again, verifying the reverse direction of the same datapath.

Because every stage of this program depends on the previous one completing correctly, a failure at any point (stall not asserting, DMA moving the wrong data, SPM width handling, or the reverse transfer) is immediately visible as incorrect data at the following stage, which made this test program especially effective for isolating the DMA-addressing and stall-propagation bugs described in Chapter 3.

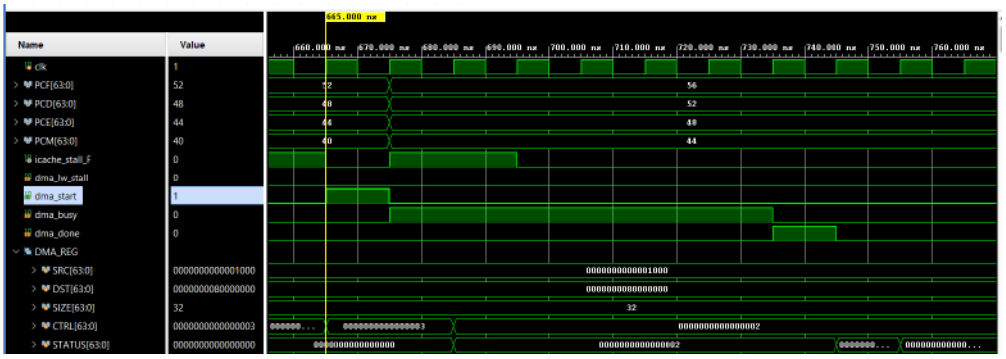


FIGURE 5.11: DMA init to load SPM, Section Of `hybrid_test.asm`.

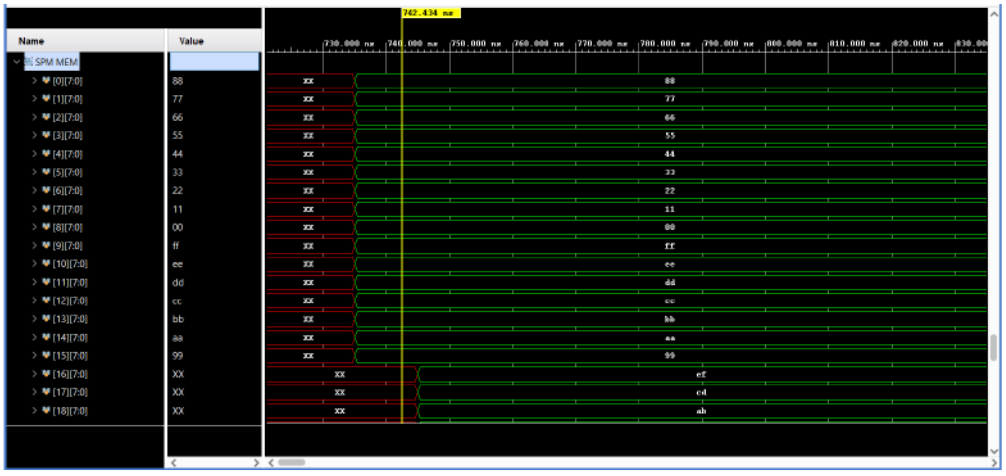


FIGURE 5.12: SPM Load, Section Of `hybrid_test.asm`.

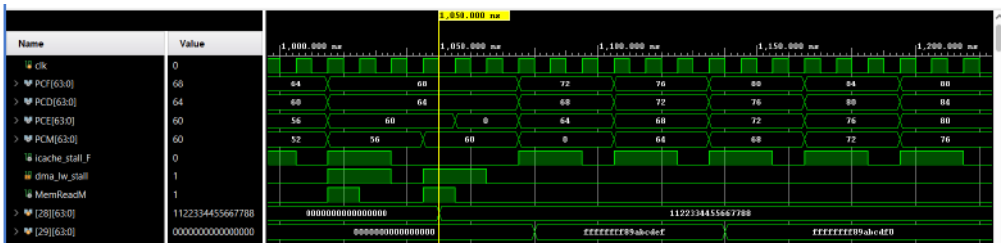


FIGURE 5.13: CPU reads From SPM, Section Of `hybrid_test.asm`.

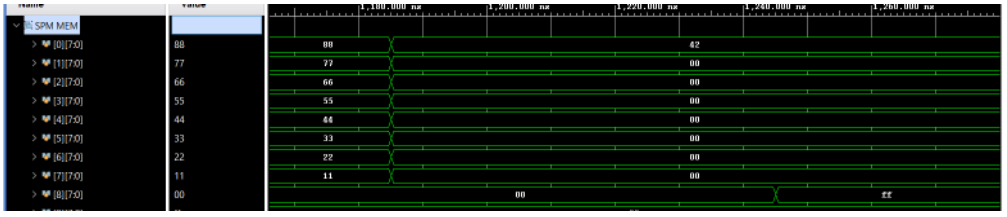


FIGURE 5.14: CPU Writes to SPM, Section Of `hybrid_test.asm`.

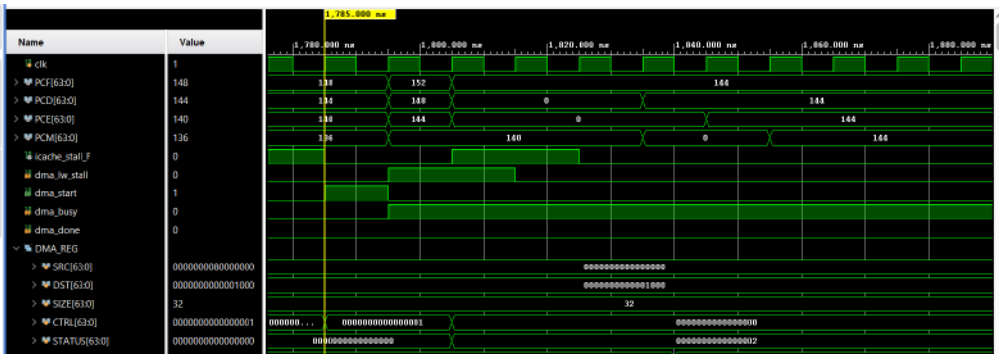


FIGURE 5.15: DMA Init for SPM Writeback, Section Of `hybrid_test.asm`.

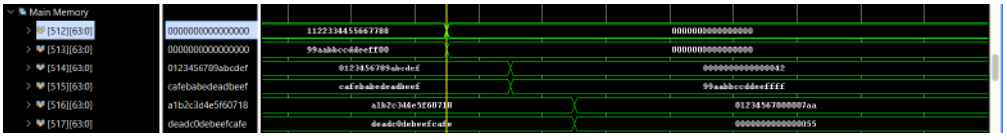


FIGURE 5.16: Data Transfer From SPM to Main Memory, Section Of `hybrid_test.asm`..

5.2.4 Exception Handling Tests

`newexception.s` verifies the trap path independently of the arithmetic/memory tests above. It installs a trap handler via `mtvec`, then deliberately triggers, in sequence, an `ecall`, a misaligned load, and an illegal instruction (encoded directly as `0xFFFFFFFF`). The trap handler reads back `mcause`, `mepc`, and `mtval`, branches to the matching handler based on `mcause`, advances `mepc` by 4 before `mret`, and returns control to the instruction immediately following the faulting one. This confirms three things simultaneously: that the correct cause code is latched for each of the three trap sources, that `mepc` correctly

captures the faulting instruction's address, and that `mret` correctly resumes execution at the intended return point rather than re-triggering the same trap.

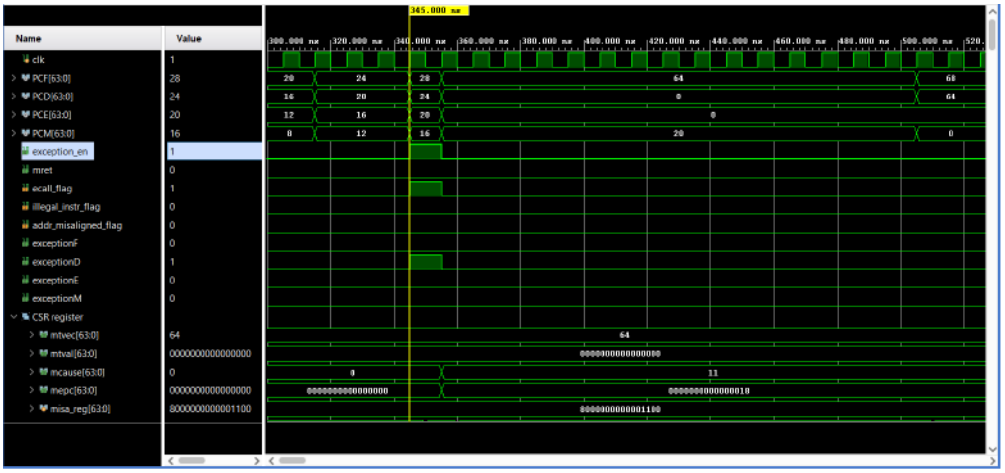


FIGURE 5.17: `ecall` exception path, showing trap entry and CSR updates for `mcause`, `mepc`, and `mtval`.

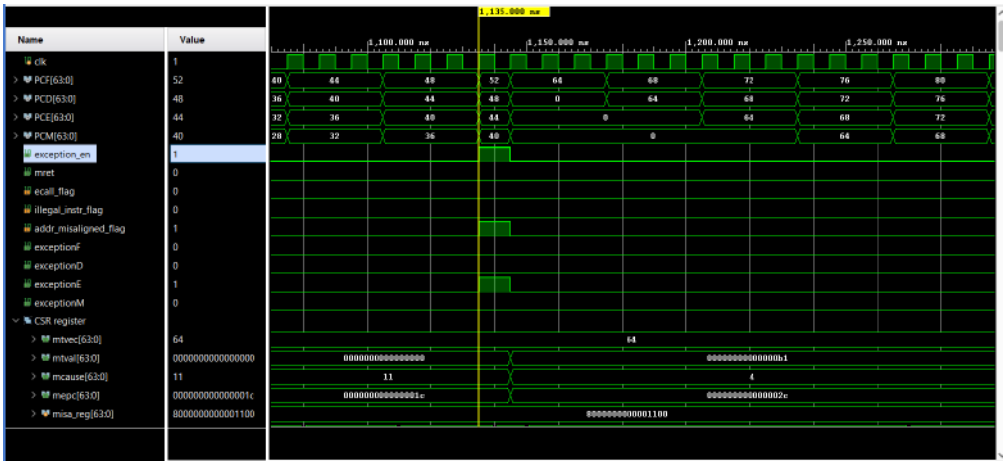


FIGURE 5.18: Misaligned-load exception path, showing trap redirection and CSR updates for the faulting load address.

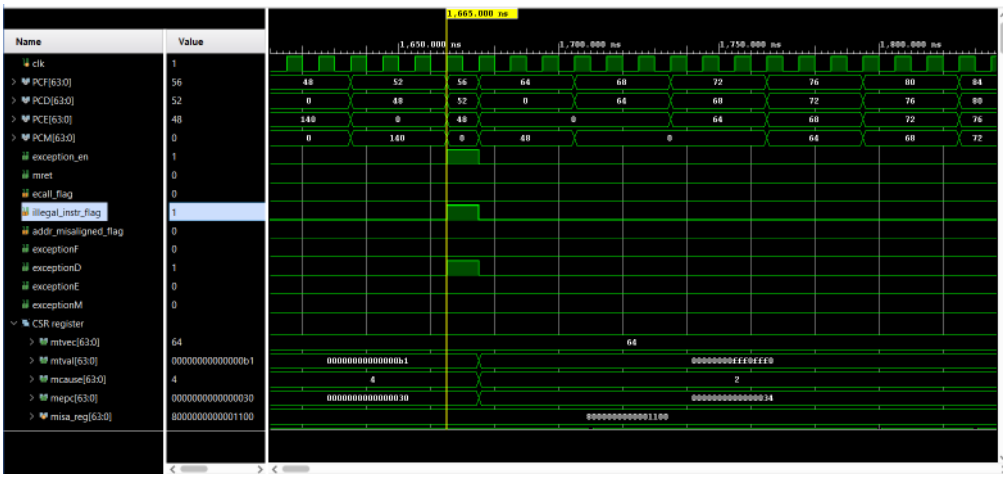


FIGURE 5.19: Illegal-instruction exception path, showing trap-handler entry and CSR updates for the invalid instruction.

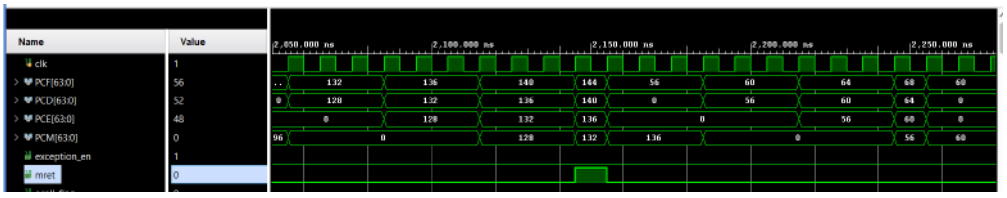


FIGURE 5.20: Exception return after `mret`, showing control leaving the trap handler and resuming at the next PC.

5.3 Summary

Across unit-level RTL checks and five targeted assembly stress programs, EdgeCore-V's base ISA (arithmetic, logic, shifts, all load/store widths), custom MAC extension, branch/jump control flow, D-cache line-packing behavior, the Hybrid configuration's DMA+SPM datapath, and the exception handling path were each independently exercised and confirmed against hand-computed expected results. This gives a reasonable degree of confidence that the functional correctness of all three configurations is established going into the performance benchmarking work of Chapters 6 and 7, where the same three configurations are compared quantitatively rather than checked for correctness.

Chapter 6

Implementation Results

6.0.1 Static Design Linting with SpyGlass

Before any RTL was carried forward into synthesis, the EdgeCore-V source was passed through Synopsys SpyGlass for static linting. Unlike simulation, which only exercises the specific stimulus applied by a testbench, static linting analyzes the RTL structurally and can flag latent issues that may never manifest under a limited set of simulation vectors but can silently corrupt behavior once the design is mapped to real gates and clocked at speed on hardware. For a design as sequentially dense as EdgeCore-V's pipeline, multiplier/ICU, and exception logic, this made SpyGlass an essential gate before implementation, rather than an optional code-quality step.

The general Lint goal was run over every module in the design and covers a broad range of structural and semantic checks:

- **Combinational loops.** Detects feedback paths in combinational logic that are not broken by a register, which can otherwise cause oscillation, simulation/synthesis mismatches, or timing analysis to be meaningless along that path. Any such loop reported was traced back to the offending always-block and restructured to insert a register or correct the intended data path.
- **Unintended latch inference.** Flags combinational `always`/`process` blocks with incomplete branch coverage (e.g., an `if` without a matching `else`, or a `case` statement missing a `default`), which cause the synthesis tool to infer a transparent latch to hold the signal's value on the unhandled branches. Latches of this kind are rarely intended in a synchronous pipeline design and were eliminated by adding explicit default assignments to every signal at the top of each combinational block.
- **Multi-driven and floating nets.** Identifies signals driven by more than one always-block or continuous assignment (multi-driven), which produces contention

or undefined resolution, as well as signals that are never assigned to anywhere in the design (floating), which typically indicates a connectivity mistake such as a missing port hookup or a typo in a signal name.

- **Incomplete sensitivity lists.** Checks that every signal read inside a combinational `always` block is also listed in that block's sensitivity list. Missing entries cause a mismatch between simulation behavior (which respects the sensitivity list) and synthesis behavior (which infers combinational logic from the block's actual reads), a classic source of simulation/hardware divergence.

- **Unused and undriven signals.** Reports declared signals, ports, or parameters that are never read (unused) or never written (undriven) anywhere in their scope. While often benign, these were reviewed individually, since an unused signal can also indicate a forgotten connection – for example, a status or flag output that was declared but never actually wired to the exception logic it was meant to report to.

- **Coding-style and syntax conventions.** Covers lower-severity stylistic findings – consistent blocking versus non-blocking assignment usage within sequential versus combinational blocks, consistent bit-width matching across assignments, and naming-convention consistency across modules – which, while not functionally fatal on their own, make the RTL easier to review and reduce the chance of introducing a real bug during later edits.

Each of these checks was run to completion and all reported violations were triaged: true violations were fixed directly in the RTL, while any findings judged to be false positives were documented with a justification rather than silently suppressed. Before synthesis, it was essential to ensure that there were no outstanding linting violations, since issues such as inferred latches, multi-driven nets, or incomplete sensitivity lists can pass simulation cleanly yet still produce a synthesized netlist that behaves unpredictably on hardware – exactly the class of bug that is hardest to trace once the design has already been mapped, placed, and routed.



FIGURE 6.1: Hazard Unit Before Linting.

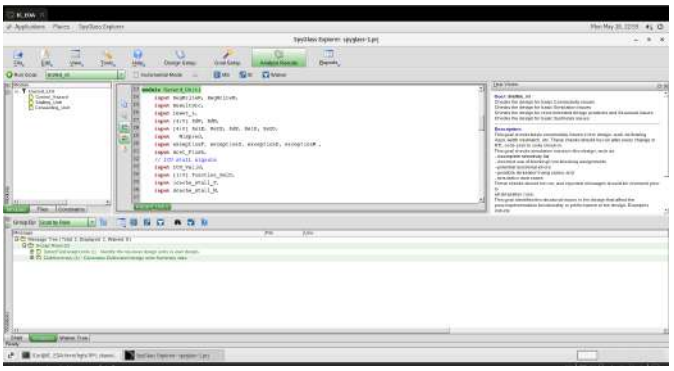


FIGURE 6.2: Hazard Unit Before Linting.

6.1 Implementation Environment

This section describes the hardware platform, software toolchain, and automation infrastructure used to synthesize, implement, and verify the EdgeCore-V processor.

6.1.1 Target FPGA Platform



FIGURE 6.3: Digilent Nexys 4 DDR development board (Xilinx Artix-7 XC7A100T), the target platform for EdgeCore-V.

EdgeCore-V was targeted at the Digilent Nexys 4 DDR development board, which is built around a Xilinx Artix-7 FPGA (part number `XC7A100T-1CSG324C`). This device provides a sufficient balance of logic resources, on-chip Block RAM, and DSP slices to accommodate the processor's 5-stage pipeline, Wallace tree multiplier, restoring division unit, and hybrid scratchpad-based memory subsystem, while remaining representative of

a resource-constrained embedded target. The board supplies a 100 MHz onboard oscillator as the primary system clock reference, along with a set of user-accessible switches, push-buttons, LEDs, and an 8-digit seven-segment display, which were used throughout development for isolated module-level hardware testing and status observation (e.g., exception flags and multiplier/ICU debug outputs).

6.1.2 Board I/O Interface

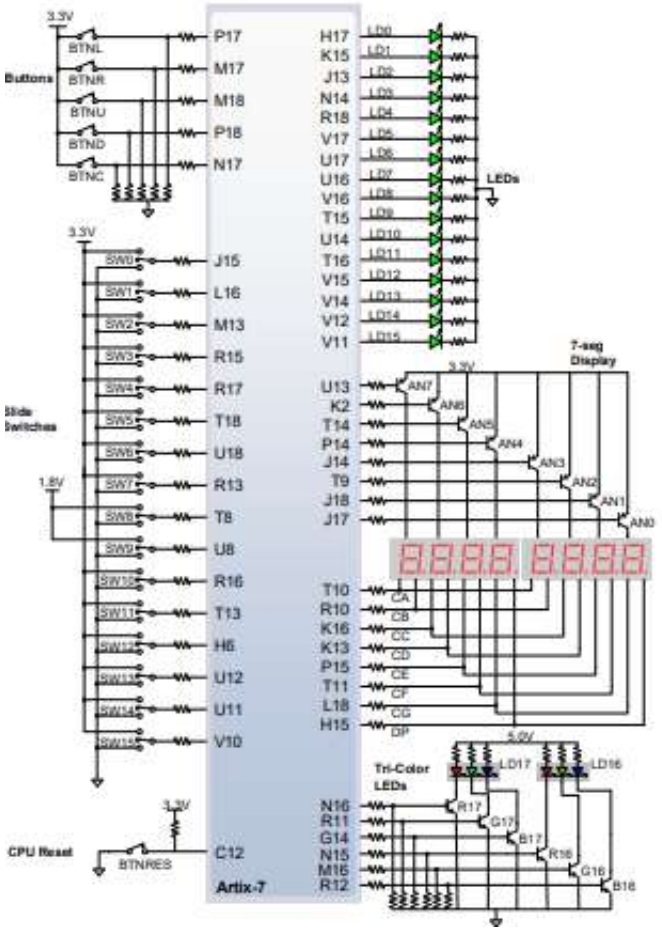


FIGURE 6.4: General Purpose I/O devices on the Nexys4 DDR.

Of the full set of I/O peripherals available on the Nexys 4 DDR (Figure 6.4), only a small, targeted subset was brought out at the top level for EdgeCore-V's hardware debugging interface: the 100 MHz onboard oscillator (`clk_in`), five slide switches (`SW1`–`SW5`) used as a register-select address, the center push-button (`BTNC`) used as the global active-low reset, and the seven-segment display (segments `a_to_g`, decimal point `dp`, and all eight digit anodes `an[7:0]`). All other onboard LEDs, switches, and buttons defined in the board's master XDC file were left commented out and unused in the current design.

6.1.2.1 Register File Observation via Switches and Seven-Segment Display

To inspect the processor's architectural state directly on hardware, without relying on simulation waveforms or an external debugger, five slide switches (`SW1`–`SW5`, mapped to the 5-bit port `ADDR[4:0]`) were used to select one of the register file's 32 entries. The selected `ADDR` value is fed into `CPU_System_Top`, which drives a dedicated 32-bit debug output, `OutToBoard`, reflecting the current contents of the addressed register.

Since `CPU_System_Top` runs on the CPU's own 40 MHz operating clock (`slow_clk`, generated by the Clocking Wizard as described in Section 6.2.1) while the display logic runs directly off the board's 100 MHz reference clock, `OutToBoard` is re-sampled into the `clk_in` domain using a free-running counter (`div`): every 41 cycles of `clk_in`, the current value of `OutToBoard` is latched into a holding register, `slow_Out`, which is what is actually driven to the seven-segment display module. This provides a simple, low-jitter mechanism for observing a value from the CPU clock domain on display hardware clocked from a different, faster domain, without requiring a full clock-domain-crossing synchronizer, since the observed value only needs to be visually legible rather than cycle-accurate.

The seven-segment display module (`top`) takes this 32-bit value and displays it as eight hexadecimal digits, multiplexed across the display's eight anodes (`an[7:0]`). A free-running 17-bit refresh counter selects one of eight 4-bit nibbles of the 32-bit input each cycle (`refresh_counter[16:14]`), routes it through the `hex2seg` decoder to produce the corresponding seven-segment pattern, and simultaneously enables only the corresponding anode (active-low), while disabling the other seven. This multiplexing occurs fast enough that, due to persistence of vision, all eight digits appear to be lit simultaneously and stably, producing a full 8-digit hexadecimal readout of the selected register's contents.

This mechanism proved valuable during bring-up and debugging of the pipeline, the Wallace tree multiplier/ICU, and the exception handling logic: by holding the processor at a known point via the reset button and selecting a register index with the switches, the full 32-bit contents of any general-purpose register could be read directly off the board in real time.

6.1.3 Custom Synthesis and Implementation Flow

Rather than relying solely on Vivado's default Project Mode GUI flow, a custom, script-driven Out-of-Context (OOC) synthesis and implementation flow was developed to support modular, reproducible builds of EdgeCore-V. As the design grew in scale and complexity — spanning a 7-stage pipeline, arithmetic units, a scratchpad-memory subsystem, and exception-handling logic — synthesizing the entire processor as a single monolithic top-level design became increasingly impractical, with long runtimes and errors that

were difficult to isolate to their source. Breaking the design down and synthesizing each structural and arithmetic building block independently, out of context, made the process significantly more manageable: each module could be verified for correct synthesis and resource usage on its own, before being integrated into the full system, rather than diagnosing failures across the entire design at once.

The flow consists of a sequence of Tcl scripts, orchestrated by a single top-level driver script, `RUN_ALL.tcl`, so that the entire synthesis-through-bitstream process can be launched with one command from the Vivado Tcl Console. Each script targets a specific structural group of the processor, progressing from small standalone leaf modules up to full system integration, as summarised below.

Stage 0 — Environment setup. `00_settings.tcl` defines the target device part, the source and output directory paths, and a reusable source-reading procedure. It performs no synthesis itself but is sourced at the start of every subsequent stage.

Stage 1 — Leaf modules. `01_synth_ooc_leaves.tcl` synthesizes 38 standalone structural and arithmetic leaf modules forming the base of the design hierarchy:

- **Arithmetic primitives:** `Full_adder`, `Half_adder`, `FA_array`, `HA_array`
- **Multiplexers:** `mux2x1`, `mux3x1`
- **Program-counter units:** `PC`, `PCPlus4`, `PCTarget`
- **Decoders:** `ALU_Decoder`, `Main_Decoder`
- **Branch prediction:** `Branch_State_Machine`, `Branch_Table_Buffer`, `Branch_reg`
- **Memory:** `Instruction_Memory`
- **Pipeline registers:** `Decode_Register`, `Execute_Register`, `Memory_Register`
- **Exception handling:** `Exception_Controller`, `Exception_decoder`, `csr_file`
- **Hazard handling:** `Stalling_Unit`, `Control_Hazard`, `Forwarding_Unit`
- **Multiplier tree:** `Wallance_tree_8`, `Wallace_64x64_shifter`,
- **Scratchpad/DMA subsystem:** `SPM`, `Mem_Selector`, `DMA_Register`, `DMA_Controller`, `Arbiter`
- **Support modules:** `Extend`, `hex2seg`, `ALU_UNIT`, `WRITEBACK_UNIT`, `Register_File`

Stage 2 — Integer compute unit. `02_synth_ooc_alu_group.tcl` synthesizes `Division_unit3`, and the top-level `ICU`.

Stage 3 — Exception handler. `03_synth_ooc_exception.tcl` synthesizes `Exception_Handler`, built from `Exception_Controller`, `Exception_decoder`, and `csr_file`.

Stage 4 — Scratchpad memory, DMA, and arbitration. `04_synth_ooc_spm.tcl` synthesizes the memory subsystem: `Scratchpad_Memory_Top` (comprising `Mem_Selector`, `DMA_Register`, `DMA_Controller`, and `SPM`), the standalone `Arbiter` that mediates CPU and DMA access to main memory, and `Main_Memory_AXI_Slave`.

Stage 5 — Branch prediction. `05_synth_ooc_branch.tcl` synthesizes `Branch_Prediction_Top`, built from `Branch_Table_Buffer`, `Branch_State_Machine`, and `Branch_reg`.

Stage 6 — Hazard unit. `06_synth_ooc_hazard.tcl` synthesizes `Hazard_Unit`, built from `Forwarding_Unit`, `Stalling_Unit`, and `Control_Hazard`.

Stage 7 — Control path. `07_synth_ooc_control.tcl` synthesizes `S_C_Control`, built from `Main_Decoder` and `ALU_Decoder`.

Stage 8 — Datapath. `08_synth_ooc_datapath.tcl` synthesizes `S_C_Datapath`, the largest and most resource-intensive block in the design, integrating the pipeline stages, the ICU, branch prediction, hazard handling, and exception logic. Given its size, this stage uses `-flatten_hierarchy rebuilt` to keep sub-module boundaries visible in the utilization and timing reports.

Stage 9 — CPU top levels. `09_synth_ooc_cpu.tcl` synthesizes `Single_Cycle_TOP` (combining `S_C_Control` and `S_C_Datapath`) and `CPU_System_Top`, which integrates `Single_Cycle_TOP` with the scratchpad-memory subsystem (`Scratchpad_Memory_Top`), the `Arbiter`, and `Main_Memory_AXI_Slave`.

Stage 10 — Full-hierarchy top synthesis. `10_synth_top_FPGA.tcl` performs full-hierarchy synthesis under the top-level module `FPGA`, whose hierarchy integrates , the complete CPU system (`CPU_System_Top`), and the seven-segment display driver (`hex2seg`).

Stage 11 — Implementation. `11_implement.tcl` loads the synthesized checkpoint, applies the physical and timing constraints, and carries the design through `opt_design`, `place_design`, `phys_opt_design`, `route_design`, and bitstream generation.

Each leaf and mid-level synthesis stage (scripts `01`–`09`) is run using `synth_design` in out-of-context mode with the `RuntimeOptimized` directive, producing a design checkpoint (`.dcp`) and a utilization report for every synthesized module. This modular OOC approach allowed structural and arithmetic submodules to be independently verified

for correct synthesis and resource utilization before full top-level integration, isolating module-level synthesis issues from system-level integration and timing-closure issues.

Physical pin assignments and timing constraints were specified in a board-specific Xilinx Design Constraints (XDC) file derived from the standard Nexys 4 DDR master constraints file, retaining only the pins required by the design (100 MHz clock input, reset button, address/control switches, seven-segment display outputs, and status LEDs) and defining the primary 100 MHz input clock via `create_clock`.

6.2 Static Timing Analysis

6.2.1 Clock Generation Strategy

The system clock for EdgeCore-V was generated using the Xilinx Clocking Wizard IP core (`clk_wiz_0`), configured to derive a 40 MHz operating clock from the Nexys 4 DDR's 100 MHz onboard oscillator. This clock frequency was selected after post-route static timing analysis showed that the design's critical path, running from the multiplier output through exception-detection logic to the CSR clock-enable pins, could not meet timing at higher frequencies; 40 MHz was chosen to provide adequate positive slack margin over the measured critical path delay while implementation-level and RTL-level timing optimizations were pursued in parallel.

6.2.1.1 Clocking Wizard (MMCM) versus a Hand-Written RTL Clock Divider

An earlier version of the design used a manually written RTL clock divider, implemented as a toggle flip-flop driven directly by the 100 MHz input clock, to derive a lower-frequency clock for the CPU core. This approach was replaced with the Xilinx Clocking Wizard IP for several reasons directly relevant to timing closure and design robustness:

- **Automatic clock-relationship derivation.** A clock generated by ordinary RTL logic is not automatically recognized by Vivado's timing engine as a true clock; it is treated as combinational logic unless a generated-clock constraint is added manually via `create_generated_clock`. Omitting or misconfiguring this constraint risks the tool either failing to analyze the derived clock domain correctly or, conversely, applying an incorrect timing relationship to it. The Clocking Wizard IP does not have this issue: the constraints describing its output clock frequency and its relationship to the input clock are packaged with the IP itself and are applied automatically once the IP is instantiated, ensuring the derived clock is correctly recognized and analyzed from the start.

- **Dedicated clock-management hardware.** The Clocking Wizard instantiates an MMCM (Mixed-Mode Clock Manager) primitive, a dedicated hardware block on the Artix-7 fabric built specifically for clock synthesis and distribution. A clock produced this way is distributed through the FPGA's global clock network (via a `BUFG` buffer inserted automatically by the tool), giving it far lower skew and jitter than a clock signal toggled in general logic and routed through ordinary interconnect, as was the case with the RTL divider.
- **Frequency flexibility.** The Clocking Wizard supports fractional multiply/divide ratios, allowing the output frequency to be tuned precisely to whatever value the design's critical path can support (in this case, 40 MHz), rather than being limited to integer divisions of the 100 MHz input, as a simple toggle-based divider would be.

For these reasons, the RTL clock divider was retained in the codebase only for auxiliary, non-timing-critical purposes (e.g., driving the seven-segment display refresh), while the Clocking Wizard was adopted for generating the CPU's operating clock.

6.2.1.2 MMCM versus PLL

The Clocking Wizard was configured to use the MMCM primitive rather than a PLL (`PLLE2_ADV`), even though both are available on the target device and share the same 6-site clock-management resource budget on the XC7A100T. MMCM was chosen over PLL for two reasons specific to this design's requirements:

- **Fractional frequency synthesis.** Achieving an exact 40 MHz output from a 100 MHz input, or any other non-integer-ratio frequency that later timing-closure iterations might require, needs fractional multiply/divide capability. The PLL primitive on 7-series devices supports only a more limited set of frequency synthesis options compared to the MMCM, making the MMCM the safer choice for a design whose target frequency was still being tuned against timing closure results rather than fixed in advance.
- **No competing resource pressure.** PLL is typically preferred over MMCM only when a design is already using several MMCMs for other purposes (e.g., video, DDR, or Ethernet clocking) and needs to conserve MMCM resources for those functions. EdgeCore-V uses only a single clock-generation IP for the CPU core and has no such competing demand, so there was no resource-driven incentive to use the more limited PLL primitive instead of the MMCM.



FIGURE 6.5: Timing Summary.

6.2.2 Final Timing Results

To achieve timing closure, multiple clock frequencies were tested through several synthesis and implementation iterations and optimization for design. After evaluating different target frequencies, a clock frequency of **40 MHz** was found to provide positive slack across all paths while maintaining good overall performance for the design.

Table 6.1 summarizes the final timing results obtained at 40 MHz after implementation in Vivado.

TABLE 6.1: Design Timing Summary at 40 MHz

Metric	**Setup**	**Hold**
Worst Slack (WNS / WHS)	1.631 ns	0.041 ns
Total Negative Slack (TNS / THS)	0.000 ns	0.000 ns
Number of Failing Endpoints	0	0
Total Number of Endpoints	73934	73934

The positive worst negative slack (WNS = 1.631 ns) and worst hold slack (WHS = 0.041 ns), combined with zero failing endpoints out of 73934 total endpoints, confirm that the design successfully meets timing requirements at the selected 40 MHz operating frequency.

6.3 Area Utilization

Post-route resource utilization for EdgeCore-V was obtained via Vivado's hierarchical utilization report (`report_utilization -hierarchical`), generated on the fully routed `FPGA` top-level design targeting the Xilinx Artix-7 XC7A100T-1CSG324C. This section summarizes the device-level resource consumption and breaks down the contribution of each major functional block within the design hierarchy.

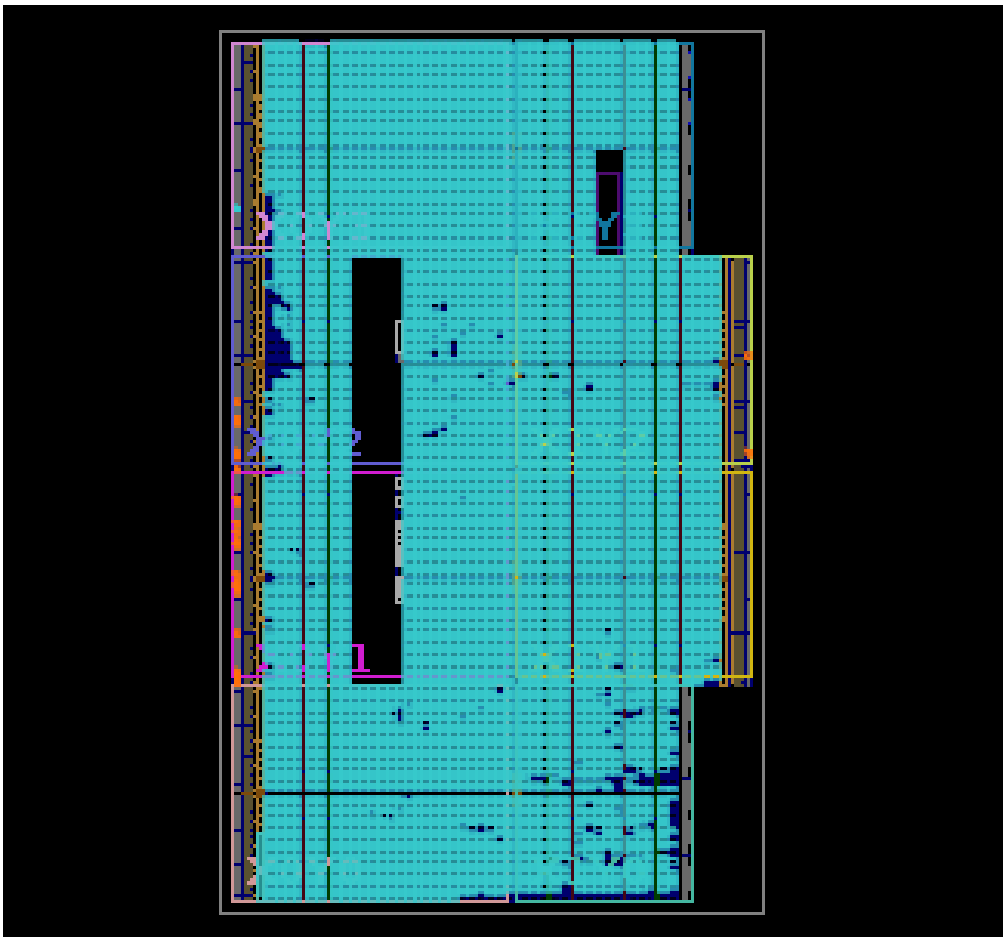


FIGURE 6.6: Utilization of RiscV.

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Block RAM Tile (135)	Bonded IPADs (2)
FPGA	42566	18964	2356	771	14859	38054	4512	18964	24
clk_wiz (clk_wiz_0)	0	0	0	0	0	0	0	0	0
CPU (CPU_System_Top)	42193	18907	2352	771	14841	37681	4512	0	0
seg7 (top)	17	17	4	0	12	17	0	0	0

FIGURE 6.7: Utilization of RiscV.

6.3.1 Device-Level Summary

Table 6.2 summarizes the total resources consumed by the design against the capacity of the target device.

The design consumes roughly two-thirds of the device's available LUT resources, but only about 15% of available flip-flops, indicating that EdgeCore-V is predominantly LUT-bound rather than register-bound on this device. Notably, no Block RAM (`RAMB36`/`RAMB18`) or DSP48 primitives were inferred anywhere in the design; the memory subsystem and arithmetic units were synthesized entirely from general fabric logic (LUTs, LUTRAM, and flip-flops) rather than dedicated hard blocks.

Table 6.2: Post-route resource utilization summary on the XC7A100T-1CSG324C.

Resource	Used	Available	Utilization
Total LUTs	42,256	63,400	66.65%
Logic LUTs	37,744	63,400	59.53%
LUTRAM	4,512	19,000*	23.75%*
CLB Flip-Flops	19,029	126,800	15.01%
RAMB36	0	135	0.00%
RAMB18	0	270	0.00%
DSP48 Blocks	0	240	0.00%

*Approximate distributed-RAM-capable LUT capacity; exact figure depends on device architecture documentation.

6.3.2 Hierarchical Breakdown

The hierarchical report shows that the datapath dominates the design's resource usage almost entirely. Of the 42,256 total LUTs and 19,029 flip-flops used by the full `FPGA` design, the `CPU_System_Top` instance accounts for 41,890 LUTs and 18,972 flip-flops, and within it, `S_C_Datapath` alone accounts for 39,770 LUTs and 16,523 flip-flops – over 94% of the entire design's LUT usage. By contrast, the control path (`S_C_Control`) consumes only 50 LUTs and no flip-flops, since it implements purely combinational instruction decoding.

Within the datapath, the largest individual contributors are:

- **D_Cache_Top** (data cache): 16,990 LUTs and 1,597 flip-flops, of which the cache controller (`Cache_Controller_Final`) alone accounts for 12,743 LUTs – the single largest submodule in the entire design.
- **execute_register** (`Execute_Register`): 11,566 LUTs but only 461 flip-flops. This is a notable imbalance for what is nominally a pipeline register stage, where LUT usage would normally be expected to track closely with flip-flop count; the disproportionate LUT usage suggests this module may contain more combinational logic (e.g., forwarding or hazard-resolution muxing) than a conventional pipeline register would require, and is flagged here as a candidate for further inspection and potential area optimization.
- **Branch_Prediction_Top**: 2,112 LUTs but 7,876 flip-flops, almost entirely from the branch target buffer (`Branch_Table_Buffer`, 7,872 flip-flops). The prediction table is implemented as distributed flip-flop-based storage rather than block or distributed RAM, which explains its comparatively heavy flip-flop footprint relative to its LUT usage.

- **I_Cache_Top** (instruction cache): 4,919 LUTs and 1,068 flip-flops, similarly dominated by its controller (`I_Cache_Controller`, 2,606 LUTs) rather than its memory array (`I_Cache_Memory`, 2,057 LUTs, almost entirely LUTRAM).
- **Main_Memory_AXI_Slave**: 2,070 LUTs and 2,449 flip-flops, reflecting the AXI protocol handling logic and buffering at the system's external memory interface.

Smaller structural components – the register file, program-counter logic, sign/immediate extension unit, exception handler, and write-back unit – each contribute proportionally little to the overall utilization, consistent with their role as comparatively simple, narrow-width datapath elements.

6.3.3 Discussion

The complete absence of inferred Block RAM and DSP48 usage is a significant characteristic of the current implementation. Both cache memories (`I_Cache_Memory`, `Cache_Memory`) and the branch target buffer are realized using distributed LUTRAM and flip-flop-based storage rather than Block RAM primitives, and the multiplier and division units (booth-radix-2 multiplier, Wallace tree, restoring divider) are synthesized entirely from LUT fabric rather than DSP48 slices. While this confirms that the design fits comfortably within the XC7A100T's LUT and flip-flop budget, it also means that two of the device's dedicated hard-block resources – Block RAM and DSP48 – remain entirely unused. Explicitly inferring these primitives (e.g., via `RAM_STYLE` or `USE_DSP` attributes, or a more targeted memory/arithmetic architecture) represents a clear direction for future area optimization, potentially freeing a substantial share of the 66.65% LUT utilization currently consumed, particularly from the cache controllers and multiplier/divider units that dominate the current resource profile.

6.4 Power Analysis

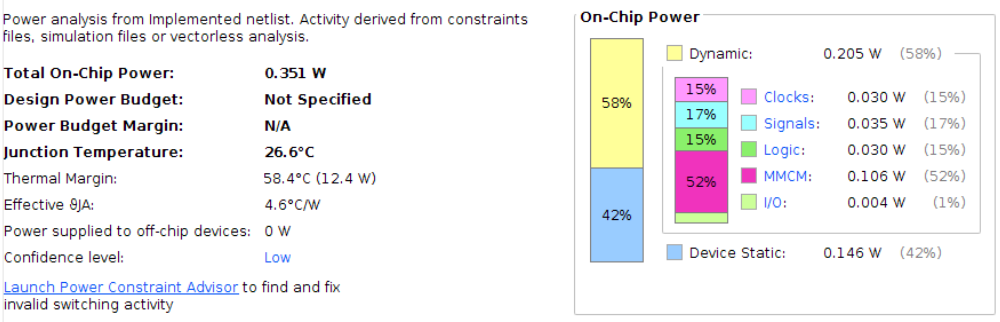


FIGURE 6.8: Power Summary.

Post-implementation power analysis was performed in Vivado using the implemented netlist, with switching activity estimated vectorlessly (i.e., without simulation-derived or constraint-file-based activity data). The resulting on-chip power summary is shown in Table 6.3.

TABLE 6.3: Post-implementation on-chip power summary.

Metric	**Value**
Total On-Chip Power	0.351 W
Dynamic Power	0.205 W (58%)
Device Static Power	0.146 W (42%)
Junction Temperature	26.6°C
Thermal Margin	58.4°C (12.4 W)
Effective θ_{JA}	4.6°C/W
Confidence Level	Low

The total on-chip power of 0.351 W is dominated by dynamic power (58%), with the remaining 42% attributed to device static (leakage) power. The junction temperature of 26.6°C, well below any thermal concern, leaves a thermal margin of 58.4°C, indicating that thermal dissipation is not a limiting factor for this design on the target device.

6.4.1 Dynamic Power

Dynamic power in CMOS digital logic arises from the repeated charging and discharging of capacitive loads (gate inputs, interconnect, and wire capacitance) each time a signal switches state, and is governed by the standard switching power equation:

$$P_{dynamic} = \alpha \cdot C \cdot V_{DD}^2 \cdot f \tag{6.1}$$

where:

- α is the switching activity factor – the average number (or probability) of logic transitions per clock cycle for a given node,
- C is the total capacitance being switched (gate capacitance plus routing/interconnect capacitance),
- V_{DD} is the supply voltage, and
- f is the clock (or switching) frequency.

This equation explains two key properties of dynamic power directly relevant to EdgeCore-V's implementation: first, dynamic power scales linearly with switching frequency, so

operating the CPU core at a reduced 50 MHz clock (as discussed in Section 6.2) substantially reduces dynamic power relative to running at a higher frequency; and second, dynamic power scales with the *square* of supply voltage, making V_{DD} the most significant single factor in dynamic power consumption, although this is fixed by the device's operating specification and not a parameter under designer control on a fixed FPGA platform.

6.4.2 Breakdown by Contributor

Vivado further decomposes the 0.205 W of dynamic power by source, as shown in Table 6.4.

TABLE 6.4: Dynamic power breakdown by contributor.

Contributor	Power	Share of Dynamic Power
MMCM	0.106 W	52%
Signals	0.035 W	17%
Clocks	0.030 W	15%
Logic	0.030 W	15%
I/O	0.004 W	1%

The most notable observation is that the MMCM (the Clocking Wizard IP discussed in Section 6.2.1) accounts for over half of the design's total dynamic power, more than the combined contribution of clock distribution, signal switching, and logic activity. This is an expected characteristic of MMCM-based clock generation rather than a symptom of inefficiency in the CPU design itself: the MMCM's internal voltage-controlled oscillator (VCO) and feedback circuitry must continuously run at its configured operating frequency to maintain a locked, stable output clock, regardless of how much switching activity is actually occurring in the downstream logic. Because EdgeCore-V's core logic is clocked at only 50 MHz – a substantial reduction from the 100 MHz reference input – the fixed overhead of maintaining the MMCM lock becomes proportionally large relative to the comparatively modest dynamic power drawn by the CPU logic itself at that low frequency. Clock distribution (15%) and signal toggling (17%) together account for a further third of dynamic power, consistent with a design whose logic depth and fan-out are non-trivial, while the core combinational/sequential logic itself (15%) and I/O (1%) contribute comparatively little.

6.4.3 Discussion

The reported confidence level of *Low* reflects the fact that this analysis was performed vectorlessly, without switching-activity data derived from post-implementation timing simulation or a Switching Activity Interchange Format (SAIF) file. Vectorless estimation

relies on Vivado's default toggle-rate assumptions rather than the design's actual runtime behavior, and can therefore over- or under-estimate power for specific modules whose real activity patterns differ from these defaults. A more accurate power estimate could be obtained by driving the power analysis with switching activity captured from a post-implementation functional simulation (e.g., running representative RISC-V test programs through the pipeline in simulation and exporting a SAIF file), which would raise Vivado's confidence level and better reflect the actual power profile of EdgeCore-V under realistic workloads.

Nonetheless, the overall total on-chip power of 0.351 W is modest, and the dominant contribution from the MMCM suggests that, from a power-efficiency standpoint, further reduction would come primarily from reconsidering the clock generation strategy (e.g., a lower MMCM input/output frequency ratio, or evaluating whether an even lower core clock frequency remains compatible with timing closure) rather than from optimizing the CPU's logic or memory subsystem, which together account for a comparatively small share of total dynamic power.

Chapter 7

Benchmarking Using MNIST Inference

7.1 Benchmarking Methodology

The purpose of this chapter is to evaluate and compare the performance of the three processor configurations introduced in Chapter 4 using a realistic edge-AI workload. Rather than relying on synthetic micro-benchmarks or randomly generated instruction streams, the evaluation is carried out using a complete quantized neural network inference application: digit classification on the MNIST dataset. MNIST can be regarded as the "hello world" of machine learning workloads, making it a natural and widely recognized choice for validating the correctness and performance of our processor design. This choice allows the benchmark to capture the instruction mix, memory access patterns, and control-flow behaviour that a real embedded inference workload exhibits, which is precisely the class of workload this processor targets.

Three architectural variants of the processor are compared:

- **Version 1 – Direct:** the processor is connected directly to main memory over AXI, with only an instruction cache present and no data cache or scratchpad memory. Every load and store issued by the core is serviced directly by main memory.
- **Version 2 – Caches:** the processor is augmented with both an instruction cache and a data cache, allowing temporal and spatial locality in the inference workload's memory accesses to be exploited.
- **Version 3 – Hybrid (DMA + SPM):** the processor retains the instruction and data caches of Version 2, but additionally integrates a scratchpad memory (SPM) that is populated ahead of time by a DMA engine. The inference kernel

is restructured to stage the network weights and intermediate activations in the SPM, removing the majority of data-side traffic from the AXI interconnect during the compute-intensive phases of inference.

The three versions are evaluated using the identical MNIST inference application (with Version 3 using an adapted RISC-V assembly implementation, as described in Section 7.3), the same quantized weight set, and the same set of test images, so that the resulting performance differences can be attributed purely to the memory subsystem architecture rather than to differences in the workload itself. To ensure a fair comparison:

- All three configurations run at the same clock frequency on the same FPGA platform.
- All three configurations execute the same trained and quantized model parameters, so classification accuracy is identical across versions and only execution-time performance varies.
- Hardware performance counters (PMU) embedded in the RTL are used to collect cycle counts, instruction counts, cache statistics, stall breakdowns, and DMA activity directly from hardware execution rather than from simulation estimates.

The metrics collected for each configuration include total cycle count, total dynamic instruction count, cycles-per-instruction (CPI) and its inverse, instructions-per-cycle (IPC), instruction and data-cache hit/miss rates, a breakdown of pipeline stall cycles by cause, and, for Version 3, DMA transfer counts and overhead. These metrics are used in Section 7.4 to quantify the performance gap between the three designs and to explain, at a microarchitectural level, why the hybrid D-Cache + SPM design achieves the best performance.

7.2 MNIST Neural Network

Image classification is the task of assigning an input image to one of a fixed set of semantic categories. In a neural-network classifier, the image is first converted into a numerical representation, then passed through a sequence of learned layers that extract increasingly useful features for distinguishing between classes. The final layer produces one score for each possible class, and the class with the largest score is selected as the prediction.

MNIST is a simple but representative example of this workflow. Each handwritten digit image is treated as the input to the classifier, and the model must decide which of the ten digit classes, 0 through 9, it represents. Although MNIST is much smaller than modern image-classification datasets, it captures the essential structure of an inference

workload: input loading, repeated multiply-accumulate operations, activation functions, intermediate data reuse, and final class selection. For this reason, it provides a useful benchmark for evaluating how effectively the processor and memory subsystem support embedded machine-learning inference.

7.2.1 MNIST Dataset

The MNIST database of handwritten digits is a widely used benchmark dataset in the machine learning community and is well suited for evaluating embedded inference hardware due to its modest input dimensionality and well-understood baseline accuracy. Each image in the dataset is a 28×28 pixel, single-channel (grayscale) image of a handwritten digit between 0 and 9, giving 784 input features per sample once the image is flattened into a vector. The dataset defines 10 output classes, corresponding to the digits 0–9.

The standard MNIST split consists of 60,000 training images and 10,000 test images. For this work, the training set was used exclusively to train the floating-point reference model in Python (Section 7.2.3), while a subset of the test set was used for on-hardware inference benchmarking.

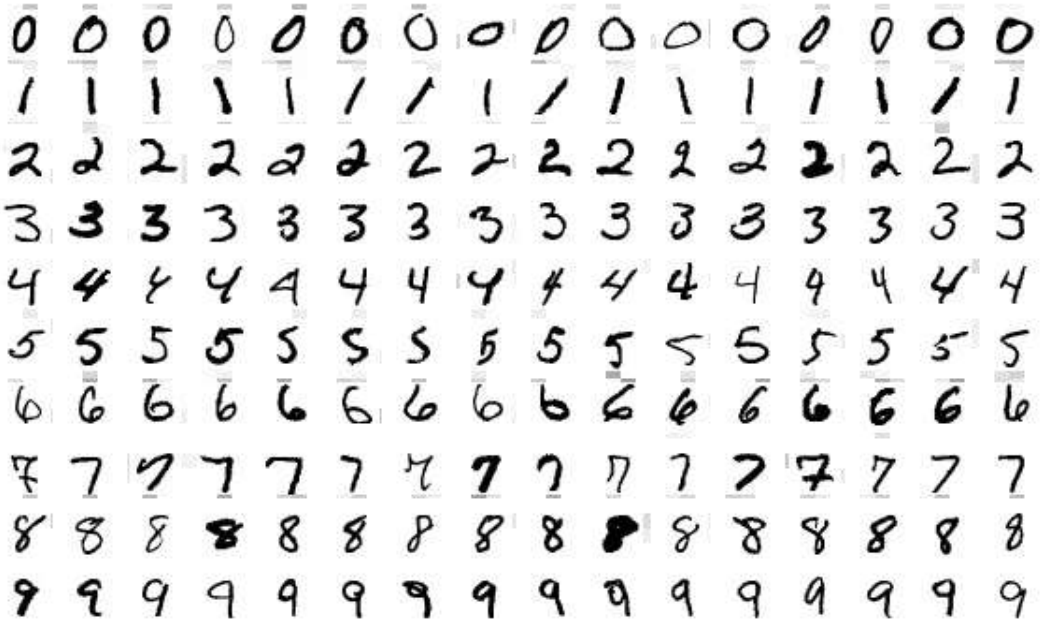


FIGURE 7.1: Sample handwritten digits from the MNIST dataset.

7.2.2 Neural Network Architecture

The classifier implemented for this project is a compact, fully connected (multi-layer perceptron) neural network, chosen deliberately over a convolutional architecture to keep the instruction count and memory footprint compatible with a resource-constrained embedded core, while still exercising the full range of memory-access patterns of interest (weight streaming, matrix-vector multiplication, and non-linear activation).

The network consists of an input layer, two hidden layers, and an output layer, corresponding to the three weight matrices W_1, W_2, and W_3 (with associated bias vectors b_1, b_2, and b_3) referenced throughout this chapter and detailed in the appendix:

- **Input layer:** 784 neurons, one per pixel of the flattened 28×28 input image.
- **Hidden layer 1:** `[N1 = 128]` neurons, fully connected to the input layer, using $W_1 \in \mathbb{R}^{784 \times N_1}$ and bias $b_1 \in \mathbb{R}^{N_1}$, followed by a ReLU activation.
- **Hidden layer 2:** `[N2 = 64]` neurons, fully connected to hidden layer 1, using $W_2 \in \mathbb{R}^{N_1 \times N_2}$ and bias $b_2 \in \mathbb{R}^{N_2}$, followed by a ReLU activation.
- **Output layer:** 10 neurons (one per digit class), using $W_3 \in \mathbb{R}^{N_2 \times 10}$ and bias $b_3 \in \mathbb{R}^{10}$.

The forward pass of the network can be expressed as:

$$z_1 = W_1^T x + b_1, \qquad a_1 = \mathrm{ReLU}(z_1) \tag{7.1}$$

$$z_2 = W_2^T a_1 + b_2, \qquad a_2 = \mathrm{ReLU}(z_2) \tag{7.2}$$

$$z_3 = W_3^T a_2 + b_3 \tag{7.3}$$

where $x \in \mathbb{R}^{784}$ is the flattened input image. Unlike a typical floating-point classifier, the deployed version of this network does not apply a softmax activation at the output: since the goal of on-device inference is classification rather than probability calibration, the predicted digit is obtained directly by taking the *argmax* of the raw logits z_3, avoiding the cost of an exponential function on hardware that has no floating-point unit.

$$\hat{y} = \arg \max_{i \in \{0,\ldots,9\}} z_{3,i} \tag{7.4}$$

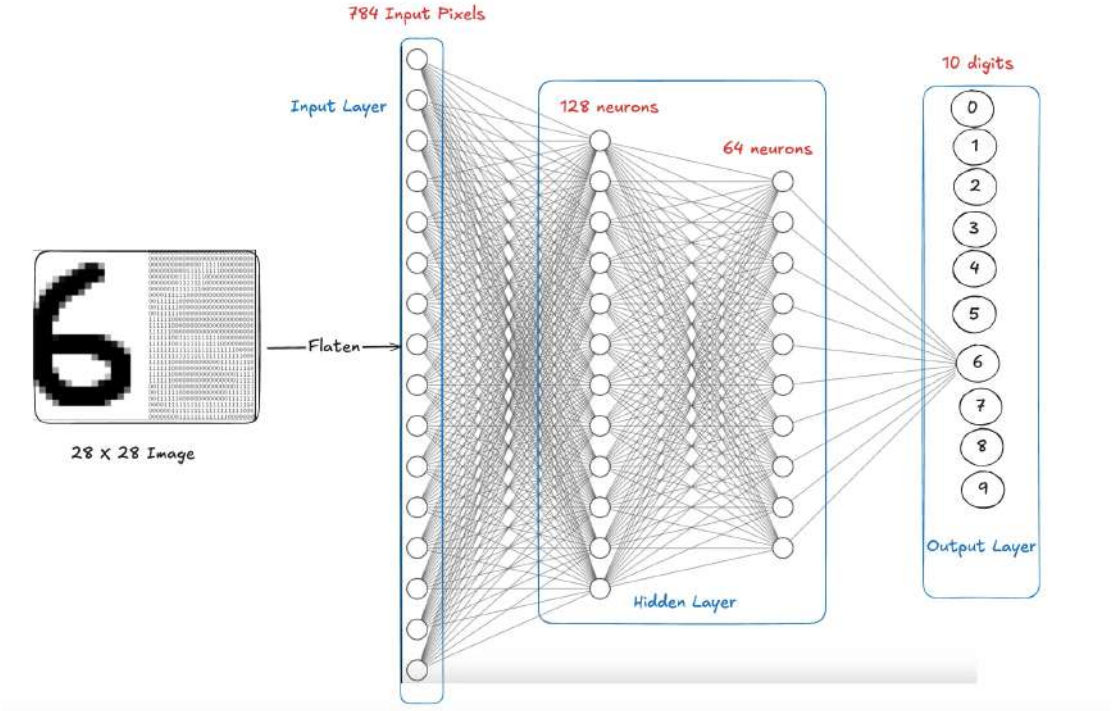


FIGURE 7.2: Architecture of the fully connected neural network used for MNIST digit classification.

7.2.3 Python Model Training

The network described above was first implemented and trained in Python using a standard floating-point (32-bit) representation, in order to establish a reference model against which the quantized and hardware-deployed versions could later be validated. This floating-point model serves two purposes: it establishes an upper-bound accuracy figure for the chosen network architecture, and it provides the trained parameters (W_1, W_2, W_3, b_1, b_2, b_3) that are subsequently quantized for deployment.

The model was trained on the full 60,000-image MNIST training set using mini-batch gradient descent with the ReLU activation described above and a cross-entropy loss computed over a softmax output during training only (the softmax is not part of the deployed inference path, as noted above). After training, the floating-point model achieves a classification accuracy of **97.6%** on the MNIST test set.

This 97.6% figure represents the theoretical ceiling for the chosen network architecture and training procedure; all subsequent accuracy figures reported in this chapter (for the fixed-point C model and the INT8 assembly implementation) are evaluated relative to this floating-point reference.

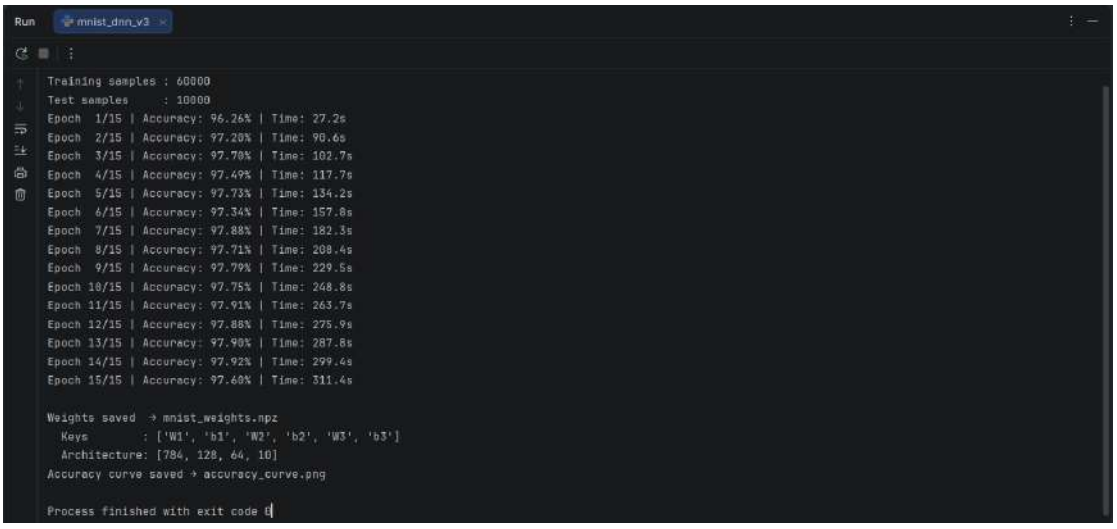


FIGURE 7.3: MNIST Python Model Training

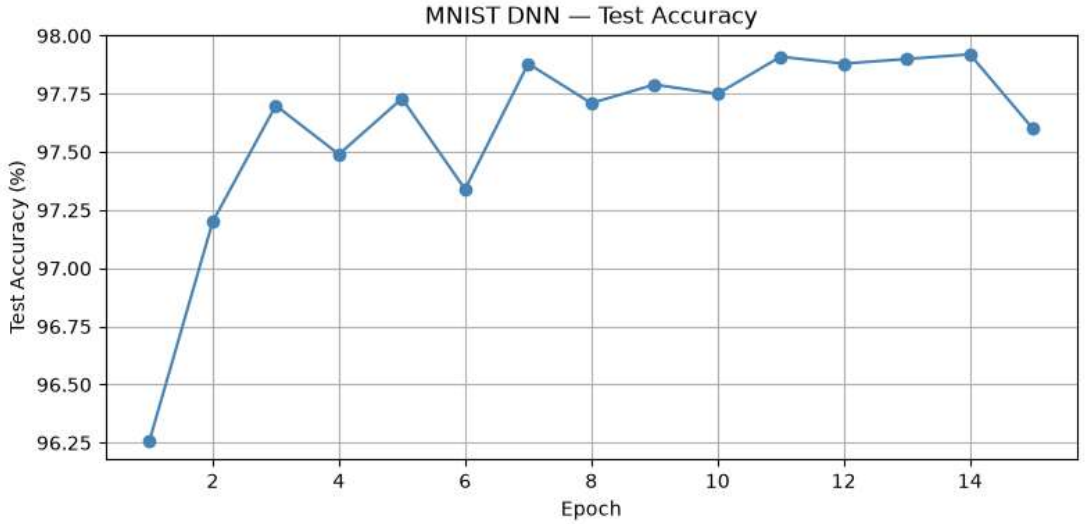


FIGURE 7.4: MNIST Training Accuracy Graph

7.2.4 INT8 Quantization

Deploying the trained floating-point model directly is impractical, since the processor has no floating-point unit and floating-point arithmetic would have to be emulated in software at significant cycle cost. To make the network suitable for integer-only embedded execution, all weights and biases were quantized from 32-bit floating point to 8-bit signed integers (INT8).

Quantization was applied per layer, mapping the floating-point range of each weight matrix and bias vector to the representable INT8 range $[-128, 127]$ using a linear (affine) scale factor derived from the maximum absolute value observed in that layer's parameters. Each of the six parameter tensors – W_1, b_1, W_2, b_2, W_3, and b_3 – was quantized independently, so that each layer uses the scale that best preserves its own dynamic

range rather than a single global scale that would be dominated by the layer with the largest magnitude weights.

At inference time, the accumulation of INT8 products is carried out in a wider integer accumulator to avoid overflow, and the result is rescaled back down before the ReLU activation and before being passed to the next layer, so that activations entering each layer remain within the INT8 representable range.

Quantizing the network from floating point to INT8 introduces a small but measurable accuracy loss, since the discretization of weights and activations reduces the precision with which the network's learned decision boundaries are represented. This effect, and its final measured impact on classification accuracy, is discussed together with the C reference implementation in Section 7.3.1, since the INT8 arithmetic was first validated in C before being ported to RISC-V assembly.

7.3 Software Deployment

7.3.1 C Implementation

Before committing to a hand-written RISC-V assembly implementation, the quantized INT8 network was first re-implemented in C. This intermediate step served as a functional bridge between the Python reference model and the final assembly implementation: it allowed the INT8 quantization scheme, the fixed-point rescaling arithmetic, and the ReLU/argmax logic to be validated on a general-purpose machine, using the exact same integer arithmetic that would later be hand-coded in assembly, before introducing the added complexity of manual register allocation and RISC-V instruction selection.

The C implementation performs the full forward pass described in Section 7.2 using only integer types, replicating the INT8 matrix-vector multiplication, per-layer rescaling, ReLU activation, and final argmax classification of the target RISC-V assembly implementation. Its output classification for each test image was compared directly against the floating-point Python model to isolate the accuracy impact of quantization alone, independent of any implementation bugs that might otherwise be introduced when hand-writing assembly.

Running the quantized INT8 model in C over the MNIST test set yields a classification accuracy of **89.6%**, compared to the **97.6%** achieved by the floating-point reference model. This roughly 8-percentage-point drop is consistent with the expected cost of aggressive 8-bit per-layer quantization on a compact fully connected network, and confirms that the quantization scheme is sound: the loss is attributable to reduced numerical precision rather than to an error in the quantization or rescaling logic, since the C model's

accuracy matches expectations for this class of quantization and forms a validated baseline for the assembly port.

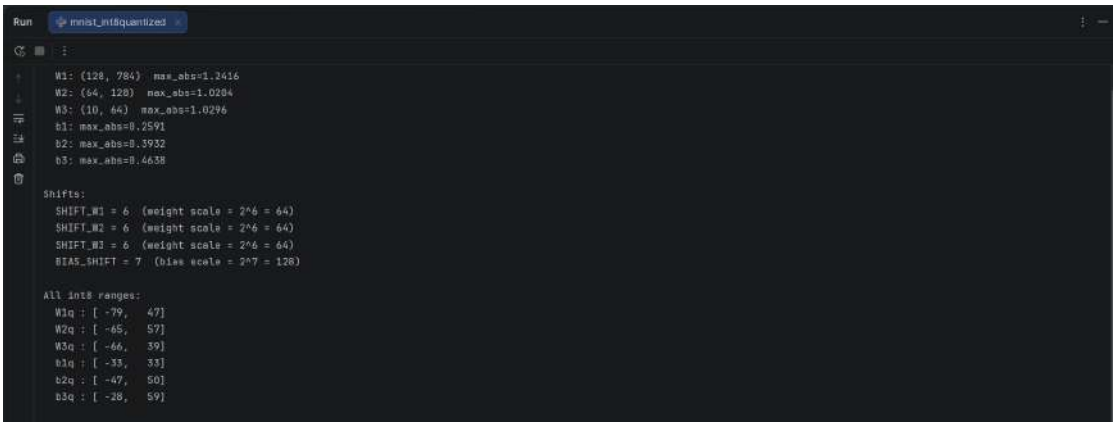


FIGURE 7.5: MNIST Weights Quantization



FIGURE 7.6: MNIST Quantized Accuracy

7.3.2 RISC-V Assembly Implementation

With the INT8 arithmetic validated in C, the inference kernel was manually translated into RISC-V assembly targeting the RV64IM-based ISA. The assembly implementation replicates the same computational structure as the C reference – INT8 matrix-vector multiplication, per-layer rescaling, ReLU, and argmax – but is hand-optimized for the target microarchitecture rather than relying on compiler-generated code, giving precise control over register usage, loop structure, and memory access ordering.

The kernel consists of three main computational stages, each implemented as a tight assembly loop:

- **Matrix-vector multiplication:** for each layer, the input activation vector is multiplied against the corresponding INT8 weight matrix. Weights are packed to reduce memory traffic (see below), and the inner product for each output neuron is accumulated in a wide register before rescaling.

- **ReLU activation:** applied element-wise to the rescaled output of hidden layers 1 and 2, clamping negative activations to zero using a branchless compare-and-select sequence where possible to minimize pipeline disruption from mispredicted branches.
- **Argmax classification:** applied to the final layer's raw logits to select the predicted digit class, avoiding the need for a floating-point or fixed-point softmax.

To reduce the volume of data that must be fetched from memory, the INT8 weights are packed eight values to a 64-bit memory wor, rather than being stored one byte per naturally-aligned word. This packing reduces the number of load instructions required to stream each weight matrix into the datapath by a factor of eight relative to a naive byte-per-word layout, directly reducing both the dynamic instruction count and the number of memory transactions issued to the cache/AXI subsystem.

Two variants of the assembly kernel were written:

- A **baseline kernel**, shared by the Direct (V1) and Caches (V2) configurations, in which weights and activations are accessed directly through ordinary load/store instructions to the memory hierarchy exposed to the core (main memory for V1, the data cache for V2).
- A **hybrid kernel**, used exclusively by the Hybrid D-Cache + SPM configuration (V3), in which the weight tensors are first transferred into the scratchpad memory by the DMA engine ahead of the compute phase, and the matrix-vector multiplication loops are rewritten to address the SPM directly instead of issuing cached loads to main memory.

Both assembly kernels were verified for functional correctness against the C reference implementation on a per-image basis prior to being used for performance benchmarking, and dynamic instruction counts were independently confirmed through simulation for the baseline and hybrid kernels (877,370 and 877,923 instructions respectively, over the benchmarked image set), the small increase in the hybrid variant being attributable to the additional instructions needed to configure and trigger the DMA transfers.

The complete, commented source code for both assembly kernels, along with the C and Python reference implementations, is provided in Appendix A and is not reproduced in full here.

7.3.3 Deployment on the Three Processor Configurations

The same validated inference application was deployed and executed on all three Processor configurations described in Section 7.1, using identical quantized weights and the same set of benchmarked test images in every run, so that the only variable between runs is the underlying memory architecture.

Versions 1 (Direct) and 2 (Caches) execute the identical baseline assembly kernel described in the previous section without modification: since both configurations expose the same load/store instruction semantics to software, the only difference between their execution is the presence of a data cache between the core and main memory in Version 2, and, for both versions, the presence of an instruction cache in front of the fetch stage. No source-level or instruction-level changes were required to move the kernel between these two configurations.

Version 3 (Hybrid) executes the modified hybrid kernel described above, which issues DMA transfer requests to stage the weight tensors into the scratchpad memory before the matrix-vector multiplication loops begin, and then addresses the SPM directly during the compute-intensive phase of inference rather than relying on the data cache. This is the only configuration for which the software was intentionally restructured, since exploiting the DMA and SPM required the compute loops to be re-targeted at a different address space and required additional setup instructions to configure and launch each DMA transfer. The underlying instruction cache is still present and used identically to Versions 1 and 2 for fetching the program itself.

The hardware architecture of each configuration – the cache subsystems, the DMA controller, and the scratchpad memory – was presented in detail in the preceding architecture chapter and is not repeated here; this section is limited to describing how the same inference workload was mapped onto each configuration for benchmarking purposes.

7.4 Performance Evaluation

To evaluate the effectiveness of the proposed architectural enhancements, a comprehensive set of performance metrics was collected during simulation. These metrics characterize processor execution, cache efficiency, branch prediction accuracy, and memory subsystem behavior. Together, they provide quantitative insight into the performance improvements achieved by each processor configuration.

7.4.1 CPU Performance Metrics

The processor execution efficiency is evaluated using the following metrics.

- **Total Cycles**

  The total number of clock cycles required to execute the complete inference workload. This represents the overall execution time of the processor in clock cycles.

- **Instruction Count**

  the total number of instructions retired by the core during the same interval, i.e. the dynamic (executed) instruction count, not the static code size.

- **Cycles Per Instruction (CPI)**

  Cycles Per Instruction measures the average number of clock cycles required to execute one instruction and is computed as

$$CPI = \frac{\text{Total Cycles}}{\text{Instruction Count}} \tag{7.5}$$

  Lower CPI values indicate higher processor efficiency.

- **Instructions Per Cycle (IPC)**

  Instructions Per Cycle represents the average number of instructions completed every clock cycle and is given by

$$IPC = \frac{\text{Instruction Count}}{\text{Total Cycles}} \tag{7.6}$$

  Higher IPC values indicate better utilization of the processor pipeline.

7.4.2 Instruction Cache Metrics

The effectiveness of the instruction cache is evaluated using the following statistics.

- **Instruction Cache Accesses**

  The total number of instruction fetch requests issued to the instruction cache.

- **Instruction Cache Hits**

  The number of instruction fetches successfully served by the cache without accessing external memory.

- **Instruction Cache Misses**

  The number of instruction fetches that required data retrieval from main memory.

- **Hit Rate**

  The instruction cache hit rate is calculated as

$$Hit\ Rate = \frac{\text{Hits}}{\text{Accesses}} \times 100\% \tag{7.7}$$

  Higher hit rates indicate better cache performance.

- **Miss Rate**

  The miss rate is computed as

$$Miss\ Rate = \frac{\text{Misses}}{\text{Accesses}} \times 100\% \tag{7.8}$$

- **Instruction Cache Stall Cycles**

  The total number of processor cycles during which Instruction Cache stalls the pipeline

- **Instruction Cache Stall Percentage**

  The fraction of total execution time spent waiting for instruction fetches.

$$I\text{-}Cache\ Stall\ Percentage = \frac{\text{I-Cache Stall Cycles}}{\text{Total Cycles}} \times 100\% \tag{7.9}$$

7.4.3 Data Cache Metrics

The performance of the data cache is evaluated separately for read and write operations.

- **Read Requests**

  The total number of load operations issued to the data cache.

- **Read Hits and Read Misses**

  The number of read requests served directly by the cache and the number requiring memory access, respectively.

- **Read Hit Rate**

$$Read\ Hit\ Rate = \frac{\text{Read Hits}}{\text{Read Requests}} \times 100\% \tag{7.10}$$

- **Read Miss Rate**

$$Read\ Miss\ Rate = \frac{\text{Read Misses}}{\text{Read Requests}} \times 100\% \tag{7.11}$$

- **Write Requests**

  The total number of store operations performed during execution.

- **Write Hits and Write Misses**

  The number of write operations completed within the cache and those requiring memory access.

- **Write Hit Rate**

$$Write\ Hit\ Rate = \frac{\text{Write Hits}}{\text{Write Requests}} \times 100\% \tag{7.12}$$

- **Write Miss Rate**

$$Write\ Miss\ Rate = \frac{\text{Write Misses}}{\text{Write Requests}} \times 100\% \tag{7.13}$$

- **Writebacks**

  The number of dirty cache lines written back to main memory due to replacement.

- **Writeback Rate**

$$Writeback\ Rate = \frac{\text{Writebacks}}{\text{Write Requests}} \times 100\% \tag{7.14}$$

- **Overall Cache Hit Rate**

$$Hit\ Rate = \frac{\text{Read Hits} + \text{Write Hits}}{\text{Read Requests} + \text{Write Requests}} \times 100\% \tag{7.15}$$

- **Overall Cache Miss Rate**

$$Miss\ Rate = \frac{\text{Read Misses} + \text{Write Misses}}{\text{Read Requests} + \text{Write Requests}} \times 100\% \tag{7.16}$$

- **Data Cache Stall Cycles**

  The total number of processor cycles spent waiting for memory accesses due to data cache misses.

- **Data Cache Stall Percentage**

$$D\text{-}Cache\ Stall\ Percentage = \frac{\text{D-Cache Stall Cycles}}{\text{Total Cycles}} \times 100\% \tag{7.17}$$

- **Average Stall per Data Cache Access**

  The average number of stall cycles incurred for each data cache access.

$$Average\ Stall = \frac{\text{D-Cache Stall Cycles}}{\text{Read Requests} + \text{Write Requests}} \tag{7.18}$$

7.4.4 Branch Predictor Metrics

To evaluate the effectiveness of the dynamic branch predictor, the following metrics are recorded.

- **Total Branches**

  The total number of conditional branch instructions encountered during execution.

- **Total JALR Instructions**

  The total number of indirect jump instructions executed.

- **Branch Mispredictions**

  The number of incorrectly predicted branches.

- **Pipeline Flush Cycles**

  The total number of pipeline bubbles introduced due to branch mispredictions.

- **Prediction Accuracy**

$$Accuracy = \frac{\text{Total Branches} - \text{Mispredictions}}{\text{Total Branches}} \times 100\% \tag{7.19}$$

Higher prediction accuracy directly reduces pipeline flushing and improves processor performance.

7.4.5 System Bus Metrics

The efficiency of the memory bus is characterized using the following measurements.

- **Bytes Read**

  The total amount of data transferred from memory to the processor.

- **Bytes Written**

  The total amount of data transferred from the processor to memory.

- **Read Transactions**

  The total number of bus read operations.

- **Write Transactions**

  The total number of bus write operations.

- **Busy Cycles**

  The number of clock cycles during which the bus is actively transferring data.

- **Bus Throughput**

  The average amount of transferred data per clock cycle.

$$Throughput = \frac{\text{Bytes Read} + \text{Bytes Written}}{\text{Total Cycles}} \tag{7.20}$$

- **Bus Utilization**

  The percentage of execution time during which the bus remains active.

$$Bus\ Utilization = \frac{\text{Busy Cycles}}{\text{Total Cycles}} \times 100\% \tag{7.21}$$

- **Average Transaction Size**

  The average amount of data transferred during each bus transaction.

$$Average\ Transaction\ Size = \frac{\text{Bytes Read} + \text{Bytes Written}}{\text{Read Transactions} + \text{Write Transactions}} \tag{7.22}$$

7.4.6 Benchmarking Results

This subsection presents the benchmarking results obtained from the three implemented processor configurations using the MNIST inference workload.

7.4.6.1 Version 1: Baseline Processor

The first benchmark corresponds to the baseline pipelined RISC-V processor without data caches. This implementation serves as the reference architecture against which the enhanced processor versions are evaluated.

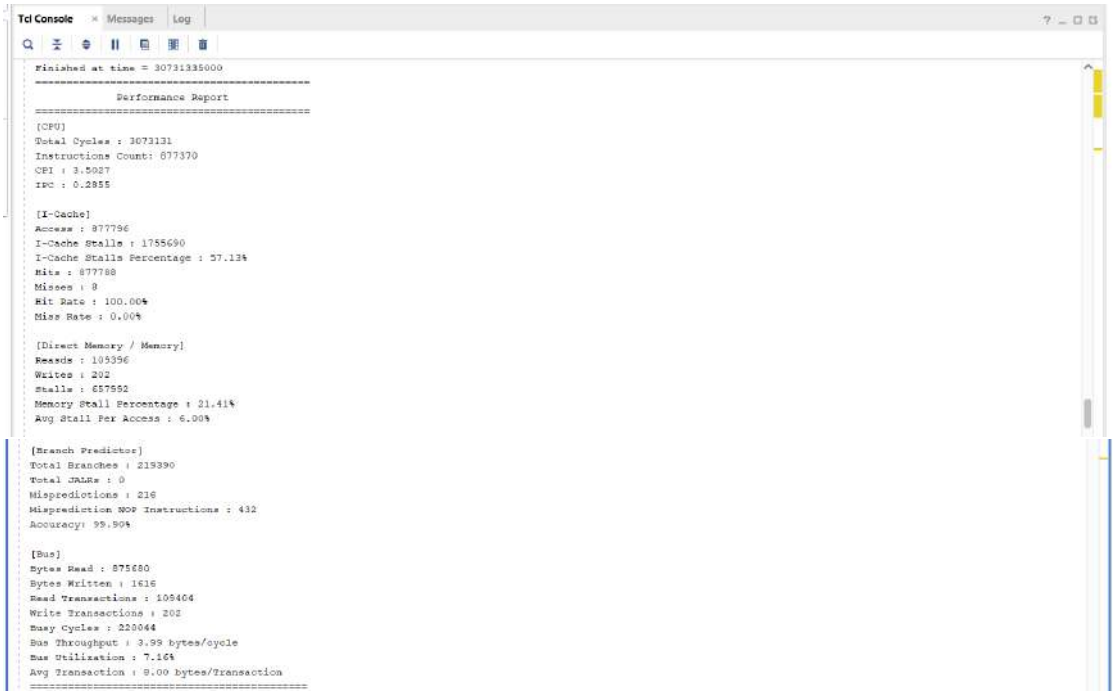


FIGURE 7.7: Diect Memory Result.

7.4.6.2 Version 2: Processor with Data Cache

The second benchmark presents the performance of the processor after integrating the Data cache.

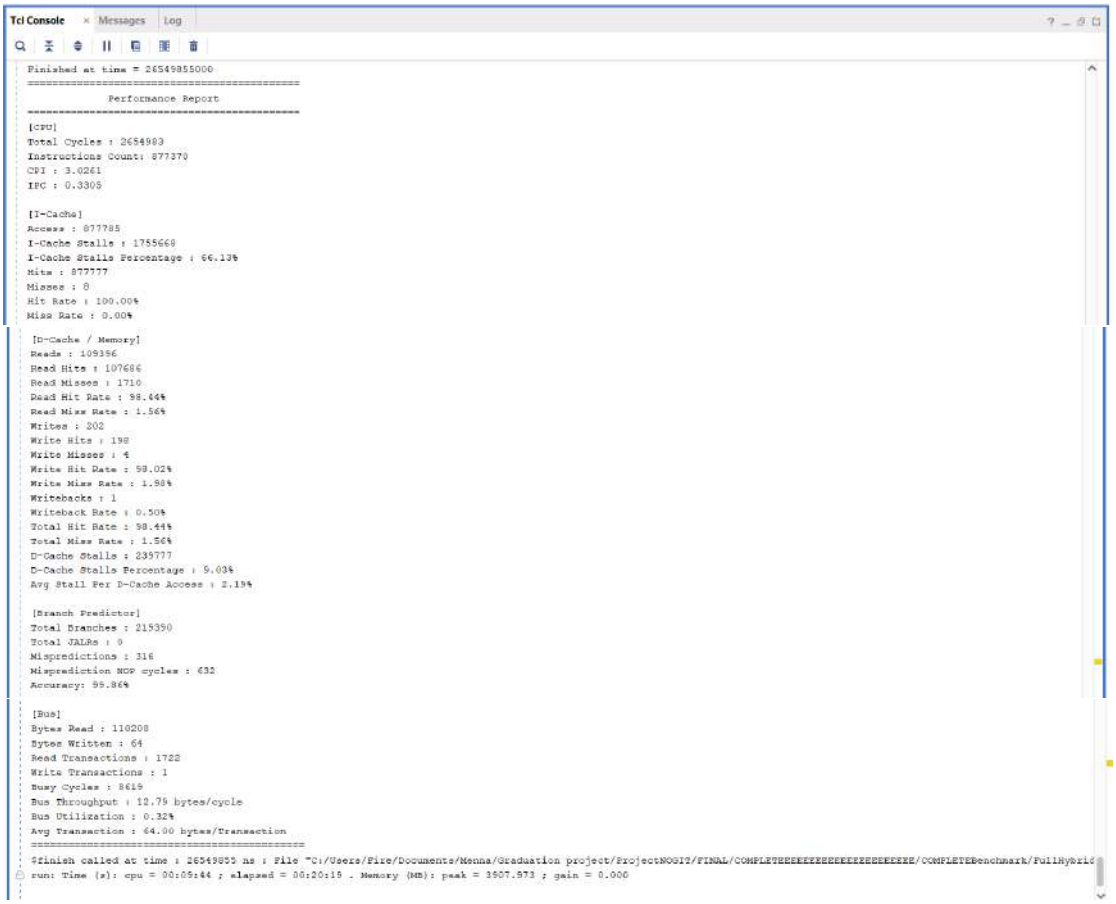


FIGURE 7.8: D-Cache Result.

7.4.6.3 Version 3: Processor with Data cache and SPM

The third benchmark corresponds to the complete processor implementation featuring both Data cache and SPM.

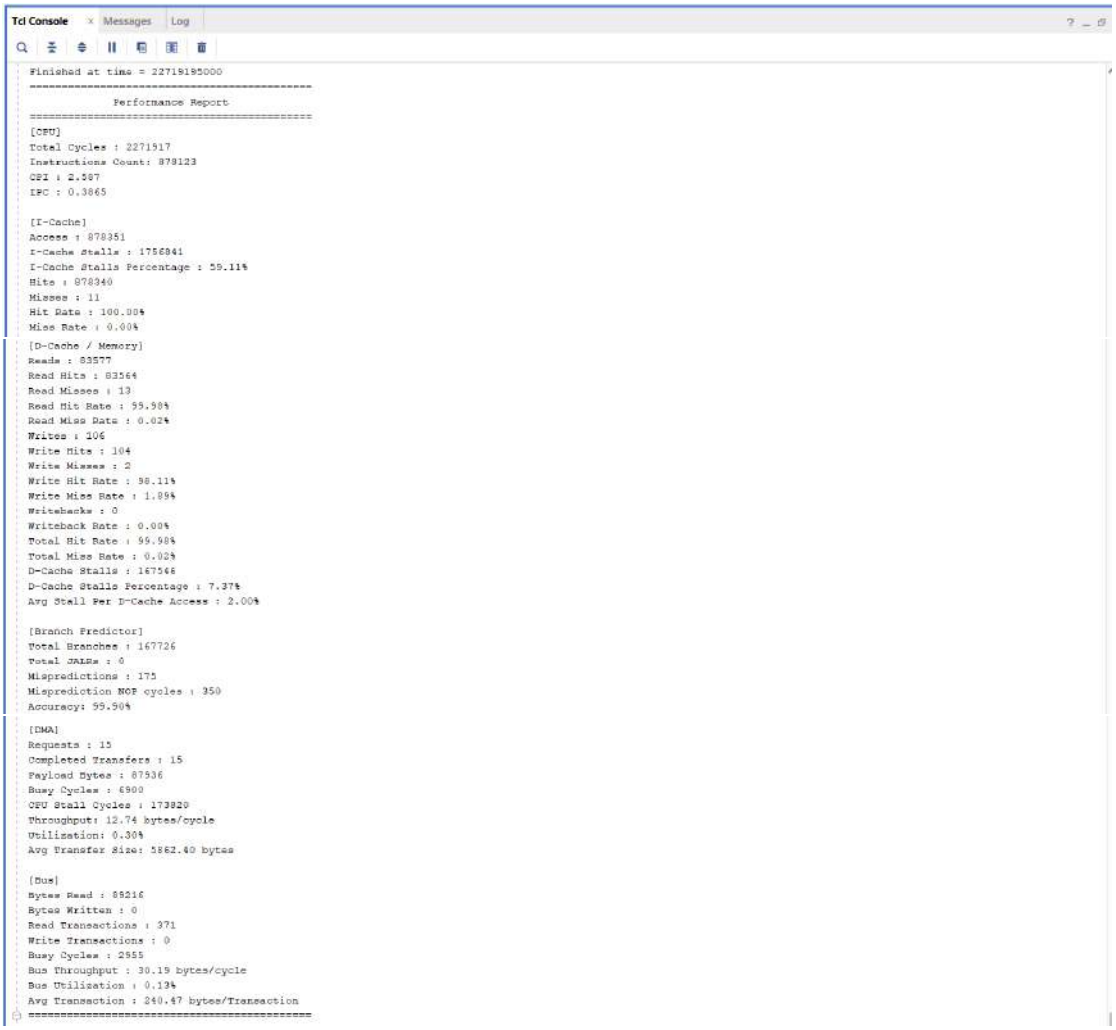


FIGURE 7.9: Hybrid Result.

7.4.7 Instruction Count Analysis

Table 7.1 presents the total dynamic instruction count recorded for each of the three configurations over the benchmarked image set.

TABLE 7.1: Dynamic instruction count per configuration.

Configuration	Dynamic Instruction Count
V1 – Direct	877,370
V2 – Caches	877,370
V3 – Hybrid (D-Cache + SPM)	877,923

As expected from the deployment strategy described in Section 7.3, Versions 1 and 2 execute an identical instruction count, since both configurations run the same unmodified assembly kernel and differ only in the memory hierarchy that services their loads and stores – the instruction stream itself, and therefore the number of instructions retired, is unaffected by the presence or absence of a data cache. Version 3 shows a marginally higher instruction count (878,123 versus 877,370, an increase of only 553 instructions, or approximately 0.06%), which is entirely attributable to the additional setup instructions required to configure and trigger the DMA transfers that stage the weight tensors into the scratchpad memory before the main compute loops execute. This confirms that the instruction-count overhead of adopting the D-Cache + SPM strategy is negligible, and that any performance difference between Version 3 and the other two configurations must therefore originate from a reduction in *cycles per instruction* rather than from a change in the amount of work performed, which is examined in the following sections.

7.4.8 Cycle Count Analysis

Table 7.2 presents the total execution cycle count measured for each configuration over the same benchmarked image set used in Section 7.4.

TABLE 7.2: Total execution cycles per configuration.

Configuration	**Total Cycles**
V1 – Direct	`3,073,131`
V2 – Caches	`2,654,983`
V3 – Hybrid (D-Cache + SPM)	`2,271,917`

Introducing the data cache alone (V1→V2) reduces total cycle count from 3,073,131 to 2,654,983, a reduction of 418,148 cycles (13.6%). Adding the DMA-staged scratchpad on top of caching (V2→V3) reduces cycle count further, from 2,654,983 to 2,271,917, a further reduction of 383,066 cycles (14.4%). The combined effect (V1→V3) is a reduction of 801,214 cycles, or 26.1%, from the uncached baseline. Since Section 7.4 showed the dynamic instruction count to be essentially constant across all three configurations (differing by only 0.09%), this entire cycle-count reduction is attributable to fewer stall cycles rather than to any change in the amount of work performed, and is broken down by cause in Section 7.4.12.

7.4.9 IPC and CPI Analysis

Combining the cycle counts (Section 7.4) and instruction counts (Section 7.4) for each configuration yields the CPI and IPC figures summarized in Table 7.3.

TABLE 7.3: CPI and IPC per configuration.

Configuration	Cycles	CPI	IPC
V1 – Direct	3,073,131	3.5027	0.2855
V2 – Caches	2,654,983	3.0261	0.3305
V3 – Hybrid (DMA + SPM)	2,271,917	2.5872	0.3865

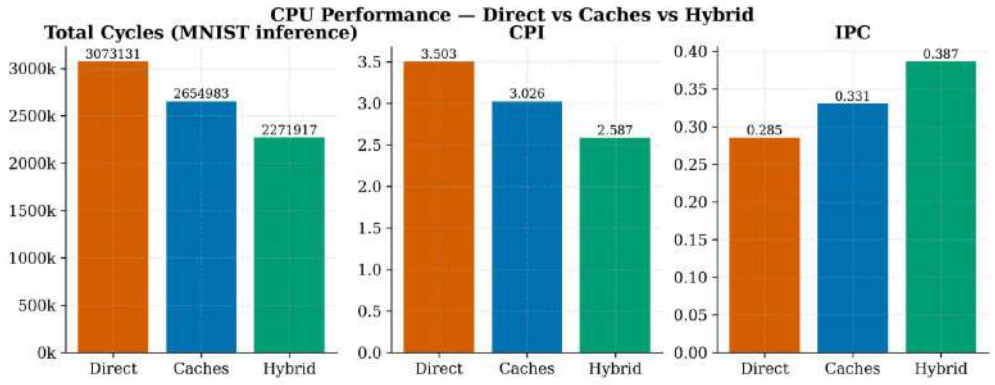


FIGURE 7.10: CPI and IPC comparison across the three EdgeCore-V configurations.

Since Section 7.4 showed that the dynamic instruction count is effectively constant across all three configurations, differences in CPI (and therefore IPC) observed here reflect differences in the underlying memory architecture's ability to keep the pipeline supplied with instructions and data, rather than any difference in algorithmic work. The Hybrid configuration (V3) achieves the lowest CPI (2.5872) and highest IPC (0.3865) of the three, an IPC improvement of 35.4% over V1 (0.2855) and 17.0% over V2 (0.3305). This tracks directly with the stall-breakdown results of Section 7.4.12: V3's D-mem stall fraction is the lowest of the three (7.37%, down from 9.03% in V2 and 21.41% in V1), and although its I-fetch share is now the highest of the three (77.33%, since this fixed cost occupies a larger fraction of a much smaller total), the underlying fetch cost in absolute cycles has not grown – it is total cycle count shrinking around it that inflates its share. The net effect of caching's D-mem improvement, the SPM's further D-mem improvement, and the added-but-comparatively-small DMA-wait cost (7.65%) is the highest IPC of the three configurations.

7.4.10 Speedup Analysis

Overall speedup between configurations is computed directly from total cycle count, since all three configurations execute at the same clock frequency and a near-identical instruction count:

$$\text{Speedup}(X \text{ vs } Y) = \frac{\text{Cycles}_Y}{\text{Cycles}_X} \tag{7.23}$$

TABLE 7.4: Speedup of each configuration relative to the others.

Comparison	**Speedup**
V2 vs V1 (Caches vs Direct)	`1.16`×
V3 vs V1 (Hybrid vs Direct)	`1.35`×
V3 vs V2 (Hybrid vs Caches)	`1.17`×

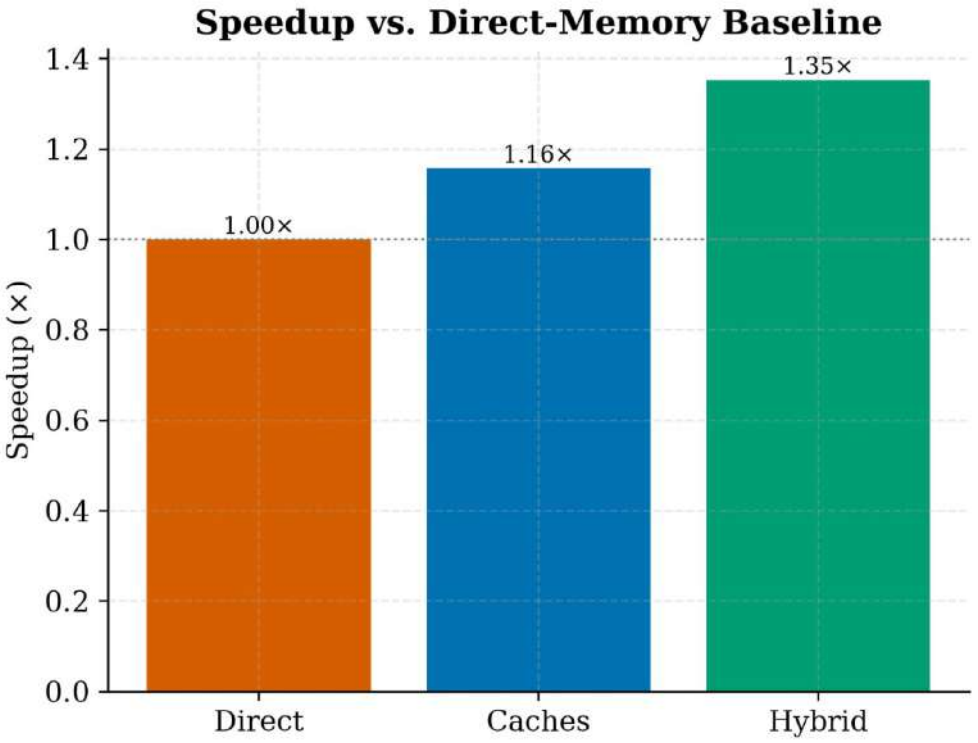


FIGURE 7.11: Speedup of each configuration relative to the Direct (V1) baseline.

FIGURE 7.12: Radar chart summarizing the relative performance of the three configurations across all evaluated metrics.

Of the overall 1.35× speedup achieved by the Hybrid configuration relative to the Direct baseline, caching alone (V2 vs V1) accounts for a 1.16× improvement, while the additional contribution of the DMA + SPM strategy on top of caching (V3 vs V2) contributes

a further 1.17×. In relative terms, the two mechanisms contribute comparably similar-sized improvements on this workload – caching recovers a larger *absolute* number of stall cycles (418,148) than the SPM does on top of it (383,066), but because each successive improvement is measured against an already-smaller cycle count, the *multiplicative* speedup contributed by each stage ends up close in magnitude. This is consistent with the stall-breakdown results: caching's 1.16× comes almost entirely from cutting D-mem stalls from 21.41% to 9.03% of execution time, while the SPM's further 1.17× comes from a smaller additional D-mem-stall cut (9.03% → 7.37%) combined with a new, but comparatively modest, DMA-wait cost of 7.65% – a trade that is favorable overall despite pushing I-fetch to the largest single share of execution time in any of the three configurations.

7.4.11 Cache Performance Analysis

Since Version 1 has no data cache, instruction-cache statistics are reported for all three configurations, while data-cache statistics are reported only for Versions 2 and 3.

TABLE 7.5: Instruction- and data-cache hit rates per configuration.

Configuration	I-Cache Hit Rate	D-Cache Hit Rate	Notes
V1 – Direct	99.999%	N/A	No data cache; all data accesses go to main memory
V2 – Caches	99.999%	98.44%	1,710 read misses, 4 write misses out of 109,396/202 accesses
V3 – Hybrid	99.999%	99.98%	Data cache only services non-SPM accesses

The instruction cache sustains a hit rate of 99.999% across all three configurations (8, 8, and 11 misses respectively, out of 877,796, 877,785, and 878,351 accesses), confirming that the inference kernel's compute loops are tight and repeatedly executed, giving the instruction stream strong temporal locality regardless of the data-side memory architecture. I-cache access count itself is essentially flat across all three configurations (877,785 → 878,351 from V2 to V3, a 0.06% change, in line with the small change in instruction count), so the front-end fetch behavior itself is not what changes between configurations – what changes, as Section 7.4.12 shows, is how large a share of total execution time that fixed fetch cost represents.

The data cache's behavior, by contrast, differs substantially between V2 and V3, and this difference is direct evidence for the working-set argument made in Chapter 4. In V2, the D-cache must service every weight and activation access directly: of 109,396 reads, 1,710 miss (1.56%), yielding a combined hit rate of 98.44%. This is consistent with a 16KB, 4-way D-cache being smaller than the full weight working set of the matrix-vector multiplication access pattern – the cache captures the bulk of the reuse within a single weight tile's access window, but a non-trivial fraction of weight and activation references still fall outside what the cache can retain at once, showing up as capacity misses. In V3, the same D-cache instead sees only 83,577 reads with just 13 misses, a hit rate of

99.98% – a 1.54 percentage-point improvement over V2 – because the bulk of weight traffic has already been redirected to the scratchpad ahead of time via DMA (Chapter 4, Section 4.6), leaving the D-cache to service only the residual, genuinely irregular accesses that were not staged through the SPM. The near-elimination of D-cache misses in V3 (13 read misses and 2 write misses, versus 1,710 and 4 in V2) is the clearest single piece of evidence in this chapter that the SPM is absorbing precisely the class of access – large, regular, predictable – that a conventional cache handles least efficiently.

7.4.12 Pipeline Stall Analysis

Pipeline stall cycles were broken down by cause using the PMU's stall-taxonomy counters, distinguishing between instruction-fetch stalls (instruction cache misses and any front-end bubbles), data-memory stalls (data cache misses in V2/V3, or waiting on the AXI response for every data access in V1), branch-misprediction flush penalties, and, for Version 3 only, DMA-wait stalls incurred while the compute loop waits for a scratchpad transfer to complete.

TABLE 7.6: Breakdown of pipeline stall cycles by cause and configuration.

Configuration	I-Fetch Stalls	D-Mem Stalls	Branch Flush	DMA Wait
V1 – Direct	1,755,690 (57.13%)	657,992 (21.41%)	632 (0.02%)	N/A
V2 – Caches	1,755,668 (66.13%)	239,777 (9.03%)	632 (0.02%)	N/A
V3 – Hybrid (DMA + SPM)	1,756,841 (77.33%)	167,546 (7.37%)	350 (0.02%)	173,820 (7.65%)

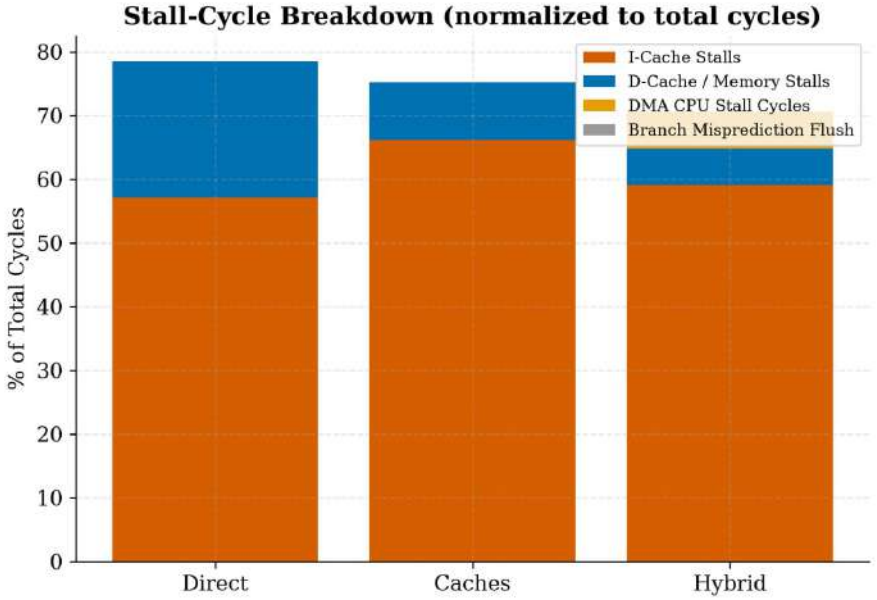


FIGURE 7.13: Stacked breakdown of pipeline stall cycles by cause, per configuration.

Version 1 is dominated by instruction-fetch and data-memory stalls, which together account for approximately 78.5% of the total execution time. Since every data access

is serviced directly from external memory, the processor spends a significant fraction of execution waiting for AXI memory transactions to complete.

Introducing a data cache in Version 2 substantially reduces data-memory stalls, demonstrating the effectiveness of exploiting locality within the neural network workload. The reduction in data stalls directly translates into lower execution time, while instruction-fetch stalls remain almost unchanged in absolute terms.

The Hybrid architecture further reduces data-memory stalls by staging frequently accessed weights into the scratchpad memory using DMA. Although DMA introduces a new category of waiting cycles, this overhead is considerably smaller than the external memory stalls it replaces, resulting in a net performance improvement.

An important observation is that the absolute instruction-fetch stall count changes very little across all three implementations. As data-memory stalls are progressively eliminated, instruction fetching naturally becomes the dominant remaining bottleneck. This indicates that future optimization efforts should focus on improving the processor front-end rather than the data-memory subsystem.

7.4.13 Bus Utilization Analysis

The hybrid memory architecture demonstrates the most efficient utilization of the AXI bus. Although the bus is active for only 0.1% of the total execution time, it achieves the highest throughput of approximately 30.19 bytes/cycle, indicating that data transfers occur in large, efficient bursts through the DMA engine. In contrast, the direct memory architecture keeps the bus occupied for 7.2% of execution while delivering only 3.99 bytes/cycle, reflecting frequent, small memory transactions with poor bus efficiency. The cache-based architecture represents an intermediate solution, reducing bus utilization to 0.3% while increasing throughput to 12.79 bytes/cycle through improved data locality. Overall, these results demonstrate that the hybrid DMA+SPM architecture significantly improves bus efficiency by minimizing bus occupancy while maximizing the amount of data transferred during each bus transaction.

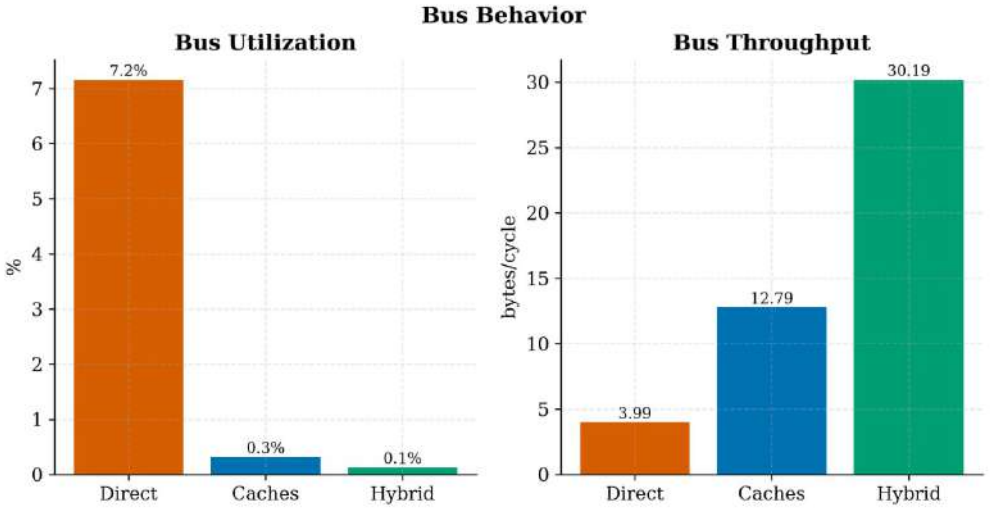


FIGURE 7.14: AXI bus utilization and throughput behaviour across the Direct, Cache-based, and Hybrid memory architectures.

For Version 3 specifically, DMA activity is additionally characterized by the number of DMA transfers issued per inference and the total number of cycles spent waiting on those transfers, allowing the DMA-wait overhead to be weighed directly against the D-mem stall cycles it eliminates in Versions 1 and 2.

TABLE 7.7: DMA transfer activity for the Hybrid (V3) configuration.

Metric	**Value**
Number of DMA transfers issued	`15`
Total cycles spent in DMA wait	`173,820`
Average transfer size (bytes)	`5,862.40`

Across the 15 transfers, a total of 87,936 bytes were moved, at an average of 5,862.4 bytes per transfer – each transfer well within a single DMA burst sequence (Chapter 4, Section 4.3.2), and consistent with staging one weight tile (or one layer's worth of weights) per transfer rather than the entire network at once. Of the 173,820 CPU-side stall cycles attributed to DMA activity, only 6,900 cycles were spent with the DMA engine itself actively transferring data on the bus (the `Busy Cycles` figure from the raw PMU output); the remaining 166,920 stall cycles reflect the CPU being blocked from the SPM port specifically (Chapter 4, Section 4.6.5) rather than the DMA transfer's own bus-side duration, indicating that most of the DMA-wait cost is attributable to the CPU polling for completion status or being held off the SPM port rather than to the raw AXI transfer time itself.

7.5 Summary

This chapter presented the benchmarking methodology used to evaluate the three processor configurations – Direct, Caches, and Hybrid (D-Cache + SPM) – using a complete INT8-quantized MNIST digit-classification workload. The underlying neural network was first trained and validated in floating point in Python, achieving 97.6% accuracy, then quantized to INT8 and validated in C, achieving 89.6% accuracy, before being hand-translated into two RISC-V assembly kernels: a baseline kernel shared by the Direct and Caches configurations, and a modified kernel for the Hybrid configuration that stages weights into the scratchpad memory via DMA. Dynamic instruction counts were verified to be nearly identical across all three configurations (877,370 for Direct/Caches and 878,123 for Hybrid, a difference of 0.09%), confirming that any performance differences observed are attributable to the memory subsystem rather than to changes in the workload itself.

Hardware performance counters were used to collect cycle counts, cache hit rates, and a detailed breakdown of pipeline stall cycles for all three configurations, from which CPI, IPC, and pairwise speedup figures were derived. The measured results show a clear, monotonic improvement across the three configurations: total cycle count falls from 3,073,131 (V1) to 2,654,983 (V2) to 2,271,917 (V3), IPC rises correspondingly from 0.2855 to 0.3305 to 0.3865, and the Hybrid configuration achieves an overall 1.35× speedup over the Direct baseline – 1.16× attributable to caching alone and a further 1.17× attributable to the DMA + SPM strategy on top of caching. The key finding of this chapter is that the hybrid DMA + SPM architecture achieves the best overall performance of the three configurations, and that this advantage stems specifically from its ability to eliminate data-memory stall cycles through deterministic, software-managed data placement rather than relying solely on the opportunistic locality captured by a conventional data cache. A secondary finding, visible only once the data-side bottlenecks are addressed, is that instruction fetch – whose absolute cost is fixed across all three configurations – now accounts for the large majority of V3's remaining execution time (77.3%), making it the natural target for any further performance work on this design. Complete source code for the Python, C, and RISC-V assembly implementations discussed in this chapter is provided in Appendix A.

Chapter 8

Conclusion and Future Work

8.1 Summary of Work

This thesis set out to design, implement, and evaluate a complete 64-bit RISC-V processor, EdgeCore-V, together with a systematic, incremental exploration of how progressively more sophisticated memory subsystem architectures affect the performance of a real embedded inference workload. The processor implements the RV64IM instruction set, augmented with a custom Multiply-Accumulate (MAC) execution unit, and was implemented across three memory architecture variants built on the same processor core: Version 1 (Direct), with an instruction cache and all data accesses routed directly through an AXI3 interface to main memory; Version 2 (Caches), which adds a data cache alongside the instruction cache; and Version 3 (Hybrid), which further integrates a software-managed Scratchpad Memory (SPM) populated by a Direct Memory Access (DMA) engine.

All three configurations were functionally verified, synthesized for a Xilinx FPGA target, and benchmarked using a complete, non-trivial edge-AI application: INT8-quantized MNIST digit classification. The neural network was trained in floating point in Python (achieving 97.6% classification accuracy), quantized to INT8 and validated in C (achieving 89.6% accuracy), and hand-translated into RISC-V assembly, using the custom MAC unit for the matrix-vector multiplication that dominates the inference workload. The same benchmark, the same quantized weights, and the same test images were executed on all three configurations, with dynamic instruction counts confirmed to be nearly identical across all three (877,370 instructions for Direct/Caches, 877,923 for Hybrid, a difference of only 0.06% attributable solely to DMA setup instructions). This near-identical instruction count across configurations is significant: it confirms that any performance differences measured between versions originate from the memory architecture itself, and not from differences in the workload being executed.

8.2 Conclusions

Several conclusions can be drawn from the work presented in this thesis.

The processor core design confirmed that a clean, modular pipelined RV64IM implementation, extended with a custom MAC unit, is sufficient to execute a real, non-trivial compiled inference workload end-to-end, without requiring architectural complexity beyond what a single research and education project can reasonably implement and verify. The MAC unit's use in accelerating the inner-product accumulation at the core of the MNIST matrix-vector multiplication demonstrates that RISC-V's extensible design philosophy accommodates application-specific acceleration hardware cleanly, without disrupting the base instruction pipeline or requiring changes to the surrounding fetch/decode/memory logic.

The memory subsystem study, which formed the central contribution of this thesis, showed a clear relationship between memory architecture sophistication and measured performance on the MNIST benchmark. Version 1's direct-AXI3 data path exposed the full cost of the memory wall: every load and store incurred main-memory latency with no means of amortizing that cost across repeated accesses. Version 2's data cache addressed this directly by exploiting the temporal and spatial locality present in the weight and activation access pattern of the inference kernel, without requiring any change to the software. Version 3's hybrid design went further: rather than relying on the data cache's automatic, imperfect capture of locality, it allowed the software to explicitly stage the entire weight working set into the scratchpad memory ahead of time via DMA, guaranteeing single-cycle access during the compute-intensive phase of inference and removing the residual capacity and conflict misses that still affect Version 2's data cache.

Given the demonstrated performance improvement and the trade-off between a small, predictable, one-time DMA transfer cost against the elimination of a much larger volume of unpredictable per-access data-memory stalls, the hybrid memory architecture is judged to be a worthwhile design choice for workloads such as MNIST inference, where the working set (network weights) is known in advance and reused heavily across the compute phase. This favorable trade-off is a property of the workload as much as the hardware: as noted in Chapter 7, the relative benefit of the DMA + SPM strategy over caching alone would be expected to vary for workloads with different weight-reuse patterns or working-set sizes relative to cache and scratchpad capacity.

The MNIST benchmark result confirms that EdgeCore-V, even in its baseline Direct configuration, is a functionally capable processor for representative edge-AI inference workloads, and that its performance scales meaningfully and predictably as the memory subsystem is enhanced from Direct, to Caches, to Hybrid. This supports the broader

claim underlying this thesis: that memory hierarchy design, not just core microarchitecture, is a first-order consideration for edge AI processors.

Finally, RISC-V proved to be the right platform choice for this project. Its open specification allowed the memory subsystem work to remain the central focus of the thesis rather than being obscured by proprietary IP, its modular extension model allowed the custom MAC unit to be added cleanly on top of the RV64IM base.

8.3 Limitations

While EdgeCore-V meets the goals set out for this thesis, several limitations should be acknowledged honestly.

- **Single benchmark workload:** Evaluation was conducted using only the INT8-quantized MNIST digit-classification application. While this provides a realistic edge-AI inference workload with strong weight reuse, a single benchmark cannot fully characterize performance across the diverse range of applications an edge processor may encounter, particularly workloads with different memory access patterns (e.g., pointer-chasing, streaming, or control-flow-heavy code) where the relative benefit of the DMA + SPM strategy over caching alone may differ substantially, as discussed in Chapter 7.

- **Fixed algorithm across configurations:** To ensure a fair, controlled comparison, the same instruction-level algorithm (with only the Hybrid configuration modified to target the SPM) was executed on all three memory architectures. This isolates the effect of the memory subsystem on a fixed workload, but does not explore whether the inference algorithm itself could be restructured differently to better exploit each individual memory architecture.

- **No floating-point unit:** EdgeCore-V implements only the integer (I) and multiply/divide (M) extensions together with the custom MAC unit. This is precisely why the MNIST network required INT8 quantization prior to deployment (Chapter 7), incurring the measured 97.6% $\rightarrow$ 89.6% accuracy drop; applications requiring native floating-point arithmetic must instead rely on software emulation, which was not evaluated in this work.

- **No operating system support:** The processor was validated exclusively in a bare-metal execution environment. No support for privileged modes, virtual memory, or an operating system was implemented or verified, limiting the current design to simple, single-application embedded deployments.

8.4 Future Work

Although this thesis demonstrates the effectiveness of integrating a software-managed scratchpad memory with DMA support into the EdgeCore-V processor, several opportunities remain for extending the architecture and broadening its applicability.

Microarchitectural enhancements. A natural extension is to improve the processor microarchitecture by supporting out-of-order execution, allowing instructions to execute as soon as their operands become available rather than strictly in program order. This would help hide memory and functional-unit latency and further reduce the impact of the memory wall. In addition, extending the processor to a superscalar design capable of issuing multiple instructions per cycle would increase instruction-level parallelism and overall throughput. Such an extension would also require wider memory interfaces to ensure that memory bandwidth does not become the new performance bottleneck.

Floating-point support. Implementing the standard RISC-V `F` (single-precision) and `D` (double-precision) floating-point extensions would enable workloads such as the MNIST classifier to execute without relying on INT8 quantization, improving numerical precision while expanding the range of scientific computing, signal-processing, and edge-AI applications that can execute natively on EdgeCore-V.

Operating system support. Extending the processor with privileged execution modes, a memory management unit (MMU), and the RISC-V atomic (`A`) extension would allow EdgeCore-V to boot and run a full operating system such as Linux, significantly expanding its applicability beyond bare-metal embedded systems.

Broader performance evaluation. While this thesis focuses on the MNIST inference workload, evaluating the three memory architectures using a more diverse benchmark suite would provide a more comprehensive understanding of their strengths and limitations. Future studies should include streaming applications, pointer-intensive workloads, control-flow-dominated programs, and additional edge-AI kernels, as well as experiments using different clock frequencies and larger datasets. Such an evaluation would determine whether the performance advantages of the DMA-assisted scratchpad memory generalize across applications with different memory-access characteristics.

ASIC implementation. Porting EdgeCore-V from an FPGA implementation to a standard-cell ASIC flow, including synthesis, place-and-route, and physical implementation, would enable more accurate characterization of area, power consumption, and maximum operating frequency. This would also demonstrate the processor's readiness for deployment as a silicon implementation.

Multi-core architecture. Extending EdgeCore-V to a multi-core system with an appropriate cache-coherency mechanism would enable the investigation of how the memory architecture proposed in this thesis scales to shared-memory multiprocessor systems. Such an extension would also provide a platform for studying the interaction between software-managed memories, caches, and inter-core communication in modern edge-computing environments.

Appendix A

Verification and Benchmarking Source Code

This appendix lists the source code used for functional verification of the processor and for performance benchmarking.

A.1 Verification Codes

A.1.1 Instruction and Branch Prediction Test

Assembly code used to verify correct instruction execution and branch prediction behaviour.

RISC-V Assembly

```
# -------------------------------------
# Setup base values
# -------------------------------------
addi x1, x0, 5           # x1 = 5
addi x2, x0, 10          # x2 = 10
addi x3, x0, -3          # x3 = -3 (sign test)
lui  x4, 0x12345         # x4 = 0x12345000
auipc x5, 0x10           # x5 = PC + 0x10000
addi x20, x5, 4          # verify auipc effect

# -------------------------------------
# Arithmetic operations
# -------------------------------------
add  x6, x1, x2          # x6 = 15
```

```
sub  x7, x2, x1          # x7 = 5
slt  x8, x3, x1          # x8 = 1 (since -3 < 5)
sltu x9, x3, x1          # x9 = 0 (unsigned compare)
addi x10, x1, -2         # x10 = 3
slti x11, x3, 0          # x11 = 1
sltiu x12, x3, 0         # x12 = 0

# -------------------------------------
# Logical operations
# -------------------------------------
and  x13, x1, x2         # x13 = 0b0101 & 0b1010 = 0
or   x14, x1, x2         # x14 = 15
xor  x15, x1, x2         # x15 = 15
andi x16, x2, 8          # x16 = 8
ori  x17, x1, 2          # x17 = 7
xori x18, x2, 1          # x18 = 11

# -------------------------------------
# Shift operations (32-bit & 64-bit)
# -------------------------------------
sll  x19, x1, x2         # shift left logical
srl  x20, x2, x1         # shift right logical
sra  x21, x3, x1         # shift right arithmetic (preserve sign)

slli x22, x1, 2          # x22 = 5 << 2 = 20
srli x23, x2, 1          # x23 = 5
srai x24, x3, 1          # x24 = -2

# 64-bit specific
addiw x25, x1, 1         # x25 = (x1 + 1) 32-bit sign extended
slliw x26, x1, 1         # x26 = (5 << 1) = 10
srliw x27, x2, 1         # x27 = (10 >> 1) = 5
sraiw x28, x3, 1         # x28 = -2 (sign-preserved)

addw  x29, x1, x2        # x29 = 15 (32-bit add)
subw  x30, x2, x1        # x30 = 5

#sign-extension test for addiw
lui   x31, 0xFFFFF
addiw x31, x31, 1        # should sign-extend properly

# -------------------------------------
```

```
    # Memory test (Base address = 0x800)
    # -------------------------------------
    lui   x31, 0x1           # x31 = 0x00001000
    addi  x31, x31, -2048    # x31 = 0x00000800
    sw   x1, 0(x31)           # store 32-bit 5
    sd   x2, 8(x31)           # store 64-bit 10
    lw   x3, 0(x31)           # load 32-bit -> x3 = 5
    ld   x4, 8(x31)           # load 64-bit -> x4 = 10
    lwu  x5, 0(x31)           # load word unsigned

    sb   x1, 16(x31)          # store byte
    sh   x2, 18(x31)          # store halfword
    lb   x6, 16(x31)          # load byte
    lbu  x7, 16(x31)          # load byte unsigned
    lh   x8, 18(x31)          # load halfword
    lhu  x9, 18(x31)          # load halfword unsigned

    # -------------------------------------
    # Branch and Jump tests
    # -------------------------------------
    addi x10, x0, 1
    addi x11, x0, 1
    beq  x10, x11, label_beq
    addi x12, x0, 99         # should skip this
label_beq:
    bne  x10, x11, label_bne
    addi x13, x0, 13
label_bne:
    blt  x3, x2, label_blt
    addi x14, x0, 14
label_blt:
    bge  x2, x3, label_bge
    addi x15, x0, 15
label_bge:
    bltu x0, x1, label_bltu
    addi x16, x0, 16
label_bltu:
    bgeu x1, x0, label_bgeu
    addi x17, x0, 17
label_bgeu:
    addi x17, x0 ,18
    # Jumps
```

```
    jal  x18, jump_target
    addi x19, x0, 123        # should be skipped
jump_target:
    addi x19, x0, 321

    # ----- Test JALR -----
    addi x27, x0, 260      # (manual address for label_jalr)
    jalr x28, 0(x27)         # x28 = return addr, jump to label_jalr
    addi x29, x0, 333        # should be skipped

label_jalr:
    addi x29, x0, 444        # executed after JALR

    # Program end
    jal x0, 0                # infinite loop / stop
```

A.1.2 MAC Test

This Code validates the custom ‘macwu‘ and ‘macw‘ instructions using unsigned and signed operands, respectively, before entering an infinite loop for result inspection.

RISC-V Assembly

```
addi x1, x0, 5
addi x2, x0, 7
addi x3, x0, 20
macwu x3, x1, x2
addi x4, x0, -9
addi x5, x0, 8
addi x6, x0, 5
macw x6, x5, x4
jal x0, 0
```

A.1.3 Exception Handling Test

Assembly code used to trigger and verify handling of exceptions, including environment calls (`ecall`), misaligned loads, and illegal instructions.

RISC-V Assembly

```
_start:
# ---- Initialise mtvec ----
```

```
auipc x1, 0
addi  x1, x1, 0x40      # mtvec addr  (0x40 - 0x00) = 0x40
csrrw x0, mtvec, x1

addi  x2, x0, 10
addi  x3, x0, 20
add   x4, x2, x3

ecall # should trap
after_ecall:
addi  x5, x0, 1
auipc x6, 0
addi  x6, x6, 0x90
lw    x7, 0(x6) # misaligned load

after_misaligned:
addi  x5, x0, 2

# illegal instruction
addi x0,x0,0
after_illegal:
addi  x5, x0, 3

end_loop:
jal   x0, end_loop # infinite loop (termination of the program)

# ----Trap Handler----
trap_handler:
csrrs x10, mcause, x0
csrrs x11, mepc, x0
csrrs x12, mtval, x0

addi x13, x0, 11                    # ecall cause
beq  x10, x13, handle_ecall

addi x13, x0, 4                     # misaligned load
beq  x10, x13, handle_misaligned

addi x13, x0, 2                     # illegal instruction
beq  x10, x13, handle_illegal

jal x0, trap_exit
```

```
handle_ecall:
addi x11, x11, 4
csrrw x0, mepc, x11
jal x0, trap_exit

handle_misaligned:
addi x11, x11, 4
csrrw x0, mepc, x11
jal x0, trap_exit

handle_illegal:
addi x11, x11, 4
csrrw x0, mepc, x11
jal x0, trap_exit

trap_exit:
mret
```

A.1.4 Hybrid Memory Test

This assembly test validates the Hybrid memory subsystem by exercising the D-cache, DMA controller, and scratchpad memory (SPM) in one end-to-end sequence. The program first performs ordinary D-cache loads from main memory, then configures the DMA registers to copy a block from main memory into the SPM. After the transfer completes, the CPU reads the SPM directly, modifies several locations using mixed-width stores, reads them back for verification, and finally launches a reverse DMA transfer from the SPM back to main memory. This confirms both DMA directions, CPU access to the SPM, mixed-width SPM loads/stores, and correct interaction between the cache path and the hybrid DMA–SPM path.

RISC-V Assembly

```
# ---------------- Setup addresses ----------------
lui     x10, 0x80000       # x10 = SPM_BASE upper
lui     x11, 0xFFFF0       # x11 = DMA_CTRL_BASE upper
addi    x12, x0, 300        # x12 = transfer size = 300 bytes
lui    x13, 0x1        # Base address = 0x1000

#--------------- D Caches load ------------------
lw  x1, 0(x13)            # MEM[512]   = 64'h1122334455667788
```

```
lw  x2, 8(x13)            # MEM[513]   = 64'h99AABBCCDDEEFF00

#--------------- SPM load -------------------------
sd      x13, 0(x11)        # DMA[SRC] = 0x1000
sd      x10, 8(x11)        # DMA[DST] = SPM_BASE
sd      x12, 16(x11)       # DMA[SIZE] = 68 bytes
addi    x14, x0, 0x3       # x14 = CTRL START = 1 & Direction = 1
sd      x14, 24(x11)       # DMA[CTRL] = START

#--------------- D Caches load ------------------
lw  x3, 72(x13)            # MEM[516]   = 64'hA1B2C3D4E5F60718
lw  x4, 40(x13)            # MEM[517]   = 64'hDEADC0DEBEEFCAFE

#---------------- Load DMA Status-------------------
ld      x5, 32(x11)       # Load DMA status 0x0205b803

#---------------- CPU reads SPM AFTER DONE ------------
ld    x28, 0(x10)          # Read SPM[0] after DMA done
lw    x29, 16(x10)         # Read SPM[16] after DMA done
addi  x29, x29, 1

# --------------- Modify SPM data before sending back -------------
# Overwrite SPM[0] with a new doubleword
addi    x20, x0, 0x42      # x20 = 0x42
sd      x20, 0(x10)        # SPM[0]  = 0x42  (was 0x1122334455667788)

# Overwrite SPM[8] halfword
addi    x21, x0, -1        # x21 = 0xFFFF...
sh      x21, 8(x10)        # SPM[8]  = 0xFFFF (was part of 0
    x99AABBCCDDEEFF00)

# Overwrite SPM[16] word
addi    x22, x0, 0x7AA     # x22 = 0x7AA
sw      x22, 16(x10)       # SPM[16] = 0x7AA (was 0x89ABCDEF)

# Overwrite SPM[24] doubleword
addi    x23, x0, 0x55
sd      x23, 24(x10)       # SPM[24] = 0x55

# Overwrite SPM[32] byte
addi    x24, x0, 0x7F
sb      x24, 32(x10)       # SPM[32] = 0x7F (single byte)
```

```
# Read back to verify before DMA sends it out
ld      x25, 0(x10)         # should be 0x42
lw      x26, 16(x10)        # should be 0x7AA
lh      x27, 8(x10)         # should be 0xFFFF (-1 signed)

# ---------------- DMA: SPM -> Main Memory (back) ----------------
sd      x10, 0(x11)         # DMA[SRC] = SPM_BASE
sd      x13, 8(x11)         # DMA[DST] = main memory 0x0
sd      x12, 16(x11)        # DMA[SIZE] = 68 bytes
addi    x15, x0, 1          # START DMA
sd      x15, 24(x11)        # DMA[CTRL] = START
ld      x16, 32(x11)         # Load DMA status

# ---------------- End of program ----------------
jal     x0, 0               # Infinite loop / stop
```

A.1.5 Hello World Test

Basic "Hello World" program used as a first functional sanity check, provided both in C and in assembly.

C

```c
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

RISC-V Assembly

```
.section .rodata
msg:
    .ascii "Hello World!\n"

.section .text
.globl _start

_start:
```

```
    # a0 = 1 (stdout)
    addi    a0, x0, 1

    # a1 = address of msg
    auipc   a1, %pcrel_hi(msg)
    addi    a1, a1, %pcrel_lo(msg)

    # a2 = 13 (length)
    addi    a2, x0, 13

    # a7 = 64 (write syscall)
    addi    a7, x0, 64

    ecall

end:
    jal     x0, 0
```

A.2 Benchmarking Codes

A.2.1 MNIST Training in Python

Reference Python implementation of the MNIST inference model used as the baseline for benchmarking.

Python

```
import numpy as np
import matplotlib.pyplot as plt
import time
import random

# -- Load data
    ----------------------------------------------------------------
with open("mnist_train.csv", 'r') as f:
    train_list = f.readlines()

with open("mnist_test.csv", 'r') as f:
    test_list = f.readlines()

print(f"Training samples : {len(train_list)}")
print(f"Test samples     : {len(test_list)}")
```

```
# -- Neural Network
    ------------------------------------------------------------
class DNN:
    def __init__(self, sizes, epochs, lr=0.001, batch_size=32):
        self.sizes      = sizes
        self.epochs     = epochs
        self.lr         = lr
        self.batch_size = batch_size

        # Adam params
        self.beta1 = 0.9
        self.beta2 = 0.999
        self.eps   = 1e-8
        self.t     = 0

        i, h1, h2, o = sizes

        # He initialisation (correct for ReLU)
        self.params = {
            'W1': np.random.randn(h1, i)  * np.sqrt(2. / i),
            'b1': np.zeros((h1, 1)),
            'W2': np.random.randn(h2, h1) * np.sqrt(2. / h1),
            'b2': np.zeros((h2, 1)),
            'W3': np.random.randn(o,  h2) * np.sqrt(2. / h2),
            'b3': np.zeros((o,  1)),
        }

        self.m = {k: np.zeros_like(v) for k, v in self.params.items()}
        self.v = {k: np.zeros_like(v) for k, v in self.params.items()}

    # -- Activations ---------------------------------------------------
    def relu(self, x):
        return np.maximum(0, x)

    def relu_deriv(self, x):
        return (x > 0).astype(float)

    def softmax(self, x):
        x   = x - np.max(x, axis=0, keepdims=True)
        exp = np.exp(x)
```

```
        return exp / np.sum(exp, axis=0, keepdims=True)

    # -- Forward ----------------------------------------------------------
    def forward(self, X):
        p = self.params

        self.Z1 = p['W1'] @ X + p['b1']
        self.A1 = self.relu(self.Z1)

        self.Z2 = p['W2'] @ self.A1 + p['b2']
        self.A2 = self.relu(self.Z2)

        self.Z3 = p['W3'] @ self.A2 + p['b3']
        self.A3 = self.softmax(self.Z3)

        return self.A3

    # -- Backward ---------------------------------------------------------
    def backward(self, X, Y):
        p = self.params
        m = X.shape[1]

        dZ3 = self.A3 - Y
        dW3 = (dZ3 @ self.A2.T) / m
        db3 = np.sum(dZ3, axis=1, keepdims=True) / m

        dA2 = p['W3'].T @ dZ3
        dZ2 = dA2 * self.relu_deriv(self.Z2)
        dW2 = (dZ2 @ self.A1.T) / m
        db2 = np.sum(dZ2, axis=1, keepdims=True) / m

        dA1 = p['W2'].T @ dZ2
        dZ1 = dA1 * self.relu_deriv(self.Z1)
        dW1 = (dZ1 @ X.T) / m
        db1 = np.sum(dZ1, axis=1, keepdims=True) / m

        return {
            'W1': dW1, 'b1': db1,
            'W2': dW2, 'b2': db2,
            'W3': dW3, 'b3': db3,
        }
```

```
    # -- Adam update ----------------------------------------------------
    def update(self, grads):
        self.t += 1
        lr_t = self.lr * np.sqrt(1 - self.beta2**self.t) / (1 - self.beta1**
    self.t)

        for k in self.params:
            self.m[k] = self.beta1 * self.m[k] + (1 - self.beta1) * grads[k]
            self.v[k] = self.beta2 * self.v[k] + (1 - self.beta2) * (grads[k
    ]**2)
            self.params[k] -= lr_t * self.m[k] / (np.sqrt(self.v[k]) + self.
    eps)

    # -- Accuracy -------------------------------------------------------
    def accuracy(self, data):
        correct = 0
        for row in data:
            vals   = row.split(',')
            x      = np.asarray(vals[1:], dtype=float) / 255.0
            x      = x.reshape(-1, 1)
            y_true = int(vals[0])
            y_pred = np.argmax(self.forward(x))
            if y_pred == y_true:
                correct += 1
        return correct / len(data)

    # -- Save weights ---------------------------------------------------
    def save_weights(self, path="mnist_weights.npz"):
        np.savez(path, **self.params)
        print(f"\nWeights saved  -> {path}")
        print(f"  Keys        : {list(self.params.keys())}")
        print(f"  Architecture: {self.sizes}")

    # -- Load weights (skip retraining) ---------------------------------
    def load_weights(self, path="mnist_weights.npz"):
        data = np.load(path)
        for k in self.params:
            self.params[k] = data[k]
        print(f"Weights loaded <- {path}")

    # -- Training -------------------------------------------------------
    def train(self, train_list, test_list):
```

```
        accs  = []
        start = time.time()

        for epoch in range(self.epochs):
            random.shuffle(train_list)

            for i in range(0, len(train_list), self.batch_size):
                batch = train_list[i:i + self.batch_size]
                X, Y  = [], []

                for row in batch:
                    vals = row.split(',')
                    x    = np.asarray(vals[1:], dtype=float) / 255.0
                    X.append(x)
                    y    = np.zeros(self.sizes[-1])
                    y[int(vals[0])] = 1.0
                    Y.append(y)

                X = np.array(X).T          # (784, batch_size)
                Y = np.array(Y).T          # (10,  batch_size)

                self.forward(X)
                grads = self.backward(X, Y)
                self.update(grads)

            acc = self.accuracy(test_list)
            accs.append(acc * 100)
            print(f"Epoch {epoch+1:2d}/{self.epochs} | "
                  f"Accuracy: {acc*100:.2f}% | "
                  f"Time: {time.time()-start:.1f}s")

        # -- Save weights after training ---------------------------------
        self.save_weights("mnist_weights.npz")

        # -- Plot --------------------------------------------------------
        plt.figure(figsize=(8, 4))
        plt.plot(range(1, self.epochs + 1), accs, marker='o', color='
    steelblue')
        plt.xlabel("Epoch")
        plt.ylabel("Test Accuracy (%)")
        plt.title("MNIST DNN -- Test Accuracy")
        plt.grid(True)
```

```
        plt.tight_layout()
        plt.savefig("accuracy_curve.png", dpi=120)
        plt.show()
        print("Accuracy curve saved -> accuracy_curve.png")

        return accs


# -- Run
    ------------------------------------------------------------------------
dnn = DNN(
    sizes=[784, 128, 64, 10],
    epochs=15,                  # 20 is enough -- Adam converges by ~epoch
    15
    lr=0.001,
    batch_size=32,
)

dnn.train(train_list, test_list)

# -- To reload weights later without retraining
    -------------------------------
# dnn2 = DNN(sizes=[784, 128, 64, 10], epochs=0, lr=0.001)
# dnn2.load_weights("mnist_weights.npz")
# print(f"Reloaded accuracy: {dnn2.accuracy(test_list)*100:.2f}%")
```

A.2.2 MNIST Weight Quantization

Code used to quantize the trained MNIST model weights prior to deployment on the target architecture.

Python

```
import numpy as np

# Load weights
data = np.load("mnist_weights.npz")

W1 = data['W1']            # (128, 784)
b1 = data['b1'].flatten()  # (128,)
W2 = data['W2']            # (64,  128)
b2 = data['b2'].flatten()  # (64,)
```

```
W3 = data['W3']            # (10,  64)
b3 = data['b3'].flatten()  # (10,)

print("Loaded weights:")
print(f"  W1: {W1.shape}  max_abs={np.max(np.abs(W1)):.4f}")
print(f"  W2: {W2.shape}  max_abs={np.max(np.abs(W2)):.4f}")
print(f"  W3: {W3.shape}  max_abs={np.max(np.abs(W3)):.4f}")
print(f"  b1: max_abs={np.max(np.abs(b1)):.4f}")
print(f"  b2: max_abs={np.max(np.abs(b2)):.4f}")
print(f"  b3: max_abs={np.max(np.abs(b3)):.4f}")


#==================================================
#  MEMORY LAYOUT -- everything stored as int8
#
#  Weights (int8):   quantized with shift per layer
#  Biases  (int8):   quantized in Q7 (scale = 128) -- added AFTER the shift
#  Activations:      int8 (temporary, just a stack array in C)
#  Accumulator:      int32 REGISTER only -- never stored, costs zero memory
#
#  Flow inside one neuron in C:
#
#    int32_t acc = 0;
#    for (i = 0; i < N; i++)
#        acc += (int32_t)W[n*N+i] * (int32_t)x[i];  // int32 register
#    acc >>= SHIFT_W;              // back to Q7 range
#    acc += (int32_t)b[n];         // add int8 bias (cast to int32 for add)
#    if (acc < 0)   acc = 0;       // ReLU
#    if (acc > 127) acc = 127;     // clip
#    output[n] = (int8_t)acc;      // store as int8
#
#  Total memory stored = weights + biases, ALL int8. Nothing else.
# ================================================

INPUT_SHIFT = 7   # pixels: x = pixel >> 1, range [0,127], scale = 2^7

def get_shift(arr):
    """Largest shift N such that max_abs * 2^N <= 127 (fits in int8)."""
    max_abs = np.max(np.abs(arr))
    return int(np.floor(np.log2(127.0 / max_abs)))

def quantize_int8(arr, shift):
```

```
    """Quantize to int8 using power-of-2 scale = 2^shift."""
    scale = 2 ** shift
    return np.clip(np.round(arr * scale), -127, 127).astype(np.int8)

# Weight shifts -- per layer
shift_W1 = get_shift(W1)
shift_W2 = get_shift(W2)
shift_W3 = get_shift(W3)

# Bias shift -- same as Q7 (= INPUT_SHIFT = 7) because bias is added
# AFTER acc >>= SHIFT_W, so the accumulator is already back in Q7 scale.
# Therefore bias_int8 = round(b_float * 128), clipped to [-127, 127].
BIAS_SHIFT = INPUT_SHIFT  # = 7

print(f"\nShifts:")
print(f"  SHIFT_W1 = {shift_W1}  (weight scale = 2^{shift_W1} = {2**shift_W1
    })")
print(f"  SHIFT_W2 = {shift_W2}  (weight scale = 2^{shift_W2} = {2**shift_W2
    })")
print(f"  SHIFT_W3 = {shift_W3}  (weight scale = 2^{shift_W3} = {2**shift_W3
    })")
print(f"  BIAS_SHIFT = {BIAS_SHIFT}  (bias scale = 2^{BIAS_SHIFT} = {2**
    BIAS_SHIFT})")

# Quantize everything to int8
W1q = quantize_int8(W1, shift_W1)
W2q = quantize_int8(W2, shift_W2)
W3q = quantize_int8(W3, shift_W3)

b1q = quantize_int8(b1, BIAS_SHIFT)
b2q = quantize_int8(b2, BIAS_SHIFT)
b3q = quantize_int8(b3, BIAS_SHIFT)

print(f"\nAll int8 ranges:")
print(f"  W1q : [{W1q.min():4d}, {W1q.max():4d}]")
print(f"  W2q : [{W2q.min():4d}, {W2q.max():4d}]")
print(f"  W3q : [{W3q.min():4d}, {W3q.max():4d}]")
print(f"  b1q : [{b1q.min():4d}, {b1q.max():4d}]")
print(f"  b2q : [{b2q.min():4d}, {b2q.max():4d}]")
print(f"  b3q : [{b3q.min():4d}, {b3q.max():4d}]")

# Warn if any bias was clipped (means float bias exceeded ~0.99)
```

```
for name, bq, bf in [("b1", b1q, b1), ("b2", b2q, b2), ("b3", b3q, b3)]:
    clipped = np.sum(np.abs(bf) * (2**BIAS_SHIFT) > 127)
    if clipped > 0:
        print(f"  WARNING: {clipped} values in {name} were clipped -- "
              f"max_abs={np.max(np.abs(bf)):.4f} exceeds int8 range at scale
     128")


# Verify: simulate exact C behaviour in Python
print("\nVerifying fixed-point accuracy on first 1000 test samples...")

with open("mnist_test.csv", 'r') as f:
    test_list = f.readlines()

correct = 0
for row in test_list[:1000]:
    vals   = row.split(',')
    label  = int(vals[0])
    pixels = np.array(vals[1:], dtype=np.int32)

    x = pixels >> 1                    # int8 range [0, 127]

    # Layer 1
    a1 = np.zeros(128, dtype=np.int8)
    for n in range(128):
        acc  = int(np.dot(W1q[n].astype(np.int32), x))   # int32 accumulator
        acc >>= shift_W1                                   # back to Q7
        acc += int(b1q[n])                                 # add int8 bias
        if acc < 0:   acc = 0                              # ReLU
        if acc > 127: acc = 127                            # clip
        a1[n] = np.int8(acc)

    # Layer 2
    a2 = np.zeros(64, dtype=np.int8)
    for n in range(64):
        acc  = int(np.dot(W2q[n].astype(np.int32), a1.astype(np.int32)))
        acc >>= shift_W2
        acc += int(b2q[n])
        if acc < 0:   acc = 0
        if acc > 127: acc = 127
        a2[n] = np.int8(acc)
```

```
    # Layer 3 -- argmax only, no softmax needed
    scores = np.zeros(10, dtype=np.int32)
    for n in range(10):
        acc  = int(np.dot(W3q[n].astype(np.int32), a2.astype(np.int32)))
        acc >>= shift_W3
        acc += int(b3q[n])
        scores[n] = acc

    if int(np.argmax(scores)) == label:
        correct += 1

print(f"  Fixed-point accuracy : {correct / 10:.1f}%  (1000 samples)")


# Memory summary
total = W1q.size + b1q.size + W2q.size + b2q.size + W3q.size + b3q.size
print(f"\nMemory footprint (everything int8, 1 byte each):")
print(f"  W1 : {W1q.size:7,} bytes")
print(f"  b1 : {b1q.size:7,} bytes")
print(f"  W2 : {W2q.size:7,} bytes")
print(f"  b2 : {b2q.size:7,} bytes")
print(f"  W3 : {W3q.size:7,} bytes")
print(f"  b3 : {b3q.size:7,} bytes")
print(f"  TOTAL : {total:,} bytes = {total/1024:.1f} KB")


# Write weights.h
def write_int8_array(f, name, arr, cols=16):
    flat = arr.flatten()
    f.write(f"static const int8_t {name}[] = {{\n")
    for i in range(0, len(flat), cols):
        line = ", ".join(f"{int(v):4d}" for v in flat[i:i+cols])
        f.write(f"    {line},\n")
    f.write("};\n\n")

with open("weights.h", "w", encoding='utf-8') as f:
    f.write("// weights.h - all int8, auto-generated by export_weights.py\n"
    )
    f.write("// Zero floats. Accumulator is int32 register only (not stored)
    .\n")
    f.write("// Bias is added AFTER acc >>= SHIFT_WN.\n\n")
    f.write("#ifndef WEIGHTS_H\n")
```

```
    f.write("#define WEIGHTS_H\n\n")
    f.write("#include <stdint.h>\n\n")

    f.write("// Network dimensions\n")
    f.write("#define INPUT_SIZE    784\n")
    f.write("#define HIDDEN1_SIZE  128\n")
    f.write("#define HIDDEN2_SIZE   64\n")
    f.write("#define OUTPUT_SIZE    10\n\n")

    f.write("// Fixed-point shifts (used as >> in C after MAC loop)\n")
    f.write(f"#define SHIFT_W1  {shift_W1}\n")
    f.write(f"#define SHIFT_W2  {shift_W2}\n")
    f.write(f"#define SHIFT_W3  {shift_W3}\n\n")

    f.write("// Weights - int8, row-major\n")
    f.write("// Access: W1_data[neuron * INPUT_SIZE + input_index]\n")
    write_int8_array(f, "W1_data", W1q)
    write_int8_array(f, "W2_data", W2q)
    write_int8_array(f, "W3_data", W3q)

    f.write("// Biases - int8, scale = 2^7 = 128\n")
    f.write("// Added to acc AFTER acc >>= SHIFT_WN\n")
    write_int8_array(f, "b1_data", b1q)
    write_int8_array(f, "b2_data", b2q)
    write_int8_array(f, "b3_data", b3q)

    f.write("#endif // WEIGHTS_H\n")

print(f"\nweights.h written (UTF-8, all int8).")
```

A.2.3 MNIST Inference in C

C implementation of the MNIST inference model, used for benchmarking on the target processor.

C

```
/*
 * inference.c
 * Single-image MNIST inference.
 * Paste your image pixels into the input[] array below.
 * Compile test: gcc -O2 -o inference inference.c && ./inference
```

```
 */

#include <stdint.h>
#include <stdio.h>
#include "weights.h"

/* -- INPUT IMAGE
    ------------------------------------------------------------
 * 784 values, range 0-127  (original pixel >> 1)
 * Use extract_image.py to generate this array from any mnist_test.csv row.
 * Paste the output here and recompile (or re-assemble for your ISA).
 */
static const int8_t input[784] = {
    /* PASTE YOUR IMAGE VALUES HERE -- example: */
    0, 0, 0, 0, 0, 0, 0, 0  /* replace with 784 real values */
};

/* -- Working buffers
    ------------------------------------------------------------
 * a1  : 128 bytes  output of layer 1
 * a2  :  64 bytes  output of layer 2
 * out :  10 int32  raw scores from layer 3 (not stored after argmax)
 */
static int8_t  a1[HIDDEN1_SIZE];
static int8_t  a2[HIDDEN2_SIZE];
static int32_t out[OUTPUT_SIZE];

int main(void)
{
    int32_t acc;
    int n, i;

    /* -- Layer 1: 784 inputs -> 128 neurons ------------------------------
     */
    for (n = 0; n < HIDDEN1_SIZE; n++) {
        acc = 0;
        for (i = 0; i < INPUT_SIZE; i++) {
            acc += (int32_t)W1_data[n * INPUT_SIZE + i]
                 * (int32_t)input[i];
        }
        acc >>= SHIFT_W1;           /* rescale: right-shift, no float   */
        acc  += (int32_t)b1_data[n];/* add bias after shift             */
```

```
    if (acc < 0)   acc = 0;     /* ReLU                              */
    if (acc > 127) acc = 127;   /* clip to int8 range                */
    a1[n] = (int8_t)acc;
}

/* -- Layer 2: 128 inputs -> 64 neurons --------------------------------
 */
for (n = 0; n < HIDDEN2_SIZE; n++) {
    acc = 0;
    for (i = 0; i < HIDDEN1_SIZE; i++) {
        acc += (int32_t)W2_data[n * HIDDEN1_SIZE + i]
             * (int32_t)a1[i];
    }
    acc >>= SHIFT_W2;
    acc  += (int32_t)b2_data[n];
    if (acc < 0)   acc = 0;
    if (acc > 127) acc = 127;
    a2[n] = (int8_t)acc;
}

/* -- Layer 3: 64 inputs -> 10 scores ----------------------------------
 */
for (n = 0; n < OUTPUT_SIZE; n++) {
    acc = 0;
    for (i = 0; i < HIDDEN2_SIZE; i++) {
        acc += (int32_t)W3_data[n * HIDDEN2_SIZE + i]
             * (int32_t)a2[i];
    }
    acc   >>= SHIFT_W3;
    acc    += (int32_t)b3_data[n];
    out[n]  = acc;
}

/* -- Argmax: highest score = predicted digit
--------------------------- */
int     pred  = 0;
int32_t best  = out[0];
for (n = 1; n < OUTPUT_SIZE; n++) {
    if (out[n] > best) {
        best = out[n];
        pred = n;
    }
```

```
    }

    printf("Predicted digit : %d\n", pred);
    printf("Scores: ");
    for (n = 0; n < OUTPUT_SIZE; n++)
        printf("[%d]=%d ", n, (int)out[n]);
    printf("\n");

    return 0;
}
```

A.2.4 MNIST Cache Behaviour and Direct Assembly Implementation

Hand-written assembly implementation of MNIST inference used to study cache behaviour and to obtain a direct, low-level performance benchmark.

RISC-V Assembly

```
# ---------------- Setup addresses ----------------
# ---------------- Local buffer addresses ----------------
addi    x2, x0, 0x280       # x2 = X   address (byte 640,  word 80)
addi    x3, x0, 0x5C0       # x3 = a1  address (byte 1472, word 184)
addi    x4, x0, 0x640       # x4 = a2  address (byte 1600, word 200)
addi    x5, x0, 0x680       # x5 = out address (byte 1664, word 208)

# ---------------- Weight/bias addresses ----------------
lui     x6, 0x1                             # x6 = w1 address (0x000800)
addi    x6, x6, -0x800

lui     x7, 0x1B                            # x7 = b1 address (0x01B280)
addi    x7, x7, 0x280

lui     x8, 0x19                            # x8 = w2 address (0x019000,
    exact)

lui     x9, 0x1B                            # x9 = b2 address (0x01B300)
addi    x9, x9, 0x300

lui     x10, 0x1B                           # x10 = w3 address (0x01B000,
    exact)

lui     x11, 0x1B                           # x11 = b3 address (0x01B340)
addi    x11, x11, 0x340

# ---------------- Setup constants ----------------
#   input size = 784
#   hidden1 size = 128
#   hidden2 size = 64
#   output size = 10
#   neurons per tile = 8
# tile  size = 6272
#   number of tiles = 16
# shift_w1 = 6
# shift_w2 = 6
# shift_w3 = 7
```

```
# ------------------  Start  --------------------
# ------------------ layer 1 --------------------
# --------------- layer1 engine--------------------
#----------- compute 128 neuron ------------
    addi    x20, x0, 0          # x20 = neuron index
    addi    x21, x0, 128        # x21 = number of neurons
    add     x22, x0, x6         # x22 = weight ptr in memory (starts at w1
    base), advances +784 per neuron)
    add     x23, x0, x3         # x23 = output ptr in a1[]
neuron_loop_l1:
    beq  x20, x21, layer1_done

#-------------- mac loop ----------------
    addi    x24, x0, 0          # x24 = acc = 0
    add     x25, x0, x2         # x25 = X input pointer
    add     x26, x0, x22        # x26 = weight ptr
    addi    x27, x0, 784        # x27 = counter = 784  (input size)

mac_l1:
    beq x27, x0, mac_l1_done

    lb   x28, 0(x26)            # w  = W1_tile[n][i]  int8
    lbu  x29, 0(x25)            # x  = input[i]       uint8
  macb  x24, x28, x29          # acc = acc + w * x (signed byte operation)
    addi x25, x25, 1            # x = x + 1  input increment
    addi x26, x26, 1            # w = w + 1  weight increment
    addi x27, x27, -1           # count = count - 1
    jal x0, mac_l1

mac_l1_done:
    srai x24, x24, 6           # acc >>= SHIFT_W1  scale back to int8

# ------------------ add b1 after the mul -------------------
    add     x25, x0, x7        # x25 = b1 base
    add     x25, x25, x20       # x25 = &b1[n]

    lb      x26, 0(x25)         # x26 = bias
    add     x24, x24, x26       # acc += bias

# --------------------- ReLU ----------------------
    bge   x24, x0, clip_l1
```

```
    addi  x24, x0, 0
clip_l1:
    addi x25, x0, 127
    blt  x24, x25, store_l1
    addi x24, x0, 127
# -------------------- store ----------------------
store_l1:
    sb   x24, 0(x23)           # a1[global_n] = (int8)acc
# --------------- next neuron setup ---------------
    addi x22, x22, 784         # next neuron weights(+784)
    addi x23, x23, 1           # output ptr in a1[] = x23 + 1  (next output
    byte)
    addi x20, x20, 1           # neuron++
    jal x0, neuron_loop_l1
# -------------------- done -----------------------
layer1_done:

# ------------------- layer 2 ---------------------
# ---------------- layer2 engine ------------------
#----------- compute 64 neuron ------------
    addi    x20, x0, 0          # x20 = neuron index
    addi    x21, x0, 64        # x21 = total neurons

    add     x22, x0, x8         # x22 = weight ptr in memory (starts at w2
    base), advances +128 per neuron)
    add     x23, x0, x4         # x23 = output pointer a2[]

neuron_loop_l2:
    beq     x20, x21, layer2_done


# ------------------ MAC loop --------------------
    addi    x24, x0, 0          # x24 = accumulator = 0

    add     x25, x0, x3         # x25 = a1 input pointer
    add     x26, x0, x22        # x26 = weight pointer

    addi    x27, x0, 128        # x27 = counter = input size

mac_l2:
    beq     x27, x0, mac_l2_done
```

```
    lb      x28, 0(x26)         # w = W2[n][i]
    lbu     x29, 0(x25)         # a1 = input[i]

    macb     x24, x28, x29       # acc += w2 * a1

    addi    x25, x25, 1         # next input
    addi    x26, x26, 1         # next weight
    addi    x27, x27, -1        # counter--

    jal     x0, mac_l2


mac_l2_done:
    srai    x24, x24, 6         # scale back


# ------------------- add b2 after the mul --------------------
    add     x25, x0, x9        # x25 = b2 base
    add     x25, x25, x20       # x25 = &b2[n]

    lb      x26, 0(x25)         # x26 = bias
    add     x24, x24, x26       # acc += bias


# ---------------------- ReLU ----------------------
    bge     x24, x0, clip_l2
    addi    x24, x0, 0

clip_l2:
    addi    x25, x0, 127
    blt     x24, x25, store_l2

    addi    x24, x0, 127


# -------------------- store ---------------------
store_l2:
    sb      x24, 0(x23)         # a2[n] = acc


# --------------- next neuron setup ---------------
    addi    x22, x22, 128       # next neuron weights
```

```
    addi    x23, x23, 1         # next output byte
    addi    x20, x20, 1         # neuron++

    jal     x0, neuron_loop_l2


# -------------------- done ----------------------
layer2_done:


# ------------------- layer 3 --------------------
# ---------------- layer3 engine ------------------
#----------- compute 10 neuron ------------
    addi    x20, x0, 0          # x20 = neuron index
    addi    x21, x0, 10        # x21 = total neurons

    add     x22, x0, x10         # x22 = weight ptr in memory (starts at w3
    base), advances +64 per neuron)
    add     x23, x0, x5         # x23 = output pointer out[]

neuron_loop_l3:
    beq     x20, x21, layer3_done


# ------------------- MAC loop --------------------
    addi    x24, x0, 0          # x24 = accumulator = 0

    add     x25, x0, x4         # x25 = a2 input pointer
    add     x26, x0, x22        # x26 = weight pointer

    addi    x27, x0, 64        # x27 = counter = input size

mac_l3:
    beq     x27, x0, mac_l3_done

    lb      x28, 0(x26)         # w = W3[n][i]
    lbu     x29, 0(x25)         # a2 = input[i]

    macb     x24, x28, x29       # acc += w3 * a2

    addi    x25, x25, 1         # next input
    addi    x26, x26, 1         # next weight
```

```
    addi    x27, x27, -1        # counter--

    jal     x0, mac_l3


mac_l3_done:
    srai    x24, x24, 7         # scale back


# ------------------- add bias --------------------
    add     x25, x0, x11        # x25 = b3 base
    add     x25, x25, x20       # x25 = &b3[n]

    lb      x26, 0(x25)         # x26 = bias
    add     x24, x24, x26       # acc += bias

# -------------------- store ----------------------
store_l3:
    sb      x24, 0(x23)         # out[n] = acc


# --------------- next neuron setup ---------------
    addi    x22, x22, 64       # next neuron weights
    addi    x23, x23, 1         # next output byte
    addi    x20, x20, 1         # neuron++

    jal     x0, neuron_loop_l3


# -------------------- done -----------------------
layer3_done:

# ---------------- ARGMAX (prediction) ---------------
  addi x17, x0, 0             # pred = 0
  lb   x18, 0(x5)             # best = out[0]

  addi x19, x0, 1             # n = 1
  add  x20, x0, x5            # x20 = pointer to out[]

  addi x21, x0, 10            # OUTPUT_SIZE = 10

argmax_loop:
```

```
    beq  x19, x21, argmax_done

  add  x22, x20, x19          # address = out + n
    lb   x23, 0(x22)            # x23 = out[n]

  bge  x18, x23, skip_update

    add  x18, x23, x0           # best = out[n]
    add  x17, x19, x0             # pred = n

skip_update:
    addi x19, x19, 1
    jal  x0, argmax_loop


argmax_done:
# ---------------- End of program ----------------
jal     x0, 0
```

A.2.5 MNIST Hybrid System Assembly Implementation

Hand-written assembly implementation of MNIST inference for the hybrid system, where the instruction and data caches are combined with scratchpad memory (SPM) and DMA-based weight staging to reduce data-memory traffic during inference.

RISC-V Assembly

```
# ---------------- Setup addresses ----------------
lui     x2, 0x80000         # x2  = SPM_BASE upper
lui     x3, 0xFFFF0         # x3  = DMA_CTRL_BASE upper
# ---------------- Local buffer addresses ----------------
addi    x4, x0, 0x280       # x4 = X   address (byte 640,  word 80)
addi    x6, x0, 0x5C0       # x6 = a1  address (byte 1472, word 184)
addi    x7, x0, 0x640       # x7 = a2  address (byte 1600, word 200)
addi    x8, x0, 0x680       # x8 = out address (byte 1664, word 208)

# ---------------- Weight/bias addresses ----------------
lui     x11, 0x1                              # x11 = w1 address (0x000800)
addi    x11, x11, -0x800

lui     x12, 0x1B                             # x12 = b1 address (0x01B280)
addi    x12, x12, 0x280
```

```
lui     x13, 0x19                           # x13 = w2 address (0x019000,
    exact)

lui     x14, 0x1B                           # x14 = b2 address (0x01B300)
addi    x14, x14, 0x300

lui     x15, 0x1B                          # x15 = w3 address (0x01B000,
    exact)

lui     x16, 0x1B                          # x16 = b3 address (0x01B340)
addi    x16, x16, 0x340

addi    x9, x0, 0x3        # x9 = CTRL START = 1 & Direction = 1
lui     x10, 0x80002
addi    x10, x10, -1192      # x10  = b address in SPM (0x80001B58)

# ---------------- Setup constants ----------------
#   input size = 784
#   hidden1 size = 128
#   hidden2 size = 64
#   output size = 10
#   neurons per tile = 8
# tile  size = 6272
#   number of tiles = 16
# shift_w1 = 6
# shift_w2 = 6
# shift_w3 = 7

# ---------------- Setup constants ----------------
#   input size = 784
#   hidden1 size = 128
#   hidden2 size = 64
#   output size = 10
#   neurons per tile = 8
# tile  size = 6272
#   number of tiles = 16
# shift_w1 = 6
# shift_w2 = 6
# shift_w3 = 7

# -------------------  Start  ---------------------
```

```
# ------------------- layer 1 ---------------------
# ------------------- load b1 ---------------------
    addi    x18, x0, 128        # x18 = hidden1 size
    sd      x12, 0(x3)          # DMA[SRC] = b1 external address
    sd      x10, 8(x3)          # DMA[DST] = b address in SPM
    sd      x18, 16(x3)         # DMA[SIZE] = hidden1 size = 128 byte
    sd      x9, 24(x3)         # DMA[CTRL] = START

# --------------- layer1 engine---------------------
    addi    x17, x0, 16         # x17 = number of tiles = 16
    addi    x18, x0, 0          # x18 = tile index from 0 to 15
    add     x19, x0, x11        # x19 = w1 tile address
layer1_tile_loop:
    beq  x18, x17, layer1_done

    #------------- W1 tile load --------------
    lui     x20, 0x2            # x20 = 0x2000 = 8192
  addi    x20, x20, -1920     # 8192 - 1920 = 6272
    sd      x19, 0(x3)          # DMA[SRC] = w1 external address
    sd      x2, 8(x3)           # DMA[DST] = SPM
    sd      x20, 16(x3)         # DMA[SIZE] = tile size = 6272 bytes
    sd      x9, 24(x3)          # DMA[CTRL] = START

    #----------- compute 8 neuron ------------
    addi    x20, x0, 0          # x20 = neuron index within the tile
    addi    x21, x0, 8          # x21 = number of neurons per tile
    add     x22, x0, x2         # x22 = weight ptr in SPM (starts at
    SPM_W_TILE (base), advances +784 per neuron)
    add     x23, x0, x6         # x23 = output ptr in a1[]
neuron_loop_l1:
    beq  x20, x21, tile_l1_done

    #-------------- mac loop ----------------
    addi    x24, x0, 0          # x24 = acc = 0
    add     x25, x0, x4         # x25 = input pointer
    add     x26, x0, x22        # x26 = weight ptr
    addi    x27, x0, 784        # x27 = counter = 784  (input size)

mac_l1:
    beq x27, x0, mac_l1_done

    lb   x28, 0(x26)            # w  = W1_tile[n][i]  int8
```

```
    lbu  x29, 0(x25)            # x  = input[i]       uint8
  macb  x24, x28, x29          # acc = acc + w * x
    addi x25, x25, 1            # x = x + 1  input increment
    addi x26, x26, 1            # w = w + 1  weight increment
    addi x27, x27, -1           # count = count - 1
    jal x0, mac_l1

mac_l1_done:
    srai x24, x24, 6           # acc >>= SHIFT_W1  scale back to int8

    #-------- add b after the mul-----------
    add  x25, x0, x10          # x25 = b1 base address in SPM
    slli x26, x18, 3           # x26 = tile * 8
    add  x25, x25, x26         # x25 = SPM_B1 + tile*8
    add  x25, x25, x20         # x25 = SPM_B1 + tile*8 + n  = b1[global_n]
    lb   x26, 0(x25)           # x26 = b1[n]  int8
    add  x24, x24, x26         # a24 = acc = acc + bias

    # ReLU linear rectifier
    bge   x24, x0, clip_l1
    addi  x24, x0, 0
clip_l1:
    addi x25, x0, 127
    bge  x25, x24, store_l1
    addi x24, x0, 127
store_l1:
    sb   x24, 0(x23)           # a1[global_n] = (int8)acc

    addi x22, x22, 784         # x22 = weight ptr in SPM next neuron weights
     in SPM (+784)
    addi x23, x23, 1           # x23 = output ptr in a1[] = x23 + 1  (next
    output byte)
    addi x20, x20, 1           # x20 = neuron index within the teil = x20 +
    1
    jal x0, neuron_loop_l1

tile_l1_done:
    addi x18, x18, 1            # x18 = teil index = tile++
  lui     x19, 0x2
  addi    x19, x19, -1920   # x19 = w1 tile address = previous tile address
    + 6272
    addi x6, x6, 8              # x6 = a1 address = a1 output ptr + 8
```

```
    jal  x0, layer1_tile_loop

layer1_done:

# ------------------- layer 2 --------------------
# ------------------- load w2 --------------------
  lui     x18, 0x2             # x18 = hidden2 size
    sd      x13, 0(x3)          # DMA[SRC] = w2 external address
    sd      x2, 8(x3)          # DMA[DST] = SPM
    sd      x18, 16(x3)         # DMA[SIZE] = hidden1 size = 8192 byte
    sd      x9, 24(x3)         # DMA[CTRL] = START

# ---------------- layer2 engine ------------------
    addi    x20, x0, 0          # x20 = neuron index
    addi    x21, x0, 64        # x21 = total neurons

    add     x22, x0, x2         # x22 = weight pointer in SPM
    add     x23, x0, x7         # x23 = output pointer a2[]

neuron_loop_l2:
    beq     x20, x21, layer2_done


# ------------------- MAC loop --------------------
    addi    x24, x0, 0          # x24 = accumulator = 0

    add     x25, x0, x6         # x25 = input pointer
    add     x26, x0, x22        # x26 = weight pointer

    addi    x27, x0, 128        # x27 = counter = input size

mac_l2:
    beq     x27, x0, mac_l2_done

    lb      x28, 0(x26)         # w = W2[n][i]
    lbu     x29, 0(x25)         # a1 = input[i]

    macb     x24, x28, x29       # acc += w2 * a1

    addi    x25, x25, 1         # next input
    addi    x26, x26, 1         # next weight
    addi    x27, x27, -1        # counter--
```

```
    jal     x0, mac_l2


mac_l2_done:
    srai    x24, x24, 6         # scale back


# ------------------- add bias --------------------
    add     x25, x0, x14        # x25 = b2 base
    add     x25, x25, x20       # x25 = &b2[n]

    lb      x26, 0(x25)         # x26 = bias
    add     x24, x24, x26       # acc += bias


# ---------------------- ReLU -----------------------
    bge     x24, x0, clip_l2
    addi    x24, x0, 0

clip_l2:
    addi    x25, x0, 127
    bge     x25, x24, store_l2

    addi    x24, x0, 127


# -------------------- store ----------------------
store_l2:
    sb      x24, 0(x23)         # a2[n] = acc


# --------------- next neuron setup ---------------
    addi    x22, x22, 128       # next neuron weights
    addi    x23, x23, 1         # next output byte
    addi    x20, x20, 1         # neuron++

    jal     x0, neuron_loop_l2


# -------------------- done -----------------------
layer2_done:
```

```
# ------------------- layer 3 --------------------
# ------------------- load w3 --------------------
  addi    x18, x0, 640        # x18 = hidden3 size
    sd      x15, 0(x3)          # DMA[SRC] = w3 external address
    sd      x2, 8(x3)           # DMA[DST] = SPM
    sd      x18, 16(x3)         # DMA[SIZE] = hidden1 size = 640 byte
    sd      x9, 24(x3)          # DMA[CTRL] = START
# ------------------- load b3 --------------------
  addi    x18, x0, 10         # x18 = 10 size
    sd      x16, 0(x3)          # DMA[SRC] = b3 external address
    sd      x10, 8(x3)          # DMA[DST] = b address in SPM
    sd      x18, 16(x3)         # DMA[SIZE] = 10 byte
    sd      x9, 24(x3)          # DMA[CTRL] = START
# ---------------- layer3 engine ------------------
    addi    x20, x0, 0          # x20 = neuron index
    addi    x21, x0, 10         # x21 = total neurons

    add     x22, x0, x2         # x22 = weight pointer in SPM
    add     x23, x0, x8         # x23 = output pointer out[]

neuron_loop_l3:
    beq     x20, x21, layer3_done


# ------------------- MAC loop --------------------
    addi    x24, x0, 0          # x24 = accumulator = 0

    add     x25, x0, x7         # x25 = input pointer
    add     x26, x0, x22        # x26 = weight pointer

    addi    x27, x0, 64         # x27 = counter = input size

mac_l3:
    beq     x27, x0, mac_l3_done

    lb      x28, 0(x26)         # w = W3[n][i]
    lbu     x29, 0(x25)         # a2 = input[i]

    macb     x24, x28, x29       # acc += w3 * a2
```

```
    addi    x25, x25, 1         # next input
    addi    x26, x26, 1         # next weight
    addi    x27, x27, -1        # counter--

    jal     x0, mac_l3


mac_l3_done:
    srai    x24, x24, 7         # scale back


# ------------------- add bias --------------------
    add     x25, x0, x10        # x25 = b3 base
    add     x25, x25, x20       # x25 = &b3[n]

    lb      x26, 0(x25)         # x26 = bias
    add     x24, x24, x26       # acc += bias

# -------------------- store ----------------------
store_l3:
    sb      x24, 0(x23)         # out[n] = acc


# --------------- next neuron setup ---------------
    addi    x22, x22, 64       # next neuron weights
    addi    x23, x23, 1         # next output byte
    addi    x20, x20, 1         # neuron++

    jal     x0, neuron_loop_l3


# -------------------- done -----------------------
layer3_done:

# ---------------- ARGMAX (prediction) ----------------
  add     x23, x0, x8         # x23 = output pointer out[]

  addi x17, x0, 0             # pred = 0
  lb   x18, 0(x8)             # best = out[0]

  addi x19, x0, 1             # n = 1
  add  x20, x0, x8            # x20 = pointer to out[]
```

```
  addi x21, x0, 10            # OUTPUT_SIZE = 10

argmax_loop:
    beq  x19, x21, argmax_done

  add  x22, x20, x19          # address = out + n
    lb   x23, 0(x22)            # x23 = out[n]

  bge  x18, x23, skip_update

    add  x18, x23, x0           # best = out[n]
    add  x17, x19, x0             # pred = n

skip_update:
    addi x19, x19, 1
    jal  x0, argmax_loop


argmax_done:
# ---------------- End of program ----------------
jal     x0, 0
```

A.2.6 Benchmark Analysis and Plotting Code

Python script used to parse the performance reports from the Direct, Caches, and Hybrid versions, extract the main architectural metrics, and generate comparison plots for cycles, CPI/IPC, cache hit rates, stall breakdowns, bus activity, DMA behaviour, speedup, and the final summary table.

Python

```
import re
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# ---------------------------------------------------------------------
# 1. Paste the raw $display output from each processor version here.
# ---------------------------------------------------------------------
```

```
RAW_DIRECT = """
RAW_CACHES = """
RAW_HYBRID = """

VERSIONS = {"Direct": RAW_DIRECT, "Caches": RAW_CACHES, "Hybrid": RAW_HYBRID
    }
ORDER = ["Direct", "Caches", "Hybrid"]
COLORS = {"Direct": "#D55E00", "Caches": "#0072B2", "Hybrid": "#009E73"}

# ---------------------------------------------------------------------
# 2. Parser
# ---------------------------------------------------------------------

SECTION_RE = re.compile(r"^\[(.+)\]$")
KV_RE = re.compile(r"^(.+?)\s*:\s*([-+0-9.]+)")


def parse_report(raw_text):
    """Parse one console report into {section: {metric: value}}."""
    data = {}
    section = None
    for line in raw_text.splitlines():
        line = line.strip()
        if not line or set(line) == {"="} or line.lower() == "performance
    report":
            continue
        match = SECTION_RE.match(line)
        if match:
            section = match.group(1).strip()
            data[section] = {}
            continue
        match = KV_RE.match(line)
        if match and section is not None:
            label, value_text = match.group(1).strip(), match.group(2)
            data[section][label] = float(value_text) if "." in value_text
    else int(value_text)
    return data


metrics = {name: parse_report(raw) for name, raw in VERSIONS.items()}
```

```
def get(version, section, label, default=np.nan):
    return metrics.get(version, {}).get(section, {}).get(label, default)


os.makedirs("figures", exist_ok=True)
plt.rcParams.update({
    "font.family": "serif",
    "font.size": 11,
    "axes.titlesize": 13,
    "axes.titleweight": "bold",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "grid.linestyle": "--",
    "figure.dpi": 150,
    "savefig.bbox": "tight",
})


def save(fig, name):
    fig.savefig(f"figures/{name}.pdf")
    fig.savefig(f"figures/{name}.png", dpi=300)
    plt.close(fig)


def bar_annotate(ax, bars, fmt="{:.2f}"):
    for bar in bars:
        height = bar.get_height()
        if not np.isnan(height):
            ax.annotate(fmt.format(height),
                        (bar.get_x() + bar.get_width() / 2, height),
                        ha="center", va="bottom", fontsize=9)


# ---------------------------------------------------------------------
# 3. Plot CPU performance and speedup
# ---------------------------------------------------------------------

cycles = [get(v, "CPU", "Total Cycles") for v in ORDER]
cpi = [get(v, "CPU", "CPI") for v in ORDER]
```

```
ipc = [get(v, "CPU", "IPC") for v in ORDER]

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, values, title, fmt in zip(
        axes, [cycles, cpi, ipc],
        ["Total Cycles", "CPI", "IPC"],
        ["{:.0f}", "{:.3f}", "{:.3f}"]):
    bars = ax.bar(ORDER, values, color=[COLORS[v] for v in ORDER])
    ax.set_title(title)
    bar_annotate(ax, bars, fmt)
    if title == "Total Cycles":
        ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x
    /1e3:.0f}k"))
fig.suptitle("CPU Performance: Direct vs Caches vs Hybrid", fontweight="bold
    ")
save(fig, "01_cpu_performance")

base_cycles = get("Direct", "CPU", "Total Cycles")
speedup = [base_cycles / get(v, "CPU", "Total Cycles") for v in ORDER]
fig, ax = plt.subplots(figsize=(6, 4.5))
bars = ax.bar(ORDER, speedup, color=[COLORS[v] for v in ORDER])
ax.axhline(1.0, color="gray", linewidth=1, linestyle=":")
ax.set_ylabel("Speedup (x)")
ax.set_title("Speedup vs Direct-Memory Baseline")
bar_annotate(ax, bars, "{:.2f}x")
save(fig, "02_speedup_vs_baseline")

# ---------------------------------------------------------------------
# 4. Plot cache, stall, bus, and DMA behaviour
# ---------------------------------------------------------------------

fig, ax = plt.subplots(figsize=(6.5, 4.5))
x = np.arange(len(ORDER))
w = 0.35
hit = [get(v, "I-Cache", "Hit Rate") for v in ORDER]
miss = [get(v, "I-Cache", "Miss Rate") for v in ORDER]
b1 = ax.bar(x - w / 2, hit, w, label="Hit Rate", color="#0072B2")
b2 = ax.bar(x + w / 2, miss, w, label="Miss Rate", color="#D55E00")
ax.set_xticks(x)
ax.set_xticklabels(ORDER)
ax.set_ylabel("%")
ax.set_title("I-Cache Hit and Miss Rate")
```

```
ax.legend()
bar_annotate(ax, b1, "{:.1f}%")
bar_annotate(ax, b2, "{:.1f}%")
save(fig, "03_icache_hit_miss")

dcache_versions = [v for v in ORDER if "D-Cache / Memory" in metrics.get(v,
    {})]
if dcache_versions:
    fig, ax = plt.subplots(figsize=(7, 4.5))
    x = np.arange(len(dcache_versions))
    w = 0.25
    read_hit = [get(v, "D-Cache / Memory", "Read Hit Rate") for v in
    dcache_versions]
    write_hit = [get(v, "D-Cache / Memory", "Write Hit Rate") for v in
    dcache_versions]
    total_hit = [get(v, "D-Cache / Memory", "Total Hit Rate") for v in
    dcache_versions]
    b1 = ax.bar(x - w, read_hit, w, label="Read Hit Rate", color="#0072B2")
    b2 = ax.bar(x, write_hit, w, label="Write Hit Rate", color="#009E73")
    b3 = ax.bar(x + w, total_hit, w, label="Total Hit Rate", color="#CC79A7"
    )
    ax.set_xticks(x)
    ax.set_xticklabels(dcache_versions)
    ax.set_ylabel("%")
    ax.set_title("D-Cache Hit Rates")
    ax.legend()
    for bars in (b1, b2, b3):
        bar_annotate(ax, bars, "{:.1f}%")
    save(fig, "04_dcache_hit_rates")

fig, ax = plt.subplots(figsize=(7.5, 5))
bottoms = np.zeros(len(ORDER))
mem_stalls = []
for version in ORDER:
    if "D-Cache / Memory" in metrics.get(version, {}):
        mem_stalls.append(get(version, "D-Cache / Memory", "D-Cache Stalls",
     0))
    else:
        mem_stalls.append(get(version, "Direct Memory / Memory", "Stalls",
    0))
stall_components = {
```

```
    "I-Cache Stalls": [get(v, "I-Cache", "I-Cache Stalls", 0) for v in ORDER
    ],
    "D-Cache / Memory Stalls": mem_stalls,
    "DMA CPU Stall Cycles": [get(v, "DMA", "CPU Stall Cycles", 0) for v in
    ORDER],
    "Branch Misprediction Flush": [
        get(v, "Branch Predictor", "Misprediction NOP Instructions",
            get(v, "Branch Predictor", "Misprediction NOP cycles", 0)) for v
     in ORDER],
}
for (label, values), color in zip(stall_components.items(),
                                  ["#D55E00", "#0072B2", "#E69F00", "#999999
    "]):
    pct = np.array(values, dtype=float) / np.array(cycles, dtype=float) *
    100
    ax.bar(ORDER, pct, bottom=bottoms, label=label, color=color)
    bottoms += pct
ax.set_ylabel("% of Total Cycles")
ax.set_title("Stall-Cycle Breakdown")
ax.legend(loc="upper right", fontsize=8.5)
save(fig, "05_stall_breakdown")

fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
utilization = [get(v, "Bus", "Bus Utilization") for v in ORDER]
throughput = [get(v, "Bus", "Bus Throughput") for v in ORDER]
bars = axes[0].bar(ORDER, utilization, color=[COLORS[v] for v in ORDER])
axes[0].set_title("Bus Utilization")
axes[0].set_ylabel("%")
bar_annotate(axes[0], bars, "{:.1f}%")
bars = axes[1].bar(ORDER, throughput, color=[COLORS[v] for v in ORDER])
axes[1].set_title("Bus Throughput")
axes[1].set_ylabel("bytes/cycle")
bar_annotate(axes[1], bars, "{:.2f}")
fig.suptitle("Bus Behaviour", fontweight="bold")
save(fig, "06_bus_behaviour")

if "DMA" in metrics.get("Hybrid", {}):
    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    dma = metrics["Hybrid"]["DMA"]
    for ax, (label, value) in zip(axes, [
            ("Throughput\n(bytes/cycle)", dma.get("Throughput", np.nan)),
            ("Efficiency", dma.get("Efficiency", np.nan)),
```

```
            ("Utilization (%)", dma.get("Utilization", np.nan))]):
        bars = ax.bar(["Hybrid"], [value], color=COLORS["Hybrid"])
        ax.set_title(label)
        bar_annotate(ax, bars, "{:.2f}")
    fig.suptitle("DMA Engine Behaviour", fontweight="bold")
    save(fig, "07_dma_behaviour")

# ---------------------------------------------------------------------
# 5. Export summary table
# ---------------------------------------------------------------------

rows = []
for version in ORDER:
    rows.append({
        "Version": version,
        "Total Cycles": get(version, "CPU", "Total Cycles"),
        "CPI": get(version, "CPU", "CPI"),
        "IPC": get(version, "CPU", "IPC"),
        "I-Cache Hit %": get(version, "I-Cache", "Hit Rate"),
        "D-Cache/Mem Hit %": get(version, "D-Cache / Memory", "Total Hit
    Rate", np.nan),
        "Branch Acc %": get(version, "Branch Predictor", "Accuracy"),
        "Bus Util %": get(version, "Bus", "Bus Utilization"),
        "Speedup vs Direct": base_cycles / get(version, "CPU", "Total Cycles
    "),
    })

summary = pd.DataFrame(rows).set_index("Version")
summary.to_csv("figures/metrics_summary.csv")
with open("figures/metrics_table.tex", "w") as table_file:
    table_file.write(summary.round(3).to_latex(
        caption="EdgeCore-V MNIST inference performance comparison.",
        label="tab:edgecorev_comparison",
        na_rep="--"))

print("Generated benchmark plots and tables in ./figures/")
print(summary.round(3))
```